

# 第5章 网络层

- 网络层涉及
  - 分组从源端→目的端
  - 处理端到端数据传输的最低层
- 数据链路层涉及
  - 数据帧从导线的一端到另一端

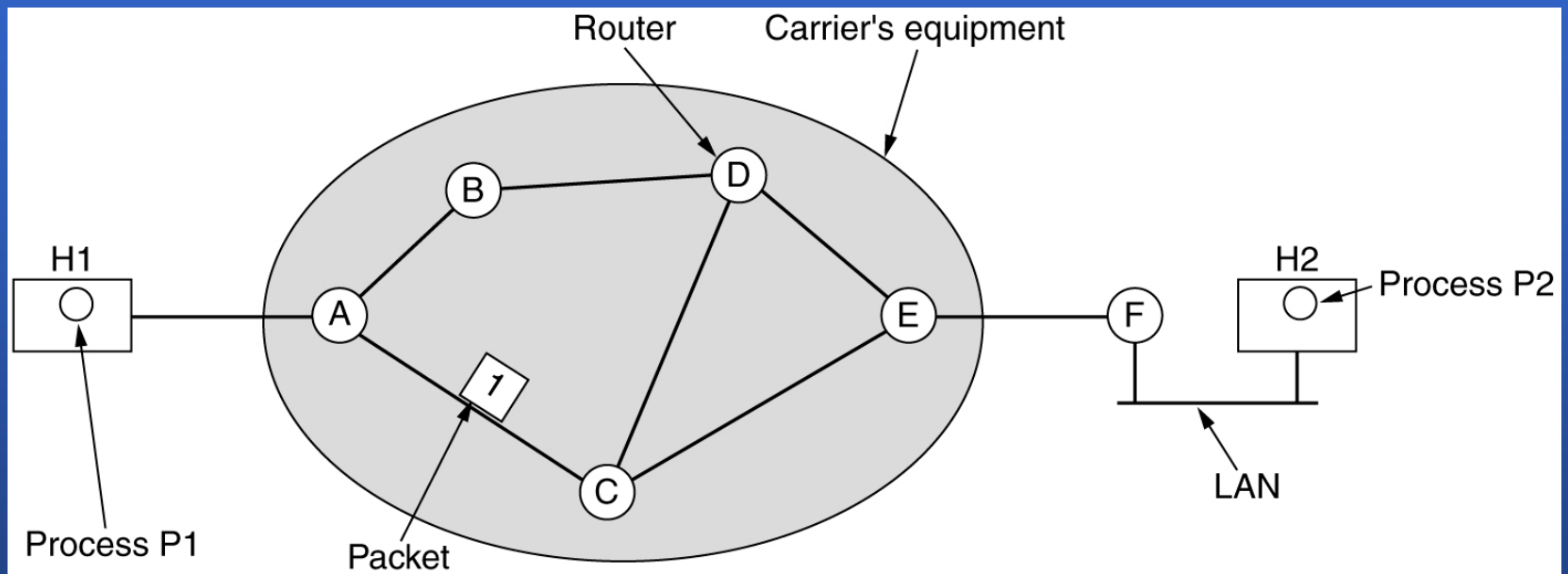


# Network layer design issues: Introduction

- Data link layer v.s. network layer
  - Data link layer: to move frames from one end of a wire to the other.
  - Network layer: to move packets from the source all the way to the destination. It deals with end-to-end transmissions.
- Network layer
  - Must know about the topology of the communication subnet and choose appropriate paths through it.
  - Must choose routes to avoid overloading some of the communication lines and routers while leaving others idle.
  - To deal with internetwork different networks.



## 5.1.1 Stored-and-forward packet switching



- A host with a packet to send transmits it to the nearest router. The packet is received, verified, and stored. Then it is forwarded to the next router. This step can be repeated many times. Finally the packet reaches the destination host.

## 5.1.2 Services provided to the transport layer

- The network layer services have been designed with the following goals in mind.
  - The services should be independent of the router technology
  - The transport layer should be shielded from the number, type, and topology of the routers present.
  - The network addresses made available to the transport layer should use a uniform numbering plan, even cross LANs and WANs.



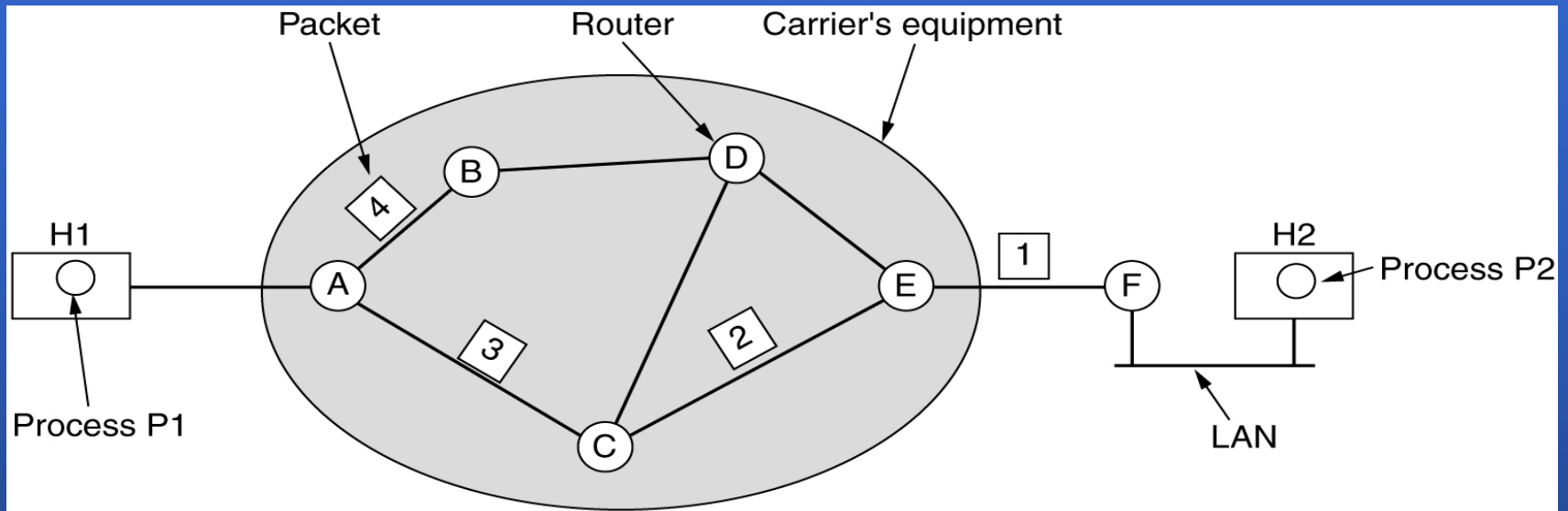


# Services provided to the transport layer

- Two-type of services:
  - **Connection-less services: (→Datagram)**
    - 30 years of experience with the computer network, unreliable internet, hosts doing error control and flow control
  - **Connection-oriented services. (→Virtual Circuit)**
    - 100 years of experience with the worldwide telephone system, quality of service
- Connection-less + connection-oriented services.



# 5.1.3 Implementation of Connectionless Service



A's table

initially

|   |   |
|---|---|
| A | - |
| B | B |
| C | C |
| D | B |
| E | C |
| F | C |

later

|   |   |
|---|---|
| A | - |
| B | B |
| C | C |
| D | B |
| E | B |
| F | B |

C's table

|   |   |
|---|---|
| A | A |
| B | A |
| C | - |
| D | D |
| E | E |
| F | E |

E's table

|   |   |
|---|---|
| A | C |
| B | D |
| C | C |
| D | D |
| E | - |
| F | F |

Dest. Line

Routing within a diagram subnet.

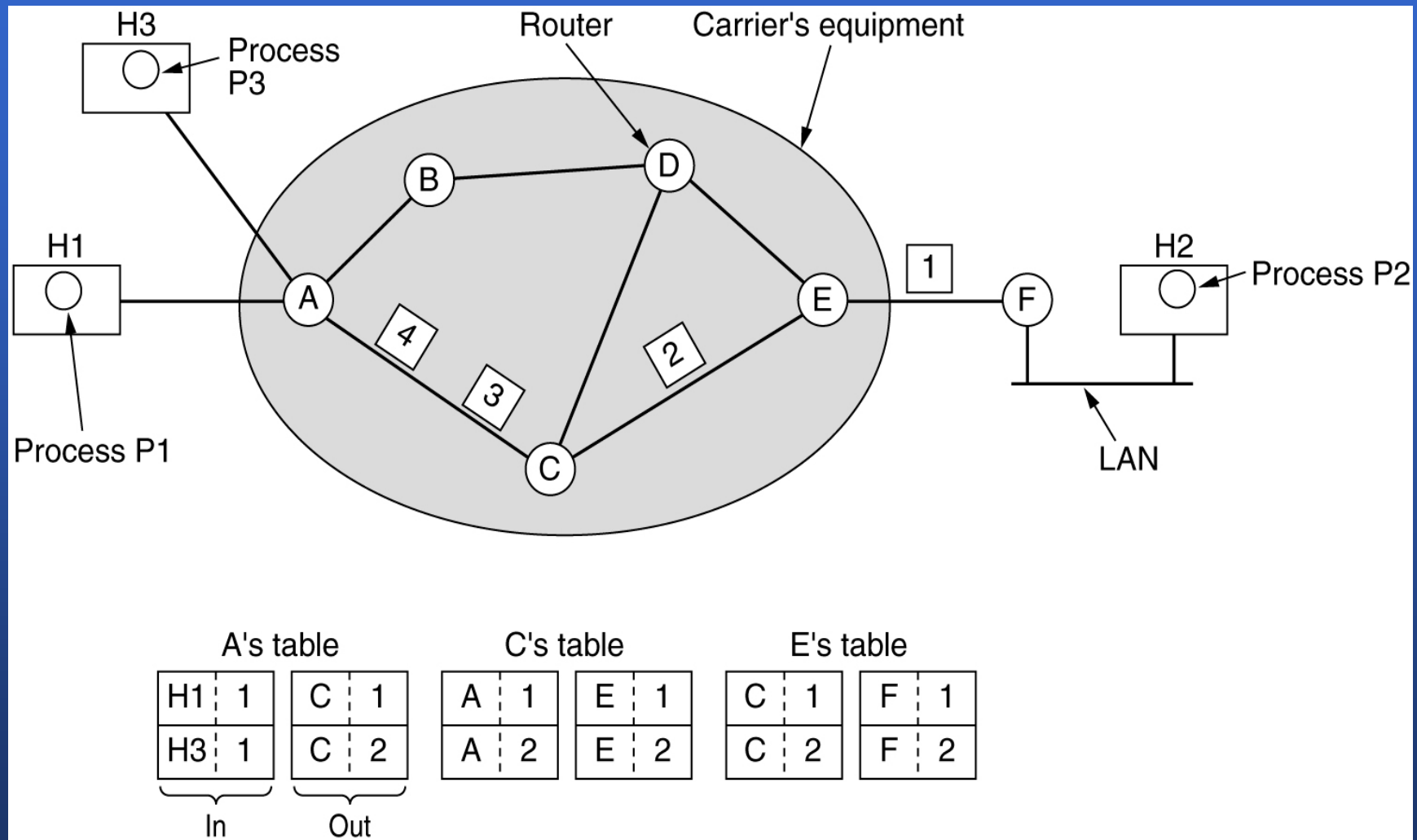
# Implementation of Connectionless Service

P1 on H1 → P2 on H2

- P1:application layer → H1: transport layer
- H1:transport layer → H1: network layer (disassemble)
- H1:network layer → H2: network layer
  - The routes for packets 1,2,3  
H1 → A → C → E → F → H2
  - The routes for packets 4  
H1 → A → B → D → E → F → H2
- H2: network layer → H2: transport layer (assemble)
- H2: transport layer → P2: application layer



## 5.1.4 Implementation of Connection-Oriented Service



Routing within a virtual-circuit subnet

## 5.1.5 Comparison of Virtual-Circuit and Datagram Subnets

| Issue                     | Datagram subnet                                              | Virtual-circuit subnet                                           |
|---------------------------|--------------------------------------------------------------|------------------------------------------------------------------|
| Circuit setup             | Not needed                                                   | Required                                                         |
| Addressing                | Each packet contains the full source and destination address | Each packet contains a short VC number                           |
| State information         | Routers do not hold state information about connections      | Each VC requires router table space per connection               |
| Routing                   | Each packet is routed independently                          | Route chosen when VC is set up; all packets follow it            |
| Effect of router failures | None, except for packets lost during the crash               | All VCs that passed through the failed router are terminated     |
| Quality of service        | Difficult                                                    | Easy if enough resources can be allocated in advance for each VC |
| Congestion control        | Difficult                                                    | Easy if enough resources can be allocated in advance for each VC |

# Comparison of Virtual-Circuit and Datagram Subnets

- Server trade-offs exist between virtual circuits and datagrams
  - Router memory space v.s. bandwidth
  - Setup time v.s. address parsing time.
  - Large routing table space for datagram subnets
  - Easy QoS for VC subnets
  - Vulnerability problem for VC subnets





## 5.2 ROUTING ALGORITHMS

- 功能：将分组从源端机器经选定路由送到目的端机器
- 路由选择算法
  - 希望具有的特征：
    - 正确性、简单性、健壮性、稳定性、公平性和最优性
    - 公平性与最优性之间的矛盾-[see fig 5-4]

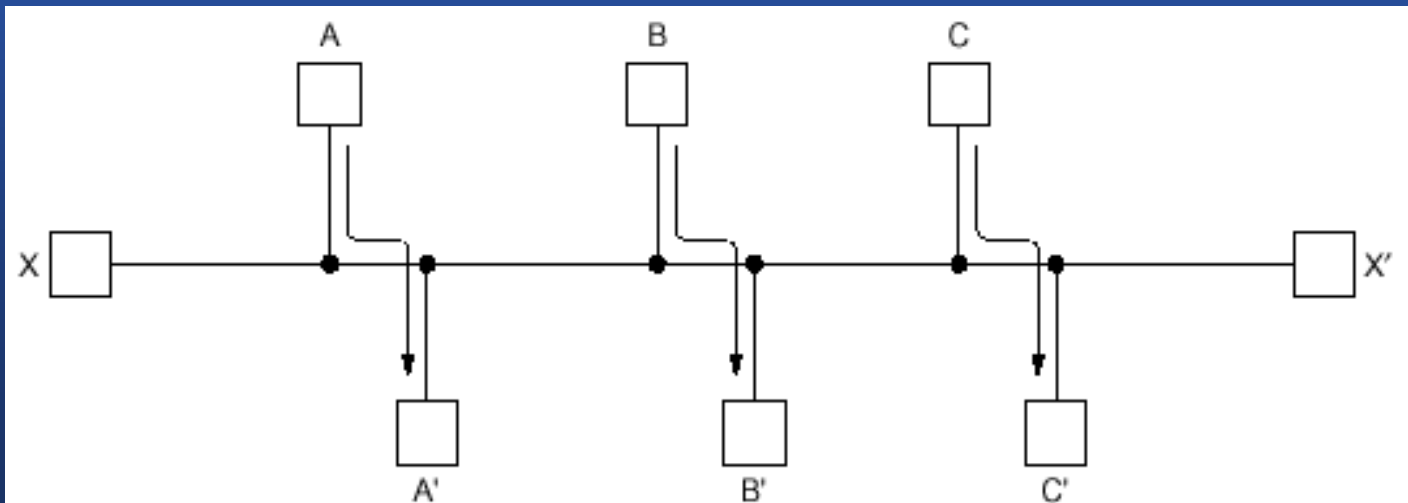


Fig. 5-4. Conflict between fairness and optimality.

# 5.2 ROUTING ALGORITHMS

- 路由选择算法
  - 数据报和虚电路采用不同的选择方法
  - 算法分类:
    - 非自适应算法 -> 静态路由算法
    - 自适应算法 -> 动态路由算法



## 5.2.1 The optimality principle

- 最优化原则
  - 如果路由器J在从路由器I到K的最佳路由上，那么J到K的最佳线路就会在同一路由上
- 汇集树-[see fig 5-5]
  - 从所有源端到目的端的最佳路由集合，形成了以目的地为根的树

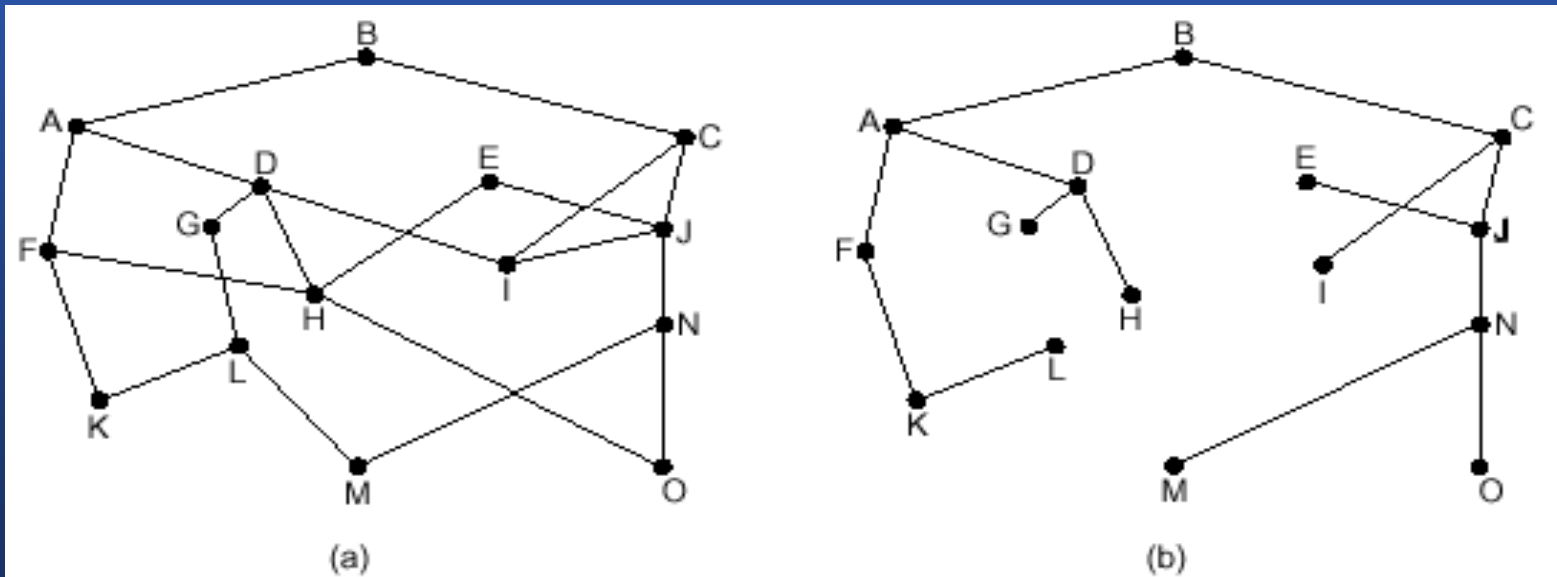


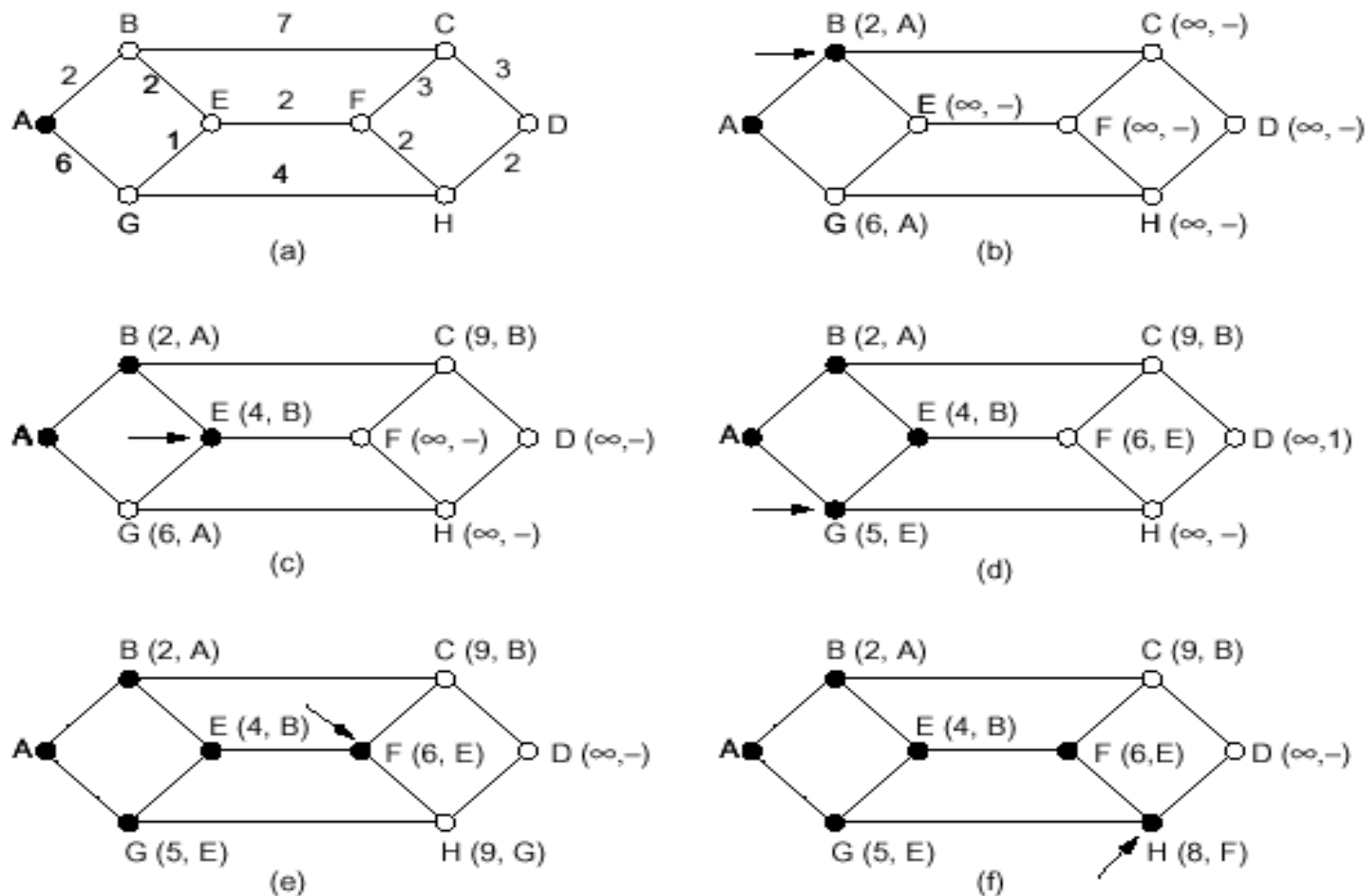
Fig. 5-5. (a) A subnet. (b) A sink tree for router B.

## 5.2.2最短路由选择 (Shortest path)

- 测量路径长度的方法（依据）：
  - 站点数量、距离、信道带宽、平均通信量、通信开销、队列平均长度、测量到的时延。。。
- (Dijkstra)算法实例：计算从A到D的最短路径-  
[see fig 5-6]
  - 从源到终节点
- 算法程序-[see fig 5-7]
  - 从终结点到源节点



## 5.2 路由选择算法



**Fig. 5-6.** The first five steps used in computing the shortest path from *A* to *D*. The arrows indicate the working node.

## 5.2 路由选择算法

```
#define MAX_NODES 1024                /* maximum number of nodes */
#define INFINITY 1000000000           /* a number larger than every maximum path */
int n, dist[MAX_NODES][MAX_NODES];   /* dist[i][j] is the distance from i to j */

void shortest_path(int s, int t, int path[])
{ struct state {                       /* the path being worked on */
    int predecessor;                  /* previous node */
    int length;                       /* length from source to this node */
    enum {permanent, tentative} label; /* label state */
} state[MAX_NODES];

int i, k, min;
struct state *
    p;
for (p = &state[0]; p < &state[n]; p++) { /* initialize state */
    p->predecessor = -1;
    p->length = INFINITY;
    p->label = tentative;
}
state[t].length = 0; state[t].label = permanent;
k = t; /* k is the initial working node */
do { /* Is there a better path from k? */
    for (i = 0; i < n; i++) /* this graph has n nodes */
        if (dist[k][i] != 0 && state[i].label == tentative) {
            if (state[k].length + dist[k][i] < state[i].length) {
                state[i].predecessor = k;
                state[i].length = state[k].length + dist[k][i];
            }
        }
    }
}
```

## 5.2 路由选择算法

```
/* Find the tentatively labeled node with the smallest label. */
k = 0; min = INFINITY;
for (i = 0; i < n; i++)
    if (state[i].label == tentative && state[i].length < min) {
        min = state[i].length;
        k = i;
    }
state[k].label = permanent;
} while (k != s);

/* Copy the path into the output array. */
i = 0; k = s;
} do {path[i++] = k; k = state[k].predecessor; } while (k >= 0);
```

**Fig. 5-7.** Dijkstra's algorithm to compute the shortest path through a graph.



## 5.2.3 扩散法 (Flooding)

- 原理：将收到的每个分组，从除了分组到来的线路外的所有线路上发出
- 问题：产生大量的分组
- 抑制措施：
  - 在分组头中包含“站点计数器”，当其减到零时即被丢弃
  - 记录下分组扩散的路径
- 改进：选择性扩散法
  - 仅将分组扩散到与正确方向接近的线路上
- 应用：实际应用较少



## 5.2.4 基于流量的路由选择 (\*)

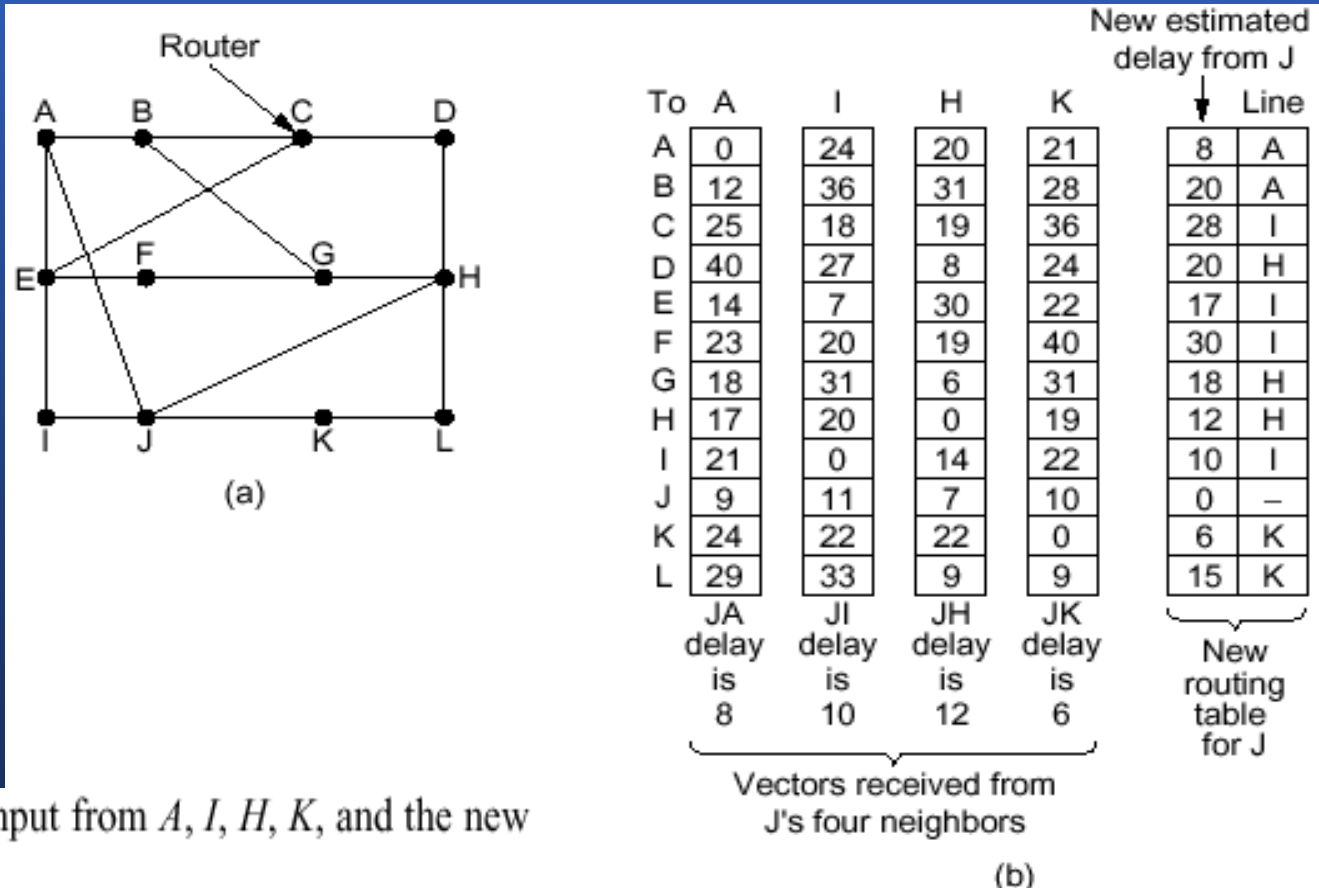
- 原理：根据已知载荷量和平均流量，计算出分组平均延迟，最终找出产生网络最小延迟的路由
- 须预先知道：网络拓扑结构、通信量矩阵、各线路容量的矩阵



## 5.2.5 Distance vector routing

- 原理：
    - 让每个路由器维护一张向量表，表中给出每个目的地已知的最佳距离和线路
    - 通过与相邻路由器交换信息来更新表的信息
  - 应用：ARPANET、因特网、DECNet、Novell IPX、Cisco Routers 等
  - 算法描述：
    - J-A:8
    - J-I:10
    - J-H:12
    - J-K:6
- The diagram shows a network topology with eight nodes labeled A through H. The nodes are arranged in two rows: A, B, C, D on top and E, F, G, H on the bottom. Connections are as follows: A-B, B-C, C-D, D-H, H-G, G-F, F-E, E-A, A-E, B-F, C-G, and D-H. An arrow points to node C, labeling it as 'Router'. To the right of the diagram is a routing table for Router C, showing the distance to four destinations: A, I, H, and K.

| To | A  | I  | H  | K  |
|----|----|----|----|----|
| A  | 0  | 24 | 20 | 21 |
| B  | 12 | 36 | 31 | 28 |
| C  | 25 | 18 | 19 | 36 |
| D  | 40 | 27 | 8  | 24 |



**Fig. 5-10.** (a) A subnet. (b) Input from  $A$ ,  $I$ ,  $H$ ,  $K$ , and the new routing table for  $J$ .

## 5.2.5 Distance vector routing

- 无穷计算问题
  - 算法的缺陷：算法收敛较慢。（对好消息反应迅速，对坏消息反应迟钝） - [see fig 5-11]

| A        | B        | C        | D        | E        |                   |
|----------|----------|----------|----------|----------|-------------------|
| ●        | ●        | ●        | ●        | ●        |                   |
| $\infty$ | $\infty$ | $\infty$ | $\infty$ | $\infty$ | Initially         |
| 1        | $\infty$ | $\infty$ | $\infty$ | $\infty$ | After 1 exchange  |
| 1        | 2        | $\infty$ | $\infty$ | $\infty$ | After 2 exchanges |
| 1        | 2        | 3        | $\infty$ | $\infty$ | After 3 exchanges |
| 1        | 2        | 3        | 4        | $\infty$ | After 4 exchanges |

(a)

| A | B        | C        | D        | E        |                   |
|---|----------|----------|----------|----------|-------------------|
| ● | ●        | ●        | ●        | ●        |                   |
|   | 1        | 2        | 3        | 4        | Initially         |
|   | 3        | 2        | 3        | 4        | After 1 exchange  |
|   | 3        | 4        | 3        | 4        | After 2 exchanges |
|   | 5        | 4        | 5        | 4        | After 3 exchanges |
|   | 5        | 6        | 5        | 6        | After 4 exchanges |
|   | 7        | 6        | 7        | 6        | After 5 exchanges |
|   | 7        | 8        | 7        | 8        | After 6 exchanges |
|   | $\vdots$ |          |          |          |                   |
|   | $\infty$ | $\infty$ | $\infty$ | $\infty$ |                   |

(b)

Fig. 5-11. The count-to-infinity problem.

## 5.2.5 Distance vector routing

- 水平分裂算法 — 解决“坏消息”反应慢的问题

- 特点：到X的距离并不由直接通向X的邻居节点报告

- 失败的情况：— [s

A→D=2, B→D=2

if CD 断了,

C告知A、B不可到达D

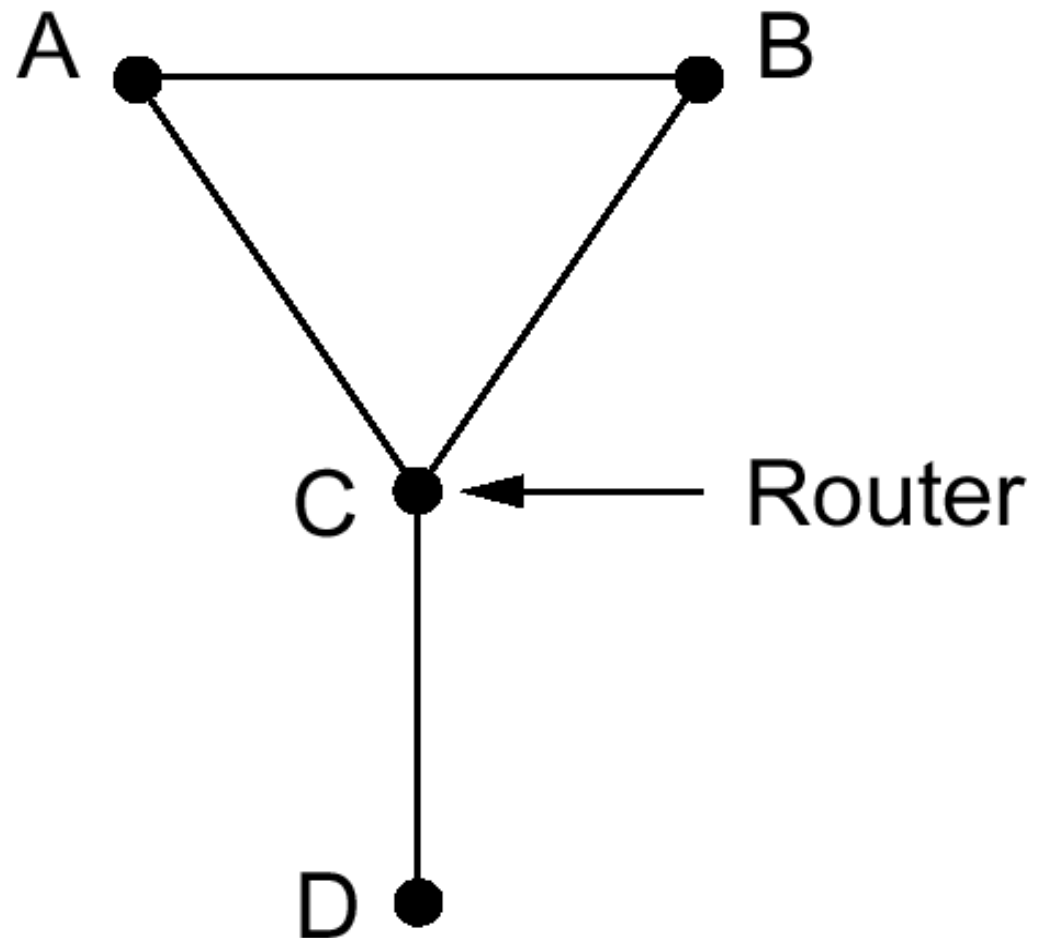
但, A知道B可达D,

长度=2, 所以A→B→D

长度=3;

同样 B→A→D 长度=3

...



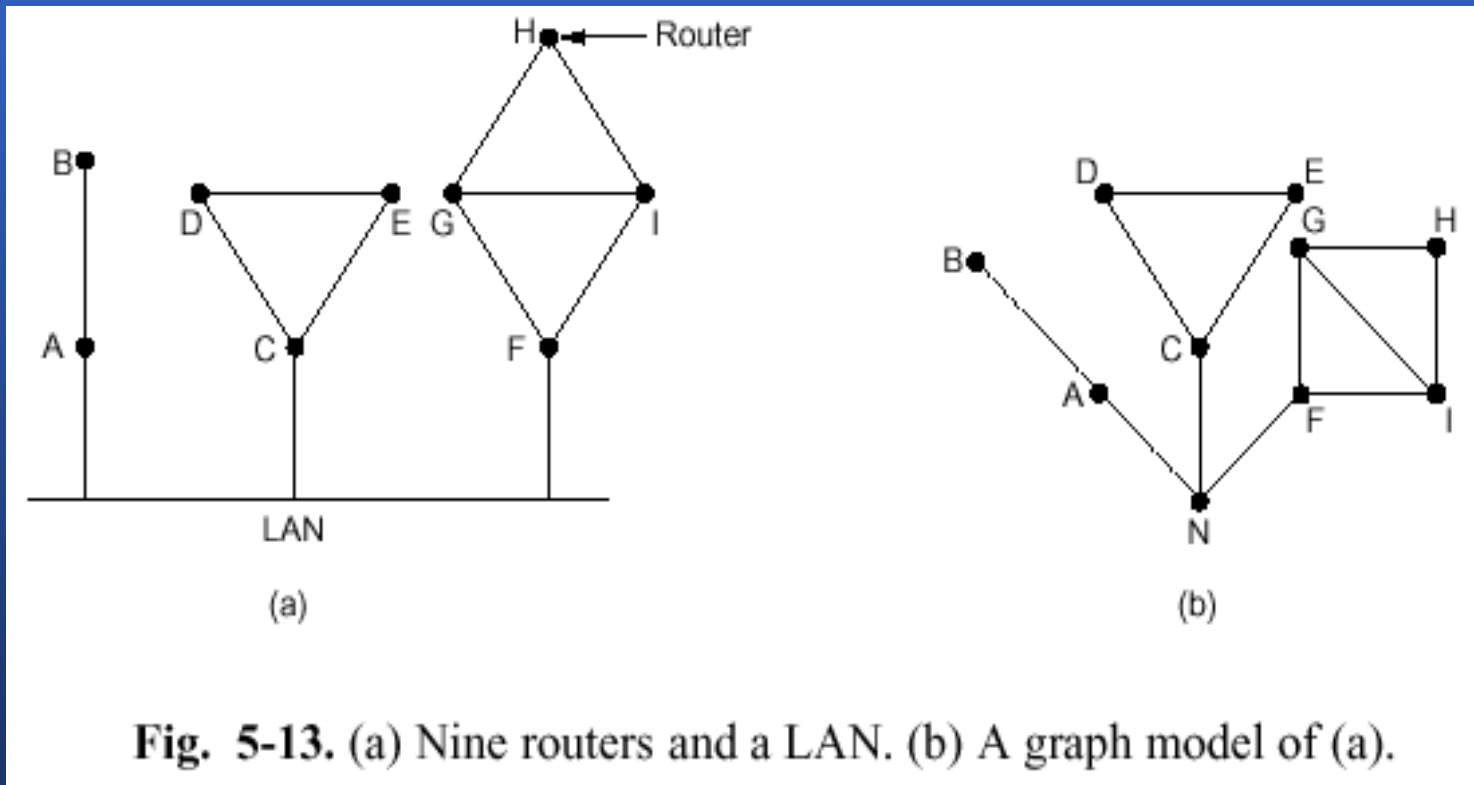
## 5.2.6 链路状态路由选择 (Link state routing)

- 距离矢量路由选择存在的问题
  - 没有考虑线路的带宽
  - 记录信息耗时过多， 算法收敛速度慢
- 工作原理（对每个路由器）：
  - 发现邻居节点，及其网络地址
  - 测量线路开销或延迟
  - 构造链路状态分组
  - 发布链路状态分组
  - 计算新最短路由



## 5.2.6 Link state routing

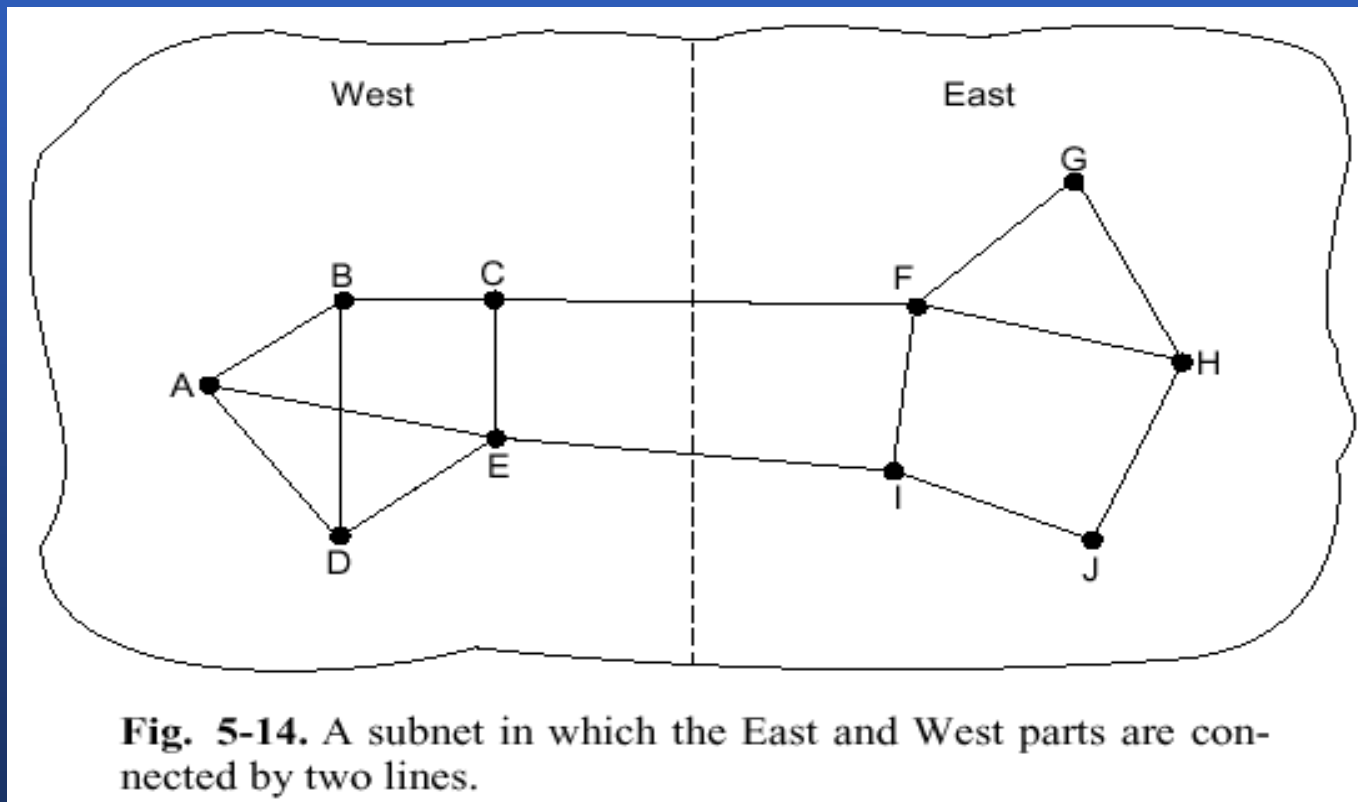
- 发现邻居节点-[see fig 5-13]
  - 通过发送 Hello 分组来实现





## 5.2.6 Link state routing

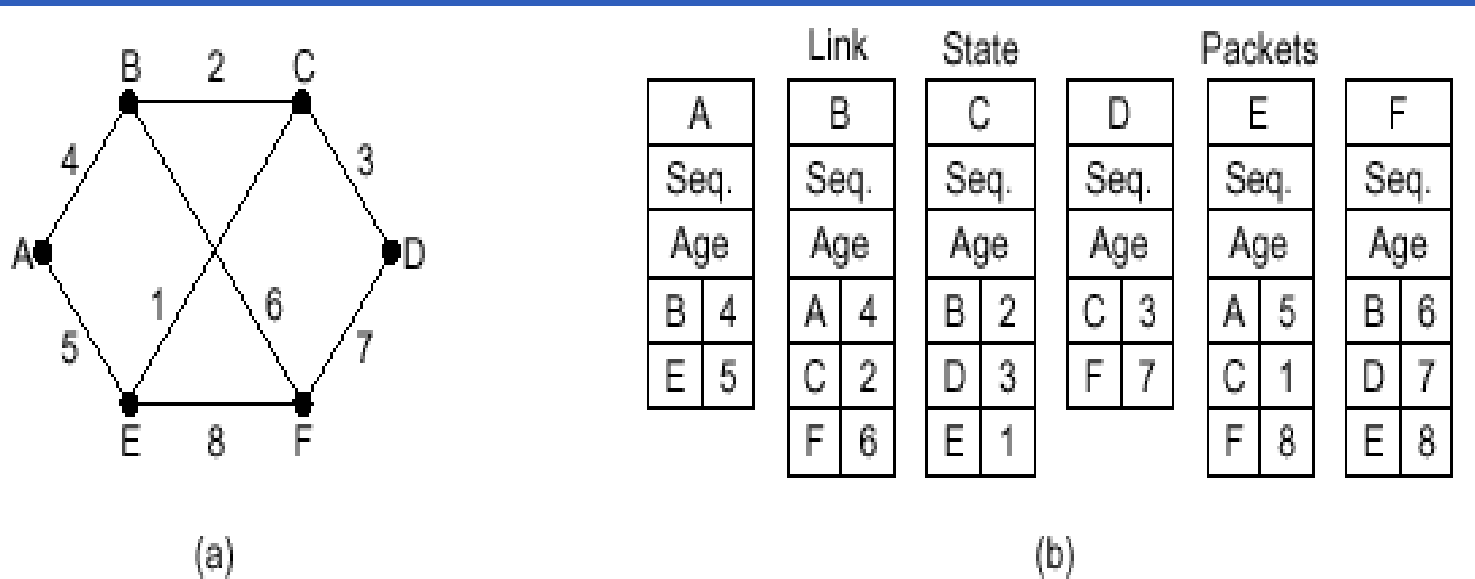
- 测量线路开销或延迟
  - 发送ECHO分组，将响应时间/2
  - 考虑载荷， 某些争议 -[see fig 5-14]





## 5. 2. 6 Link state routing

- 构造链路状态分组-[see fig 5-15]
  - 何时构造? 定期创建 或 出现重大事件时再创建



**Fig. 5-15.** (a) A subnet. (b) The link state packets for this subnet.

## 5.2.6 Link state routing

- 发布链路状态分组
  - 基本思想：利用扩散法发布链路状态分组；每个分组包含一个顺序号；每次发送新分组时顺序号加1
  - 问题
    - 顺序号的循环使用
    - 路由器崩溃，造成顺序号丢失
    - 顺序号传送错误
  - 解决办法
    - 每个分组的顺序号后加年龄字段（age）
  - 算法的改进-[\[see fig 5-16\]](#)
    - 路由器将分组先存储，比较后再发送

| Source | Seq. | Age | Send flags |   |   | ACK flags |   |   | Data |
|--------|------|-----|------------|---|---|-----------|---|---|------|
|        |      |     | A          | C | F | A         | C | F |      |
| A      | 21   | 60  | 0          | 1 | 1 | 1         | 0 | 0 |      |
| F      | 21   | 60  | 1          | 1 | 0 | 0         | 0 | 1 |      |
| E      | 21   | 59  | 0          | 1 | 0 | 1         | 0 | 1 |      |
| C      | 20   | 60  | 1          | 0 | 1 | 0         | 1 | 0 |      |
| D      | 21   | 59  | 1          | 0 | 0 | 0         | 1 | 1 |      |

Fig. 5-16. The packet buffer for router *B* in Fig.5-15.



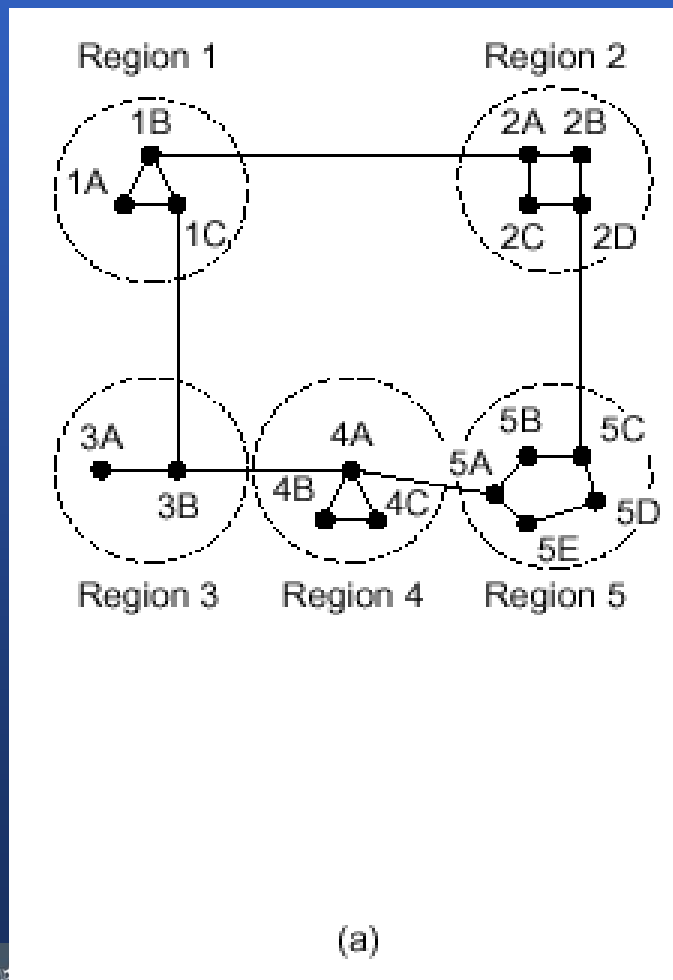
## 5. 2. 6 Link state routing

- 计算新最短路由
- 应用
  - OSPF
  - 中间系统至中间系统IS-IS
  - 两者的区别： IS-IS可同时携带多个网络层协议的信息



## 5.2.7 分级路由选择 (\*)

- 网络增大、路由表成比例增大
- 将路由器按区域分组-[see fig 5-17]



Full table for 1A

| Dest. | Line | Hops |
|-------|------|------|
| 1A    | -    | -    |
| 1B    | 1B   | 1    |
| 1C    | 1C   | 1    |
| 2A    | 1B   | 2    |
| 2B    | 1B   | 3    |
| 2C    | 1B   | 3    |
| 2D    | 1B   | 4    |
| 3A    | 1C   | 3    |
| 3B    | 1C   | 2    |
| 4A    | 1C   | 3    |
| 4B    | 1C   | 4    |
| 4C    | 1C   | 4    |
| 5A    | 1C   | 4    |
| 5B    | 1C   | 5    |
| 5C    | 1B   | 5    |
| 5D    | 1C   | 6    |
| 5E    | 1C   | 5    |

(b)

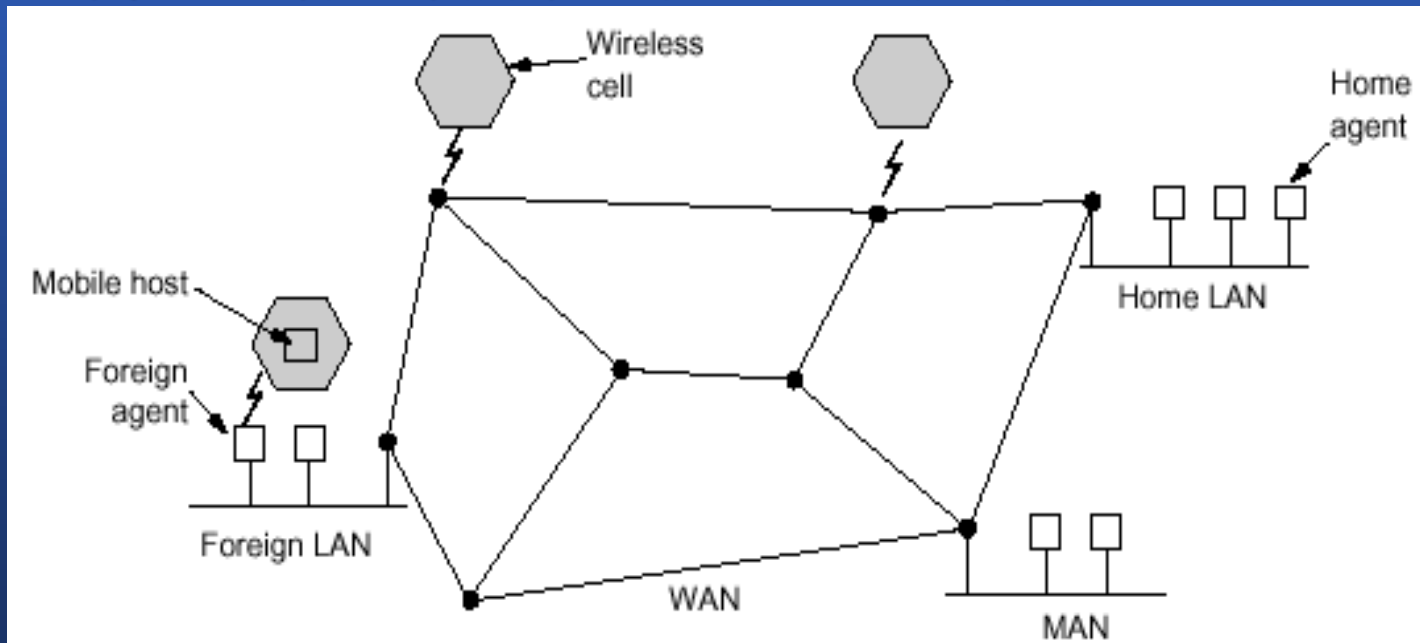
Hierarchical table for 1A

| Dest. | Line | Hops |
|-------|------|------|
| 1A    | -    | -    |
| 1B    | 1B   | 1    |
| 1C    | 1C   | 1    |
| 2     | 1B   | 2    |
| 3     | 1C   | 2    |
| 4     | 1C   | 3    |
| 5     | 1C   | 4    |

(c)

## 5.2.8 移动主机路由选择 (\*)

- 静态用户 - 从不移动的用户
- 非静态用户 (迁移用户、漫游用户) -> 移动用户
- 网络世界模型 - [see fig 5-18]
- 外地代理 - 管理所有来当地的动态用户
- 主代理 - 管理本属本区域, 但当时正在外地的用户



**Fig. 5-18.** A WAN to which LANs, MANs, and wireless cells are attached.

## 5.2.8 移动主机路由选择 (\*)

### - 典型的登录过程:

1. 外地代理定期广播分组，宣布自己的存在及地址；同时，移动主机可以广播，问“这里有没有外地代理？”
2. 移动主机登录到外地代理
3. 外地代理与移动主机的主代理联系
4. 主代理检查安全性信息
5. 外地代理得到主代理的确认后，建立一个表项，并通知移动主机已经登录了

### - 退出登录



## 5.2.8 移动主机路由选择 (\*)

- 数据发送过程: - [see fig 5-19]

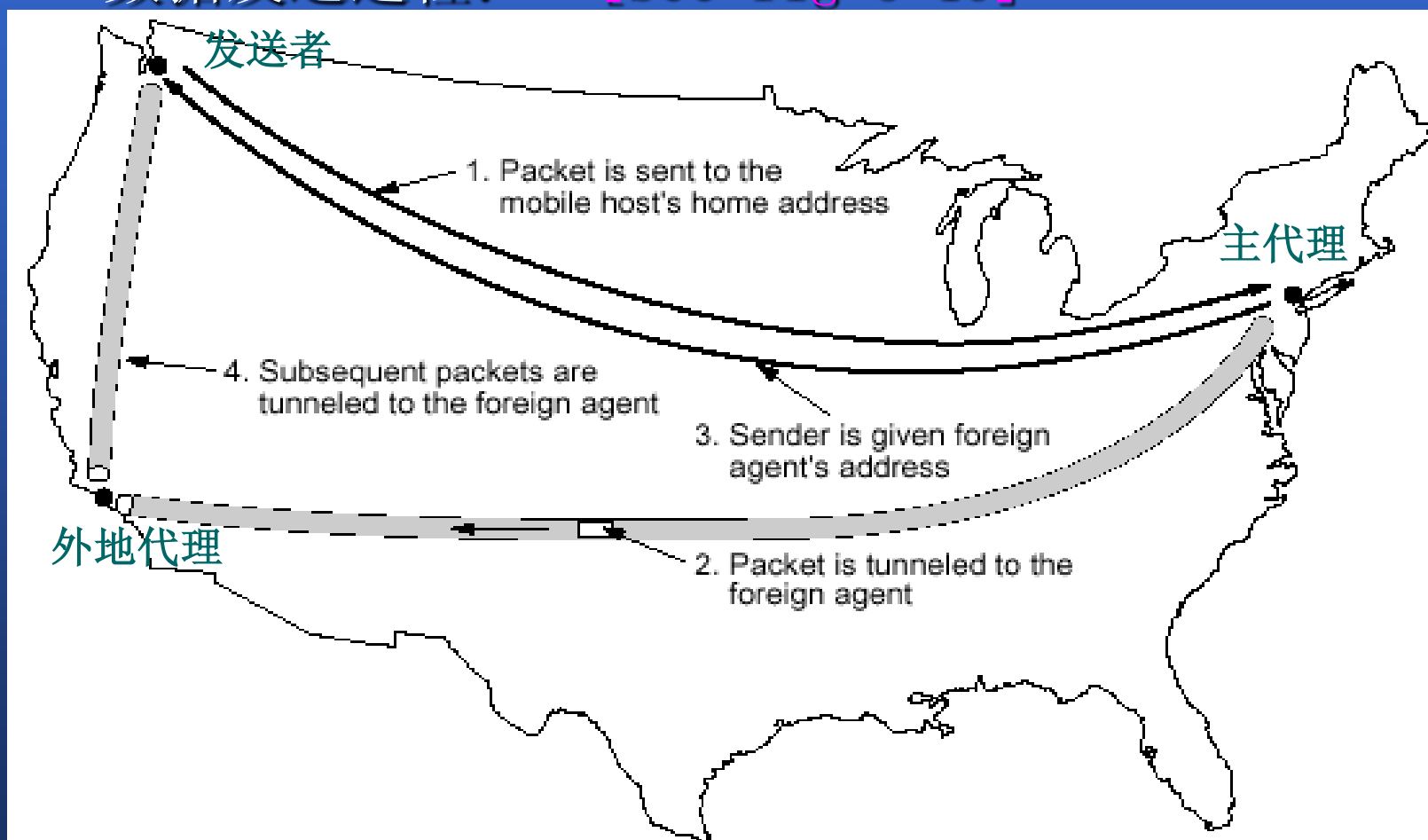


Fig. 5-19. Packet routing for mobile users.



## 5.2.9 广播路由选择 (\*)

### - 几种算法

- 算法1: 分别发送分组到每个目的端
- 算法2: 扩散法
- 算法3: 多目的地路由选择
- 算法4: 利用发起广播的路由器的信息树或利用生成树

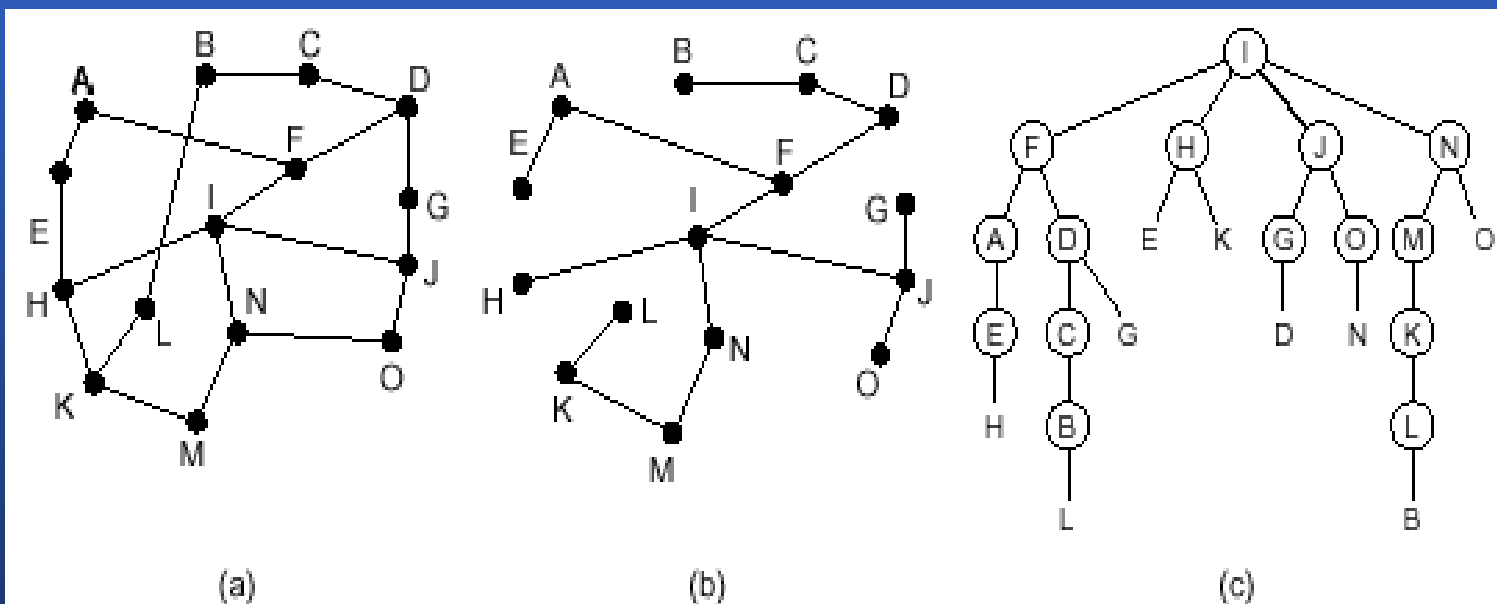
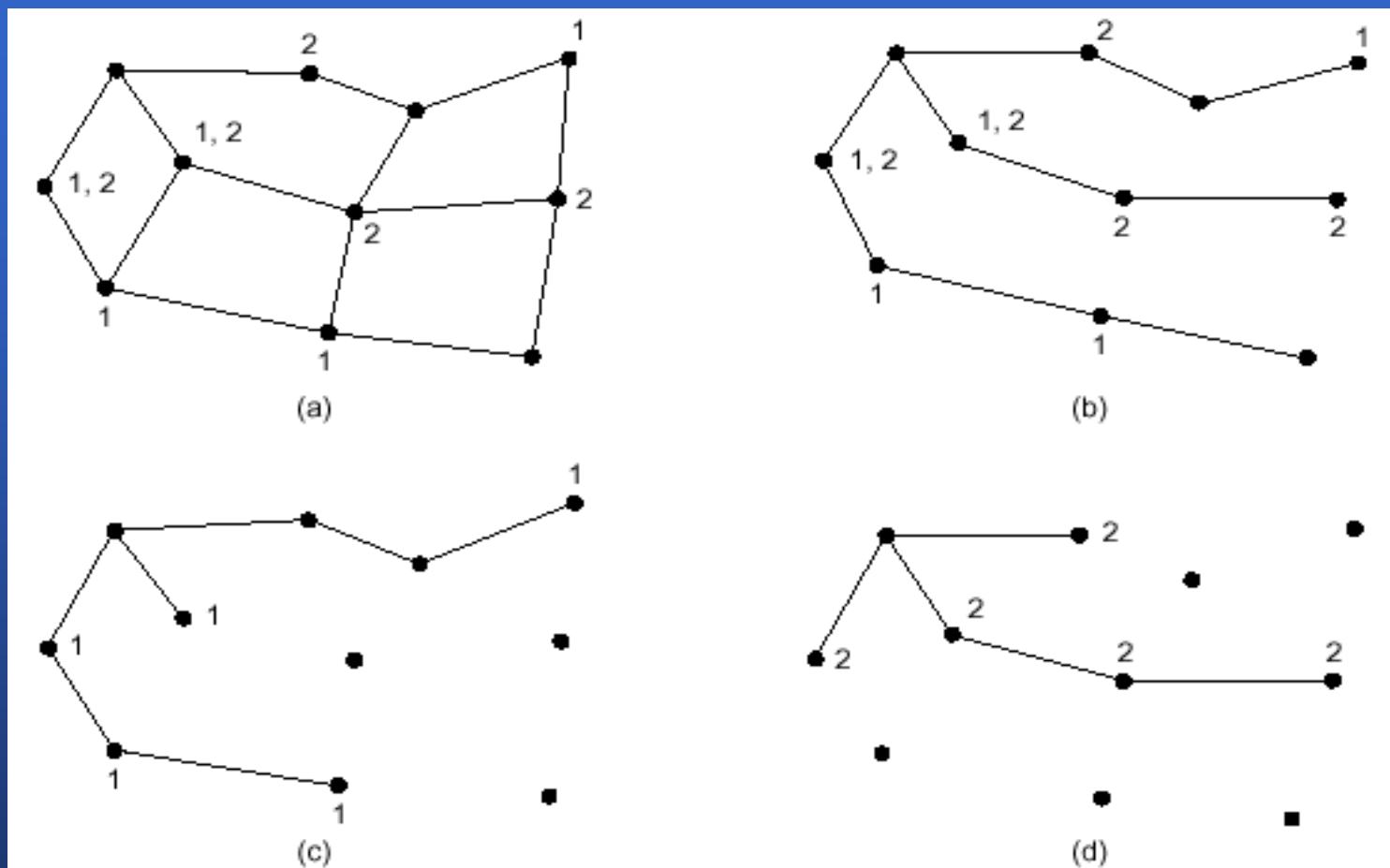


Fig. 5-20. Reverse path forwarding. (a) A subnet. (b) A spanning tree. (c) The tree built by reverse path forwarding.

## 5.2.10 多点播送路由选择 (\*)

- 多点播送 (Multicast)
- 实现方法
  - 良好的小组管理机制
    - 创建和取消小组
    - 允许进程进入、离开小组
  - 发送时，第一个路由器检查生成树，修剪小组成员不能到达的线路-[see fig 5-21]
  - 修剪生成树的方法
    - 使用链路状态时使用，从树叶开始，向树根发展
    - 对距离矢量路由，采用逆向路由转发（缺点：存储空间占用大）
    - 核心基本树：每个组只计算一棵生成树。树根靠近组的中间部位（优点：降低存储开销）

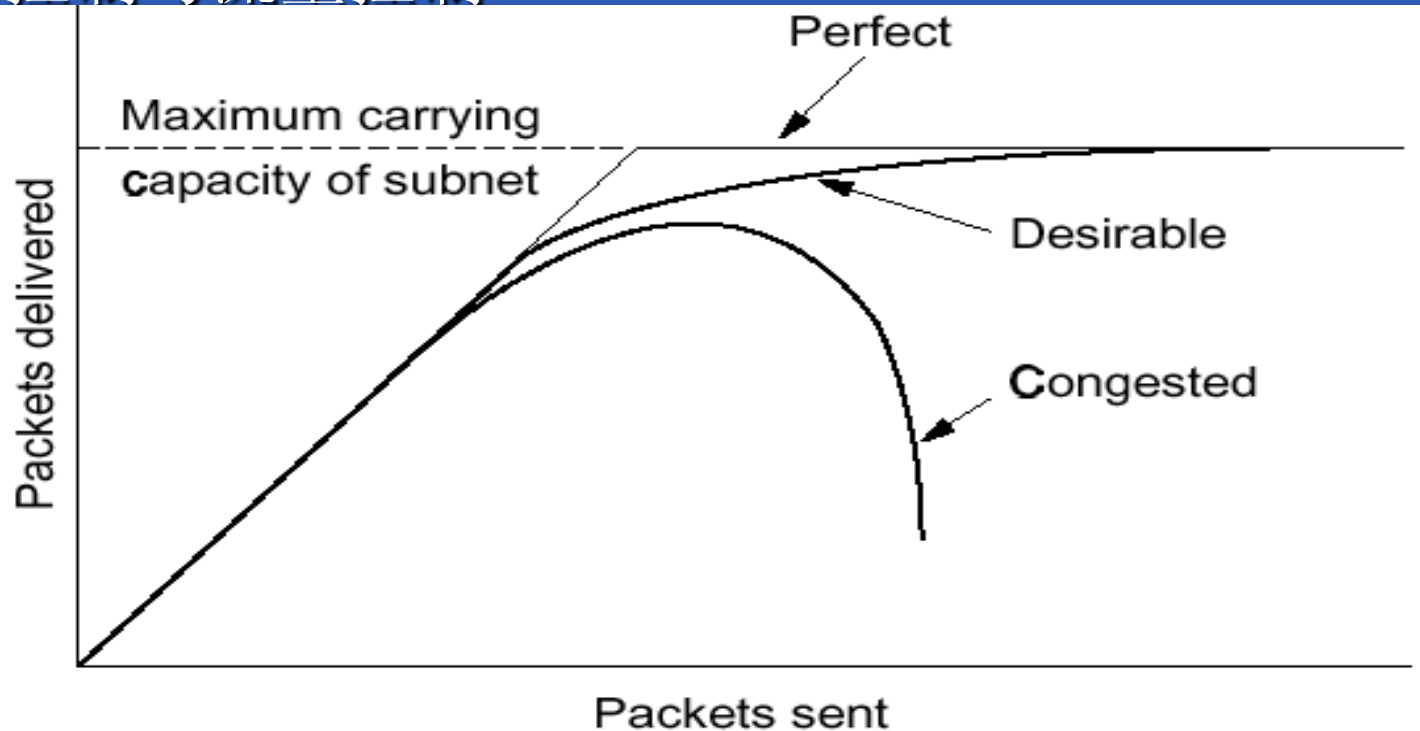
## 5.2.10 多点播送路由选择 (\*)



**Fig. 5-21.** (a) A subnet. (b) A spanning tree for the leftmost router. (c) A multicast tree for group 1. (d) A multicast tree for group 2.

## 5.3 拥塞控制算法 (\*)

- 拥塞 — 当通信子网中有太多的分组时，其性能降低—[see fig 5-22]
- 造成拥塞的原因
- 拥塞控制与流量控制



**Fig. 5-22.** When too much traffic is offered, congestion sets in and performance degrades sharply.

## 5.3.1 拥塞控制的基本原理

- 控制论
- 开环与闭环
- 解决拥塞的办法：增加资源或降低载荷
- 虚电路—网络层中解决
- 数据报—传输层中解决



## 5.3.2 拥塞预防策略

- 影响拥塞的策略-[see fig 5-23]

| Layer     | Policies                                                                                                                                                                                                                                           |
|-----------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Transport | <ul style="list-style-type: none"><li>• Retransmission policy</li><li>• Out-of-order caching policy</li><li>• Acknowledgement policy</li><li>• Flow control policy</li><li>• Timeout determination</li></ul>                                       |
| Network   | <ul style="list-style-type: none"><li>• Virtual circuits versus datagram inside the subnet</li><li>• Packet queueing and service policy</li><li>• Packet discard policy</li><li>• Routing algorithm</li><li>• Packet lifetime management</li></ul> |
| Data link | <ul style="list-style-type: none"><li>• Retransmission policy</li><li>• Out-of-order caching policy</li><li>• Acknowledgement policy</li><li>• Flow control policy</li></ul>                                                                       |

Fig. 5-23. Policies that affect congestion.



## 5.3.3 通信量整形 (\*)

- congestion的主要原因: 通信量的突发性
- 整形是调整数据传输的平均速率
- 漏桶算法-[see fig 5-24]

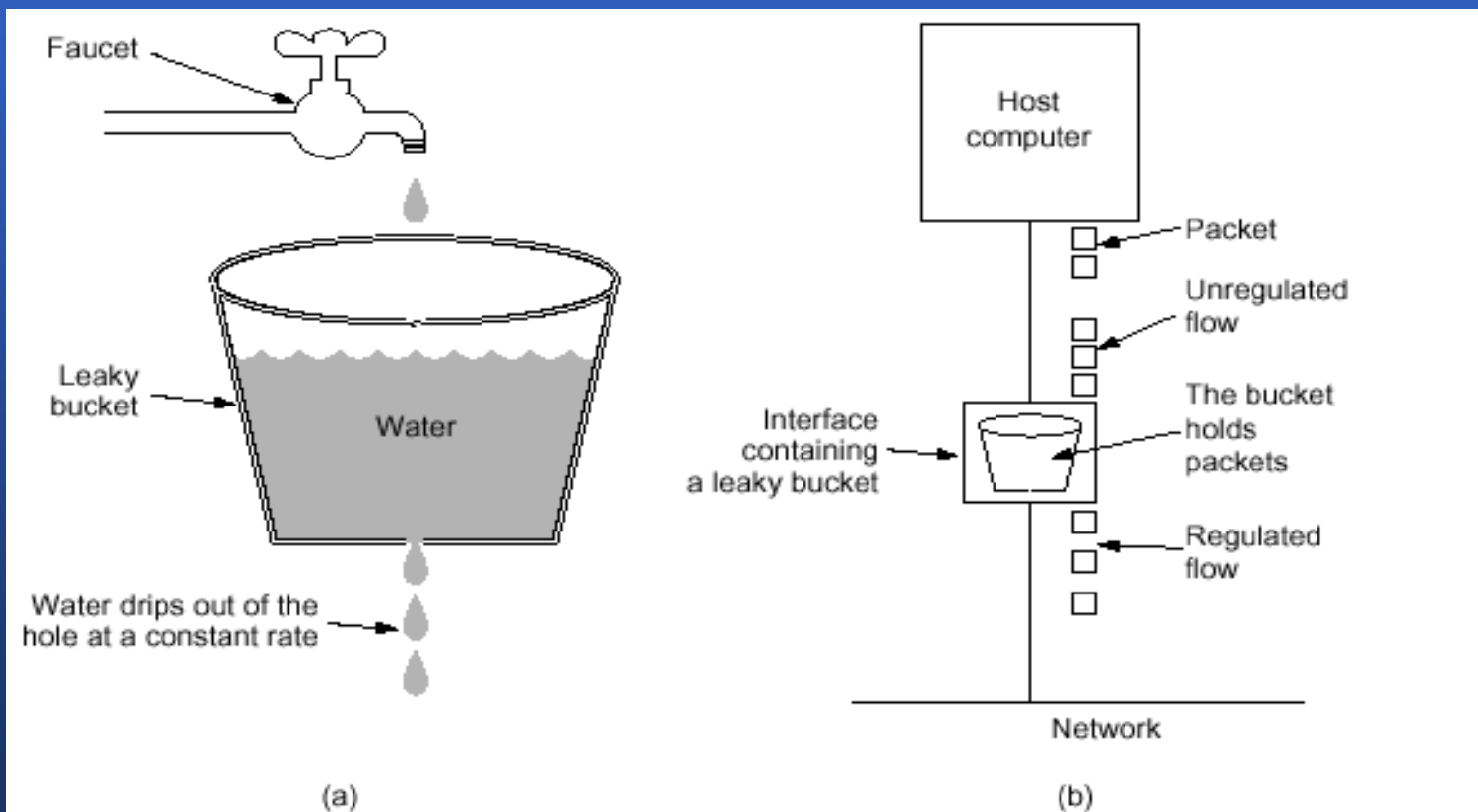
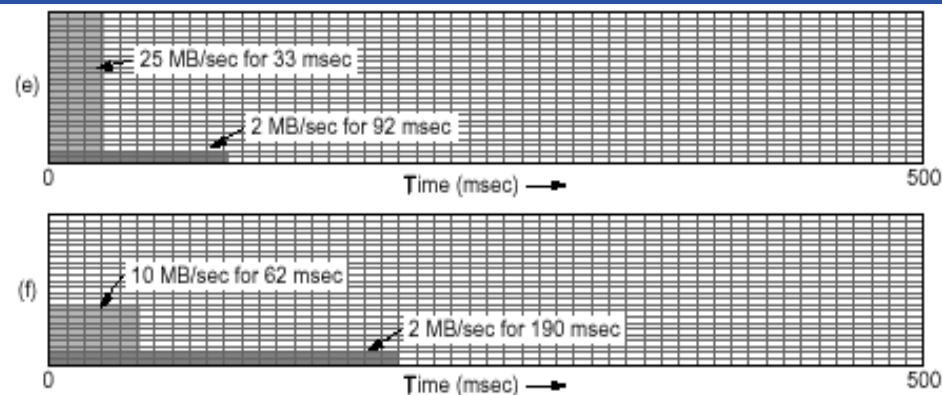
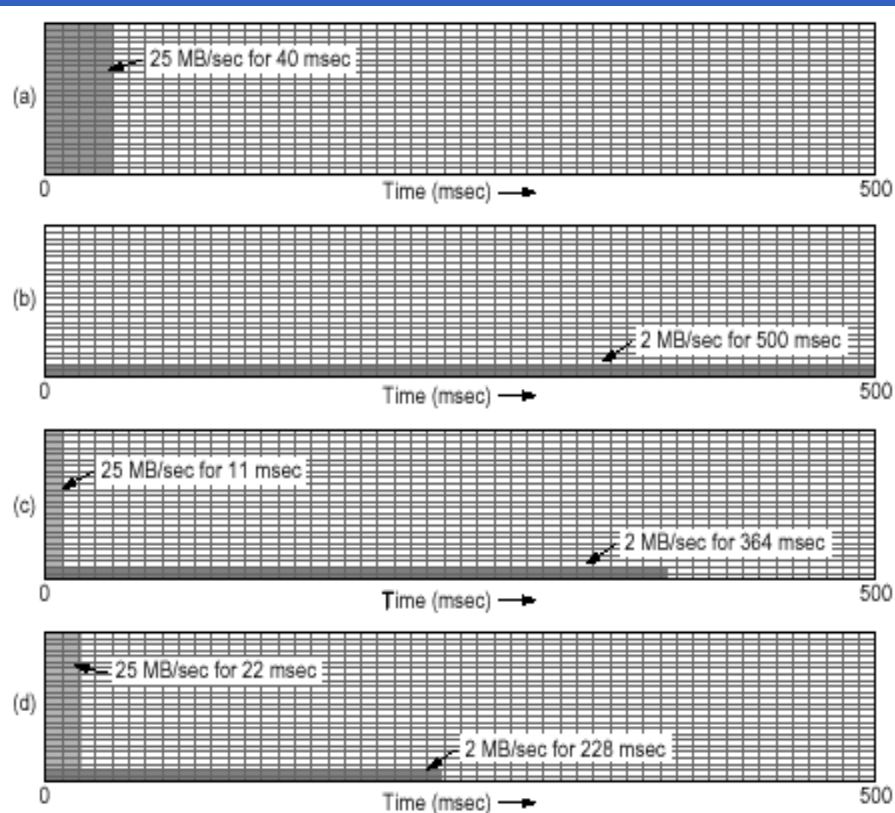


Fig. 5-24. (a) A leaky bucket with water. (b) A leaky bucket with packets.



## 5.3.3 通信量整形 (\*)

- 令牌桶算法-[see fig 5-25/26]



**Fig. 5-25.** (a) Input to a leaky bucket. (b) Output from a leaky bucket. (c) - (e) Output from a token bucket with capacities of 250KB, 500KB, and 750KB. (f) Output from a 500KB token bucket feeding a 10 MB/sec leaky bucket.



## 5.4 QUALITY OF SERVICE

- Requirements
- Techniques for Achieving Good Quality of Service
- Integrated Services
- Differentiated Services
- Label Switching and MPLS



## 5.4.1 Quality Of Service: Requirements

- A **flow** is a stream of packets from a source to a destination
  - In a connection-oriented network, all the packets belonging to a flow follow the same route;
  - In a connection-less network, they may follow different routes.
- The needs of each flow can be characterized by four primary parameters:
  - Reliability,
  - Delay
  - Jitter
  - bandwidth



# Quality Of Service: Requirements

| Application       | Reliability | Delay  | Jitter | Bandwidth |
|-------------------|-------------|--------|--------|-----------|
| E-mail            | High        | Low    | Low    | Low       |
| File transfer     | High        | Low    | Low    | Medium    |
| Web access        | High        | Medium | Low    | Medium    |
| Remote login      | High        | Medium | Medium | Low       |
| Audio on demand   | Low         | Low    | High   | Medium    |
| Video on demand   | Low         | Low    | High   | High      |
| Telephony         | Low         | High   | High   | Low       |
| Videoconferencing | Low         | High   | High   | High      |

How stringent the quality-of-service requirements are.



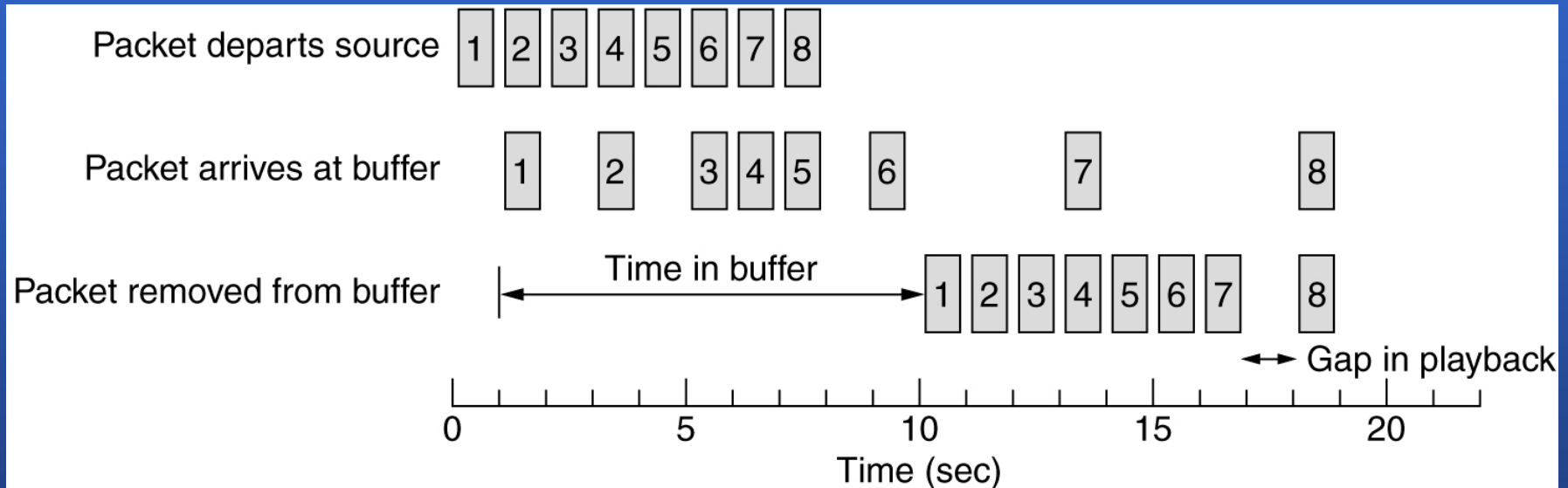
## 5.4.2 Quality Of Service: Techniques for QoS

- Overprovisioning
  - To provide so much router capacity, buffer space and bandwidth that the packets just fly easily
- Discussion
  - Easy
  - Practical
  - But expensive



## 5.4.2 Quality Of Service: Techniques for QoS

- Buffering at the client



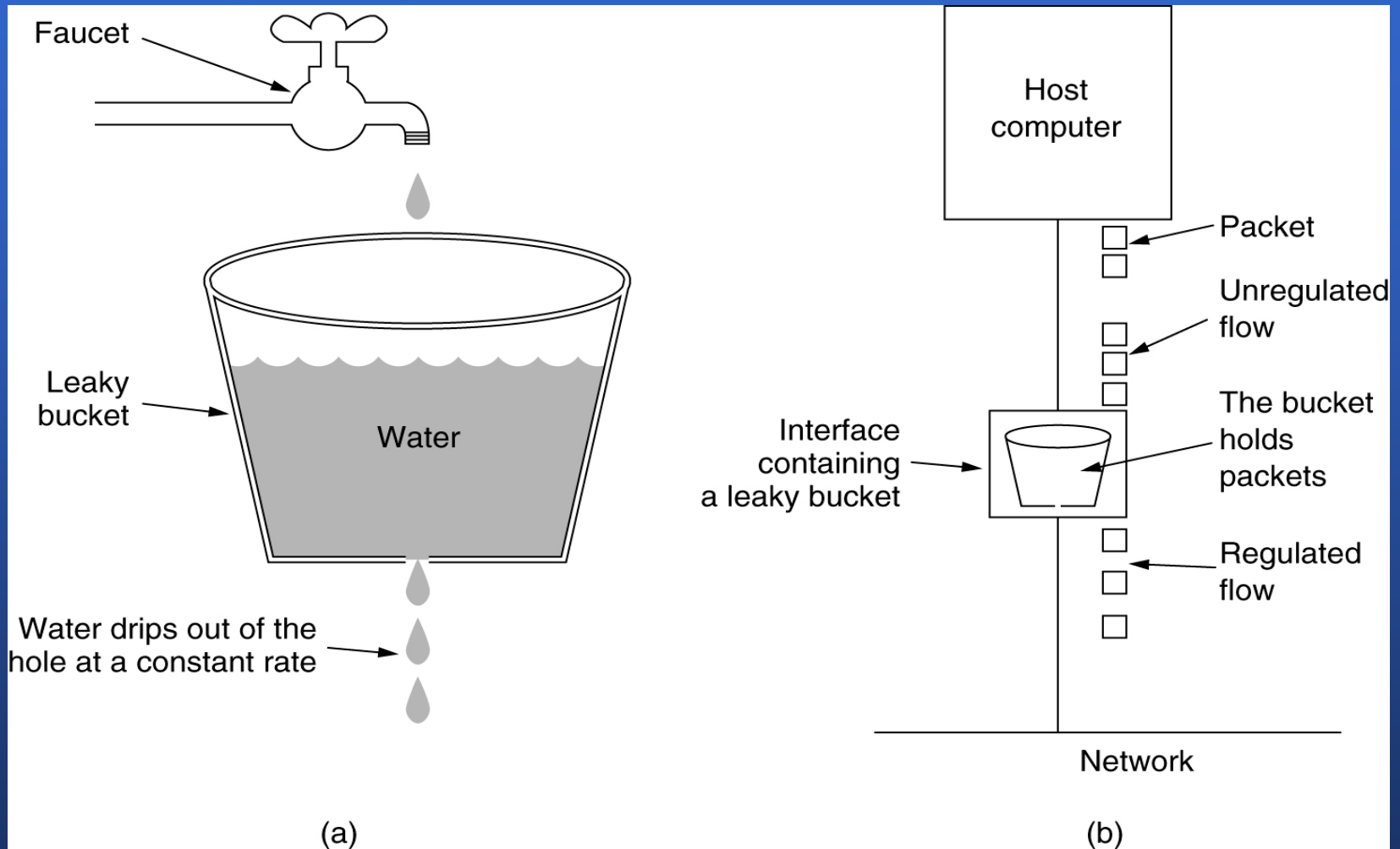
Smoothing the output stream by buffering packets.

## 5.4.2 Quality Of Service: Techniques for QoS

- **Buffering at the server** (traffic shaping): to smooth out the traffic on the server side, rather than on the client side.
- Traffic shaping (the average rate) v.s. sliding window protocols (the amount of data in transit at a time)
- Traffic shaping types:
  - The **Leaky Bucket Algorithm**
  - The **Token Bucket Algorithm**



## 5.4.2 Quality Of Service: Techniques for QoS



(a) A leaky bucket with water. (b) a leaky bucket with packets.



## 5.4.2 Quality Of Service: Techniques for QoS

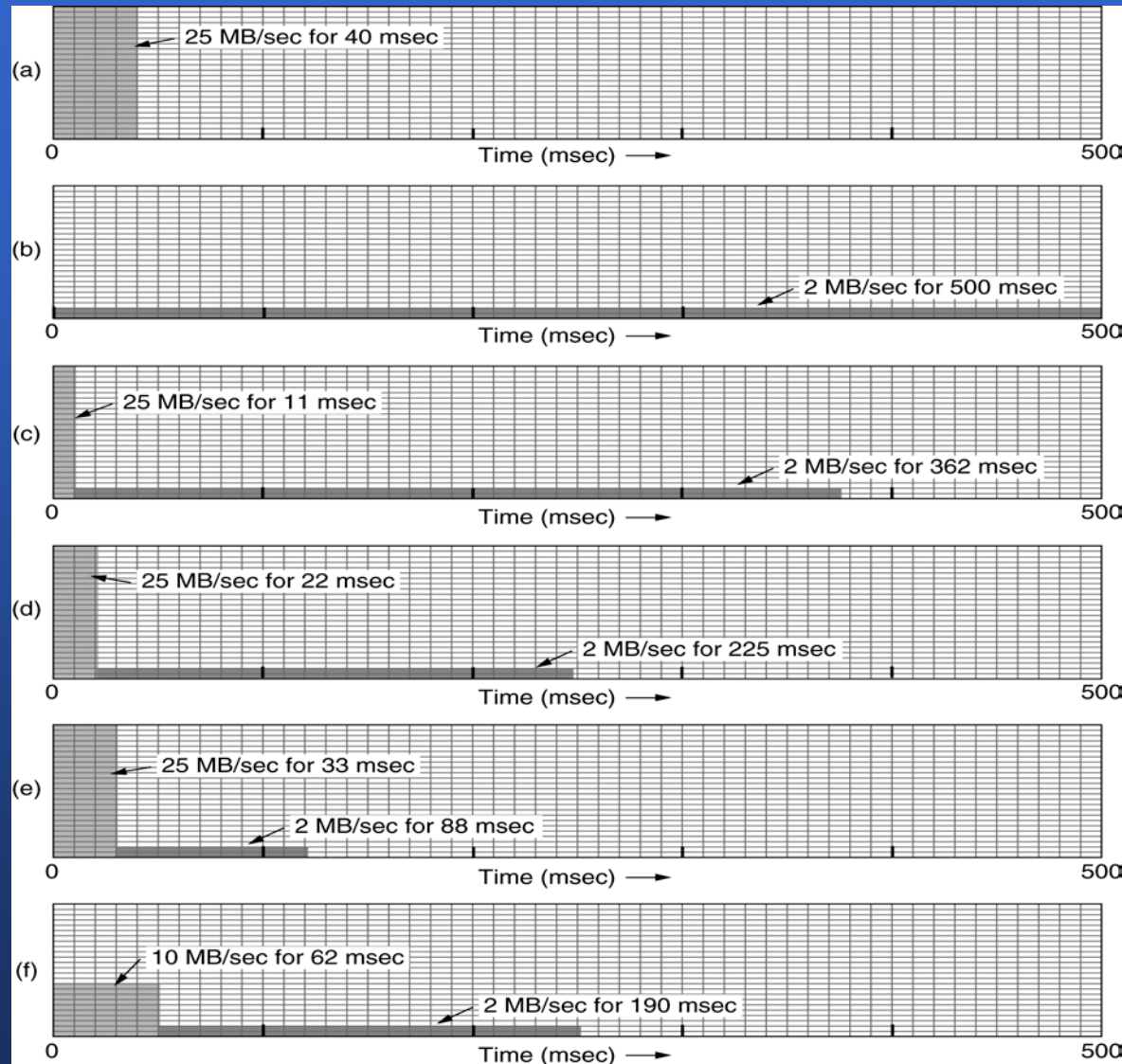
- The original leaky bucket algorithm (**packet-based**)
  - The leaky bucket consists of a finite queue
  - When a packet arrives, if there is room on the queue it is appended to the queue; otherwise, it is discarded.
  - At every clock tick, one packet is transmitted (unless the queue is empty)
- The byte-counting leaky (**byte-based**)
  - At each tick, a counter is initialized to  $n$ .
  - If the first packet on the queue has fewer bytes than  $n$ , it is transmitted, and the counter is decremented. Additional packets may be sent.
  - When the counter drops below the length of the next packet on the queue, transmission stops until the next tick.



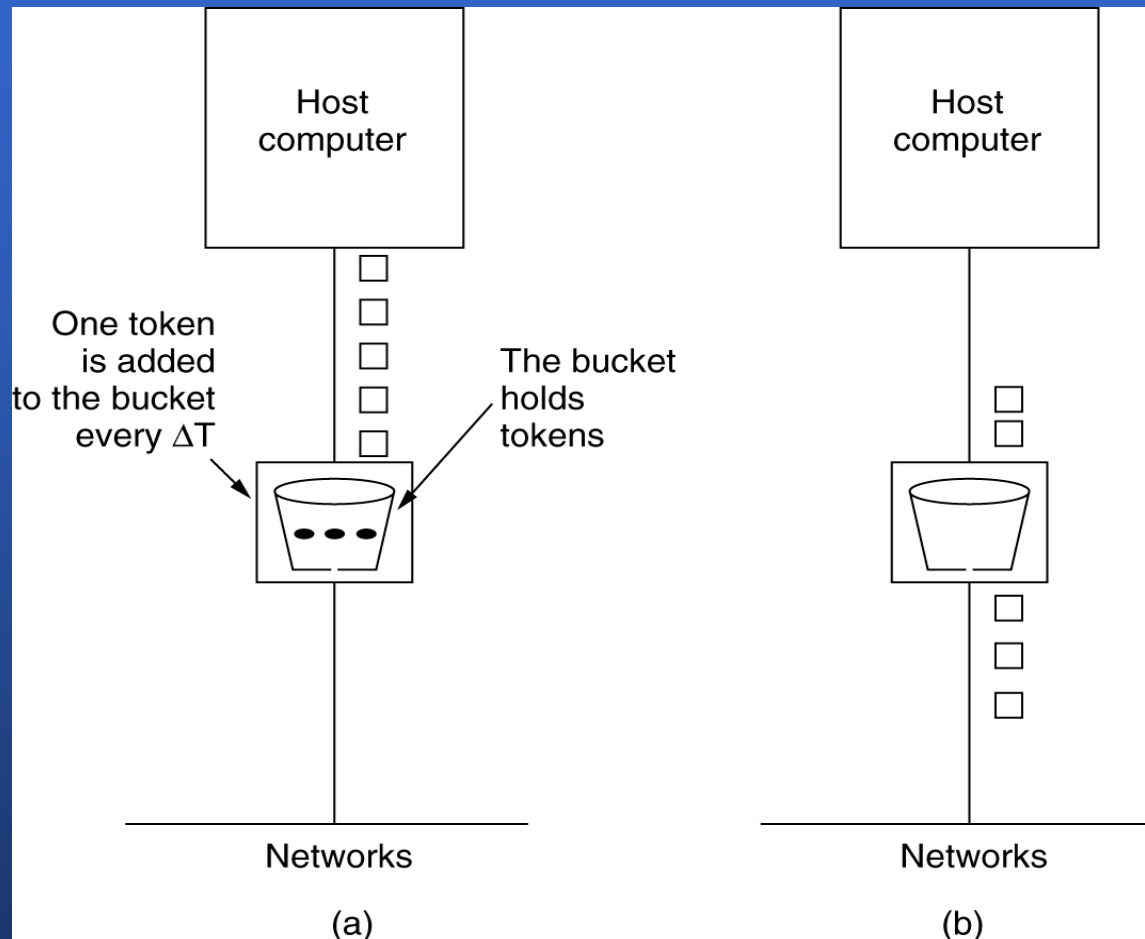


## 5.4.2 Quality Of Service: Techniques for QoS

- (a) Input to a leaky bucket.
- (b) Output from a leaky bucket. Output from a token bucket with capacities of
- (c) 250 KB,
- (d) 500 KB,
- (e) 750 KB,
- (f) Output from a 500KB token bucket feeding a 10-MB/sec leaky bucket.



## 5.4.2 Quality Of Service: Techniques for QoS



### The Token Bucket Algorithm

(a) Before.

(b) After.

## 5.4.2 Quality Of Service: Techniques for QoS

- Token Bucket Algorithm
  - Hold tokens
  - Generate a token every  $\Delta T$  sec
  - Can be used to **smooth traffic** between routers and to **regulate host output**
  - Differences (vs. leaky bucket algorithm):
    - Provides a **different kind of traffic shaping**  $\leftrightarrow$  a rigid output pattern
    - **Throw away tokens** when the bucket fills up but never discards packets  $\leftrightarrow$  **discards packets** when the bucket fills up.



## 5.4.2 Quality Of Service: Techniques for QoS

- Token Bucket Algorithm
  - **Exercise**: Calculate the length of the maximum rate burst (S):
    - $C + pS = MS \rightarrow S = C / (M - p)$ 
      - C bytes – token bucket capacity
      - P bytes/sec – token arrival rate
      - M bytes/sec – maximum output rate
      - C+pS bytes – maximum output burst
      - MS – Maximum burst byte in S seconds
    - See fig.5-33(c)-(f)



## 5.4.2 Quality Of Service: Techniques for QoS

- Resource reservation
  - Bandwidth
  - Buffer space
  - CPU cycles



## 5.4.2 Quality Of Service: Techniques for QoS

- Admission Control

- Flows must be described accurately in terms of **specific parameters** that can be negotiated for many parties may be involved in the flow negotiation.
- An example of flow specification

| Parameter           | Unit      |
|---------------------|-----------|
| Token bucket rate   | Bytes/sec |
| Token bucket size   | Bytes     |
| Peak data rate      | Bytes/sec |
| Minimum packet size | Bytes     |
| Maximum packet size | Bytes     |



## 5.4.2 Quality Of Service: Techniques for QoS

- **Proportional routing**
  - To split the traffic for each destination over multiple paths.
  - A simple method is to divide the traffic equally or in proportion to the capacity of the outgoing links.

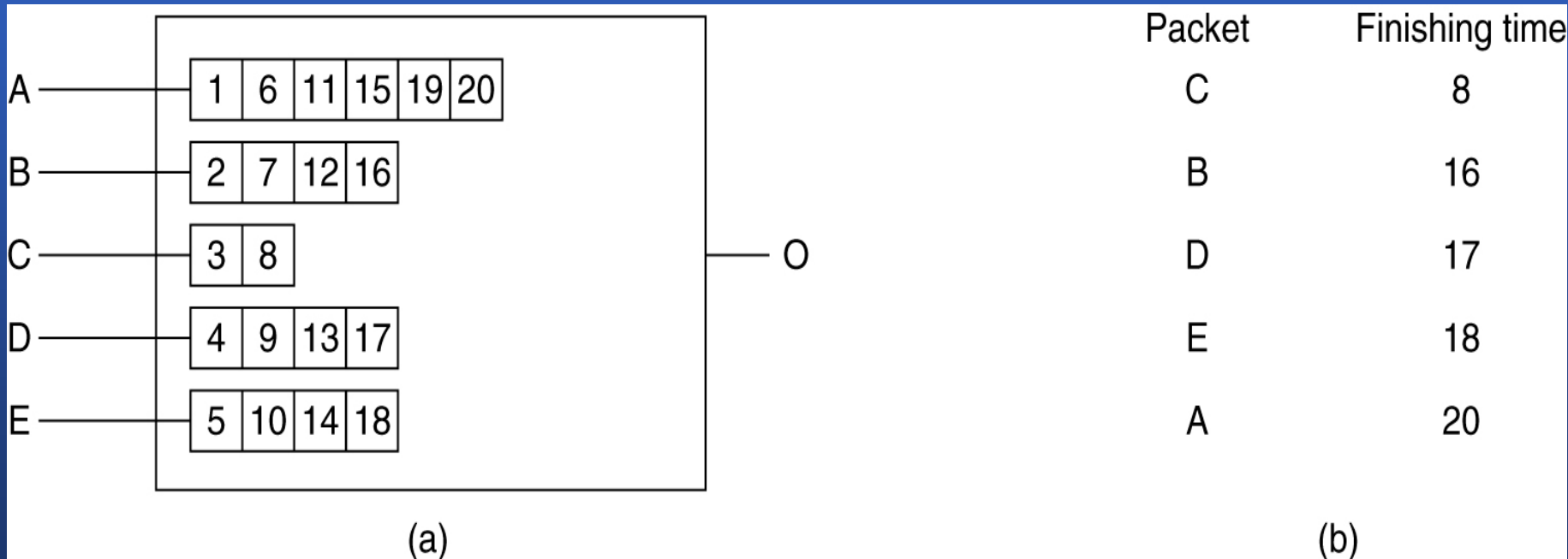




# Quality Of Service: Techniques for QoS

- Packet scheduling:

- fair queuing(公平队列) v.s. weighted fair queuing (加权公平队列) → packet-by-packet or byte-by-byte round robin

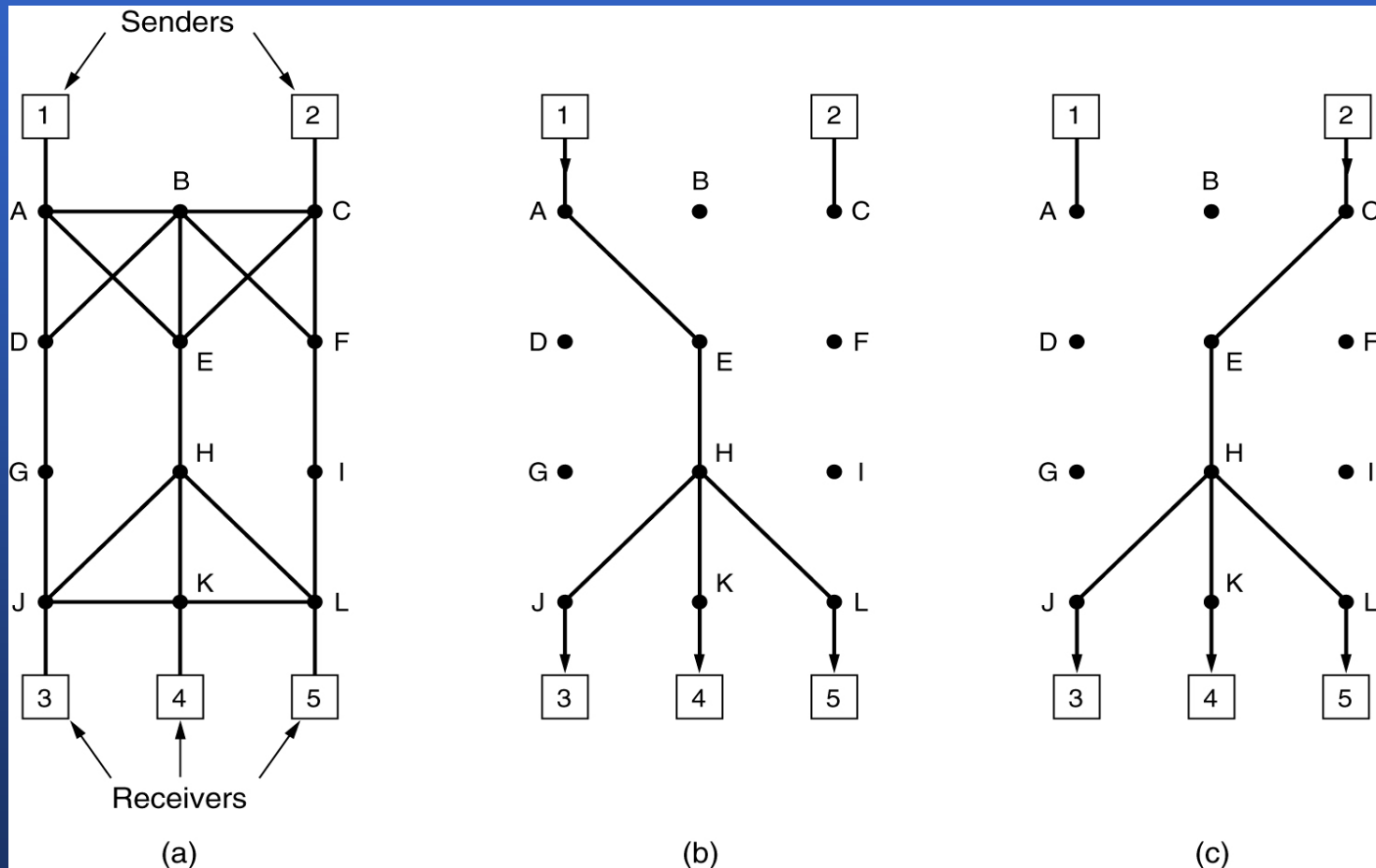


(a) A router with five packets queued for line O.

(b) Finishing times for the five packets.

## 5.4.3 Quality Of Service: Integrated services

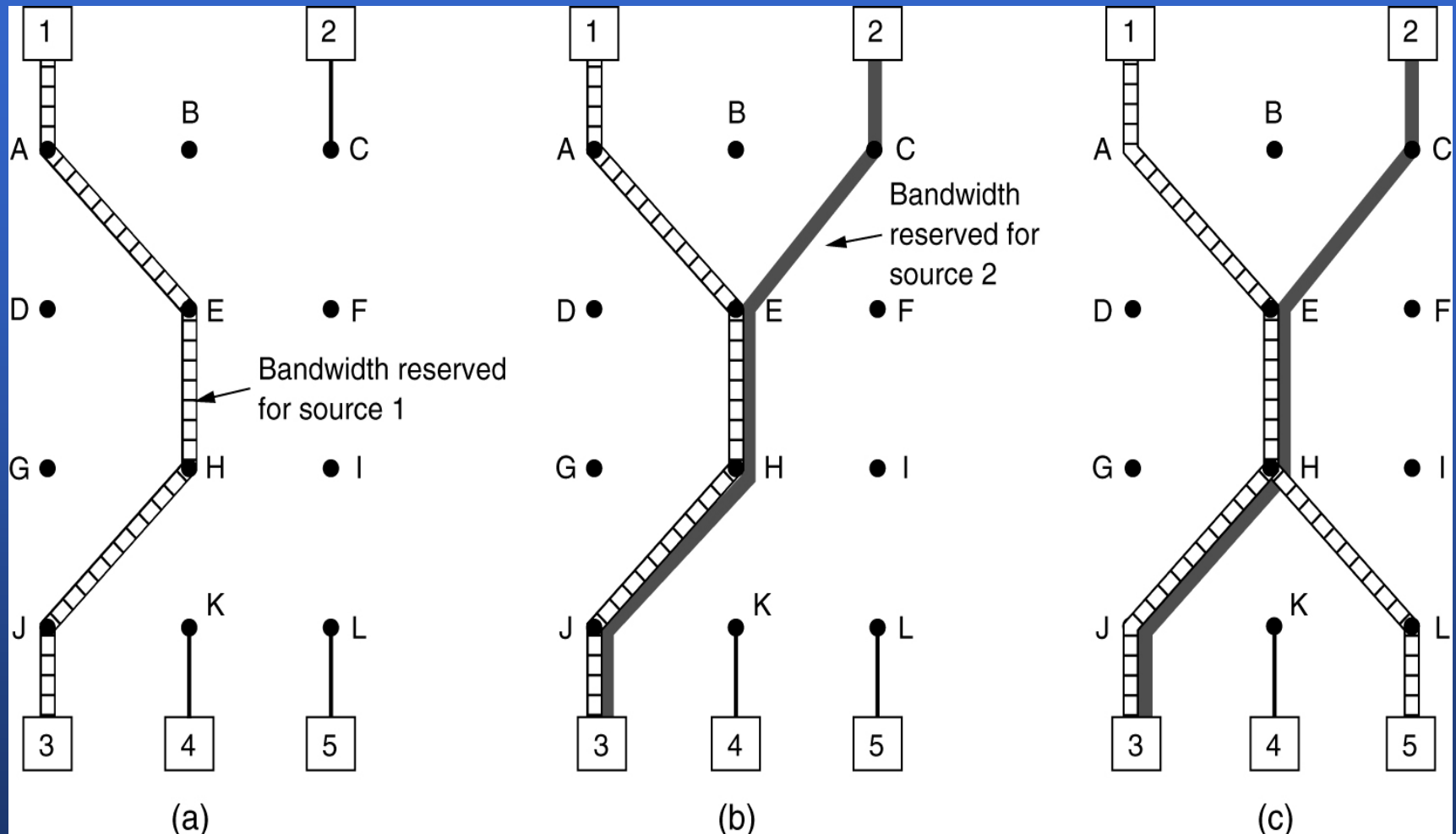
**RSVP**— the Resource reSerVation Protocol (资源重复利用协议)



(a) A network, (b) The multicast spanning tree for host 1.

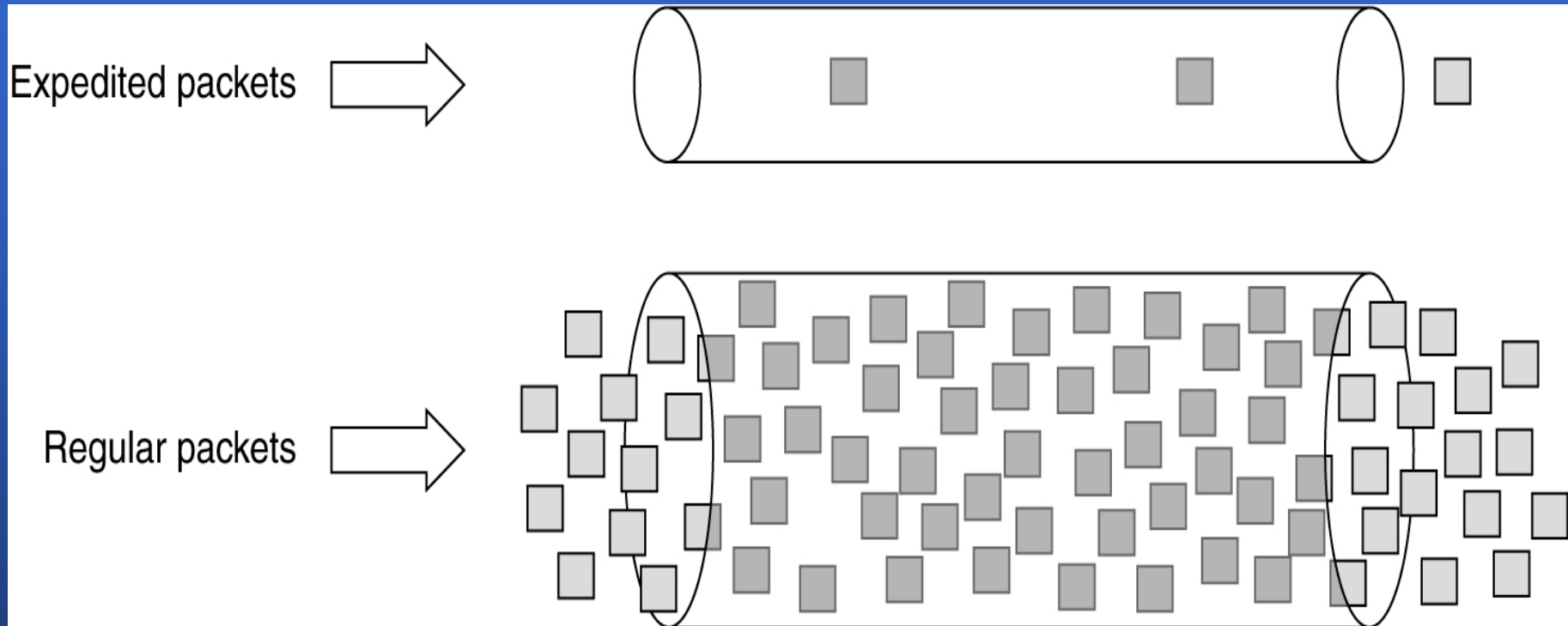
(c) The multicast spanning tree for host 2.

## 5.4.3 Quality Of Service: Integrated services



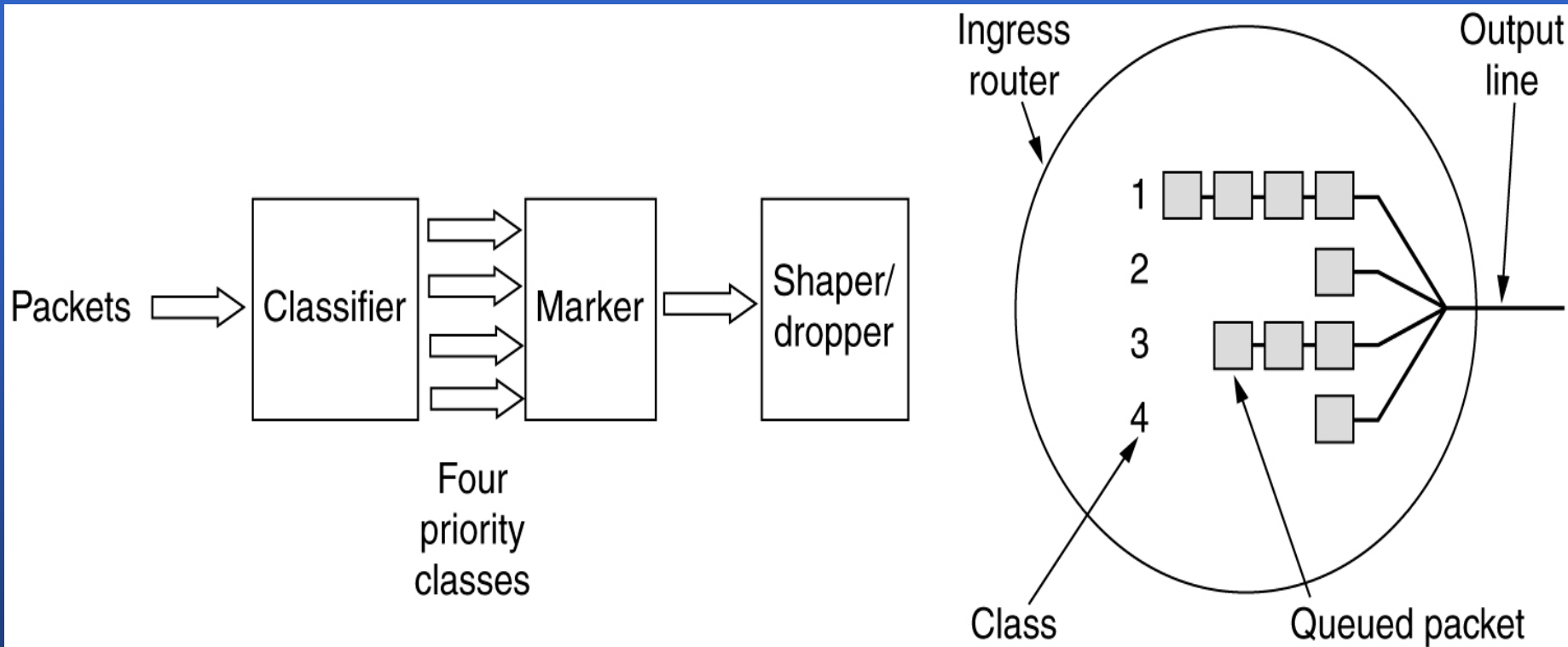
(a) Host 3 requests a channel to host 1. (b) Host 3 then requests a second channel, to host 2. (c) Host 5 requests a channel to host 1.

## 5.4.4 Quality Of Service: Differential services (\*)



Expedited (畅通的, 迅速的) packets experience a traffic-free network. (RFC 3246)

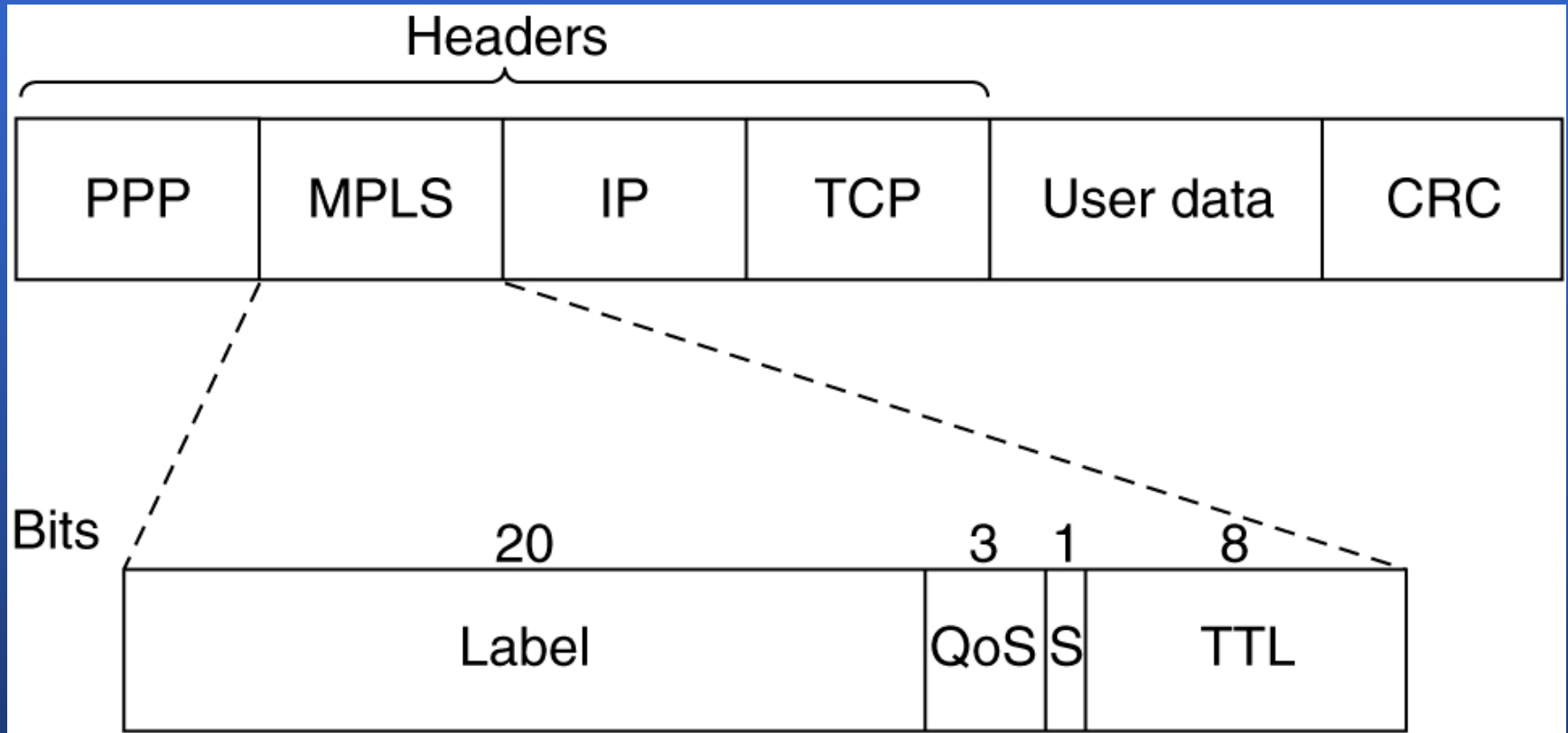
## 5.4.4 Quality Of Service: Differential services(\*)



Assured Forwarding (RFC2597)

Class-based QoS vs Flow-based QoS

## 5.4.5 Quality Of Service: Label switching and MPLS



Transmitting a TCP segment using IP, MPLS, and PPP.

(MPLS—Multi-Protocol **Label** Switching)

## 5.5 Internetworking (网络互联)

- 互联网(internet)
  - 各种不同的网络(及协议)将长期共存, 原因:
    - 不同网络的安装基础很雄厚, 不断发展. 如, IBM仍在开发 SNA;
    - 价格便宜, 决策权层层下降;
    - 不同网络采用完全不同的技术, 硬件发展, 新的软件诞生
  - 网络互联实例-[see fig 5-33]

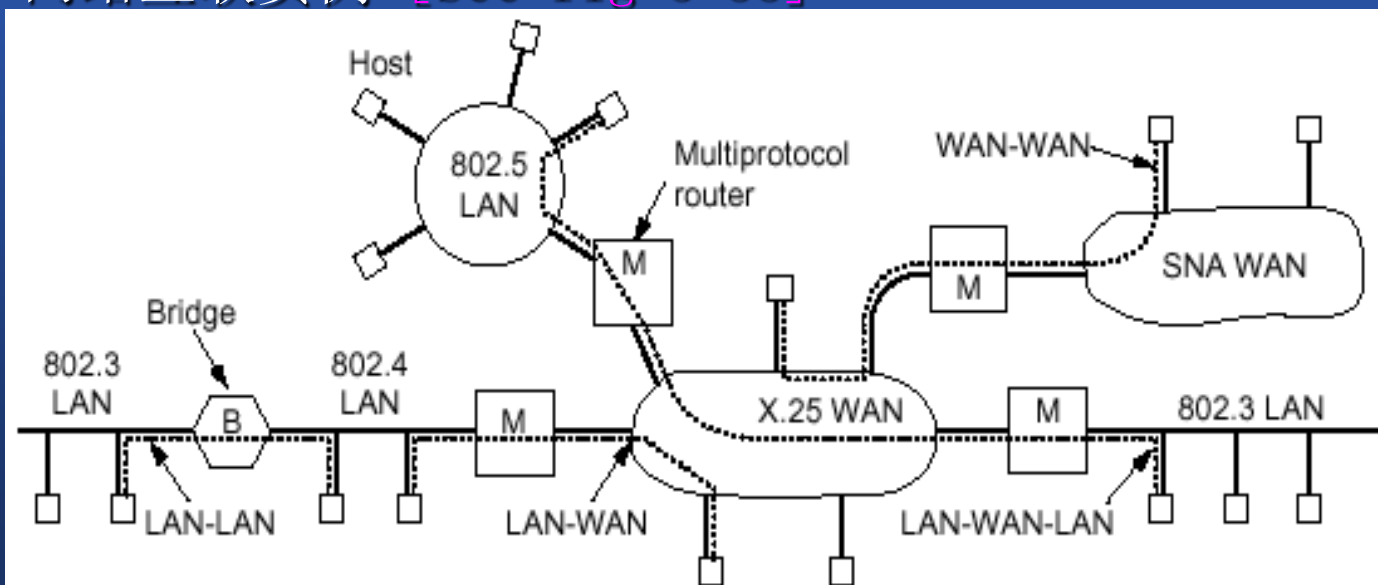


Fig. 5-33. Network interconnection.



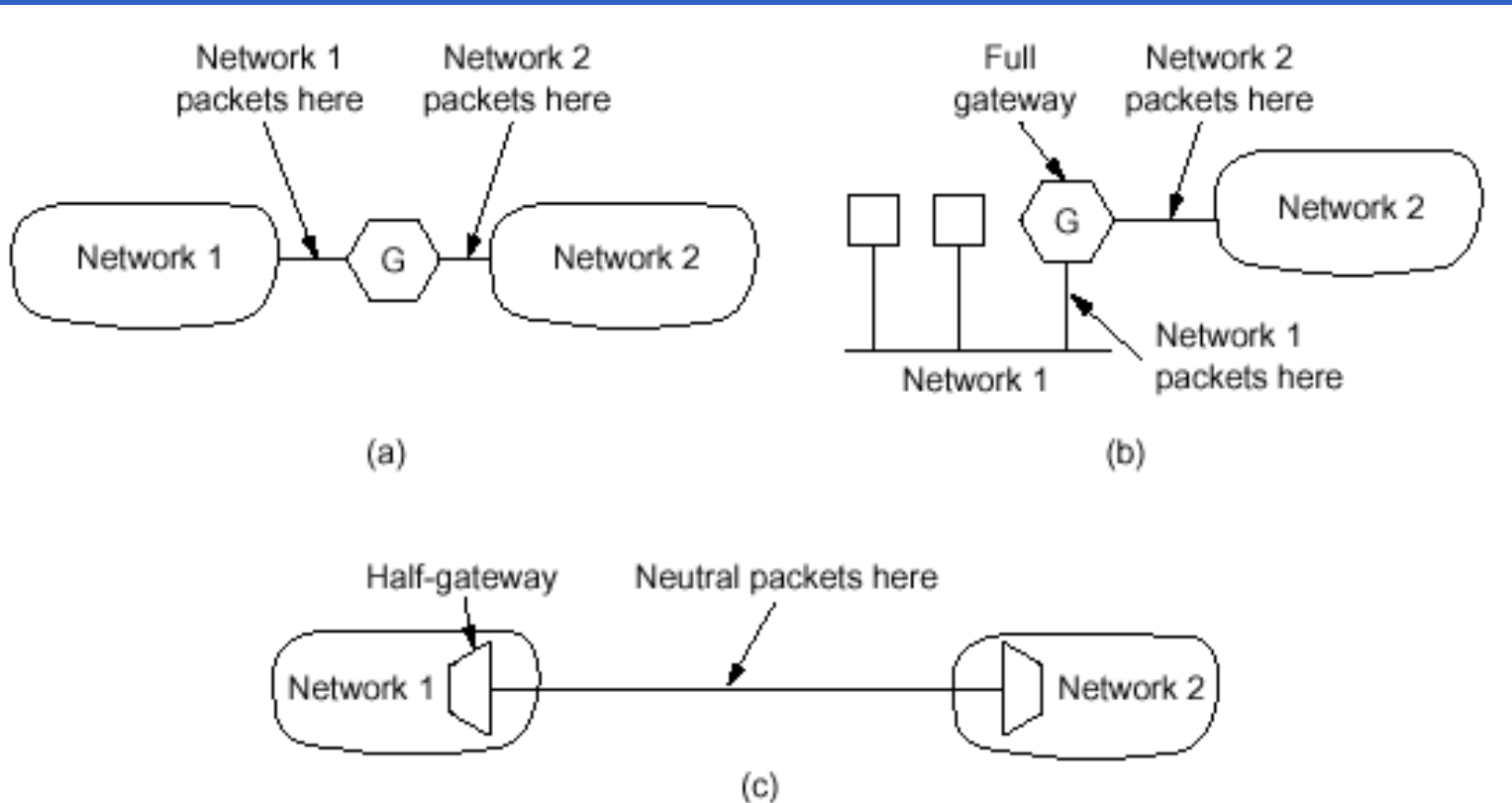
# 5.5 Internetworking

- 常用名词

- 中继器(repeater) (第一层)
- 网桥(bridge) (第二层)
- 多协议路由器(multi-protocol router) (第三层)
- 传输网关(transport gateway) (第四层)
- 应用程序网关(application gateway) (第四层以上)
- 网关(gateway)



# 5.5 Internetworking



**Fig. 5-34.** (a) A full gateway between two WANs. (b) A full gateway between a LAN and a WAN. (c) Two half-gateways.

## 5.5 Internetworking

- 网桥和路由器的特性区别:
  - 纯网桥的关键特征:检查数据链路帧的帧头,但并不查看修改帧中的网络层分组
  - 路由器:检查分组头,根据所发现的地址做出决定.



# 5.5.1 How Networks Differ

- 网络许多不同之处的某些方面-[see fig 5-35]
  - 一个分组在到达目的地网络前必须经过一个或多个外部网络时, 在网络接口处出现问题
  - 不同网络间使用的不同最大分组值
  - 差错控制 流控制和拥塞控制在不同网络上也是不同的.

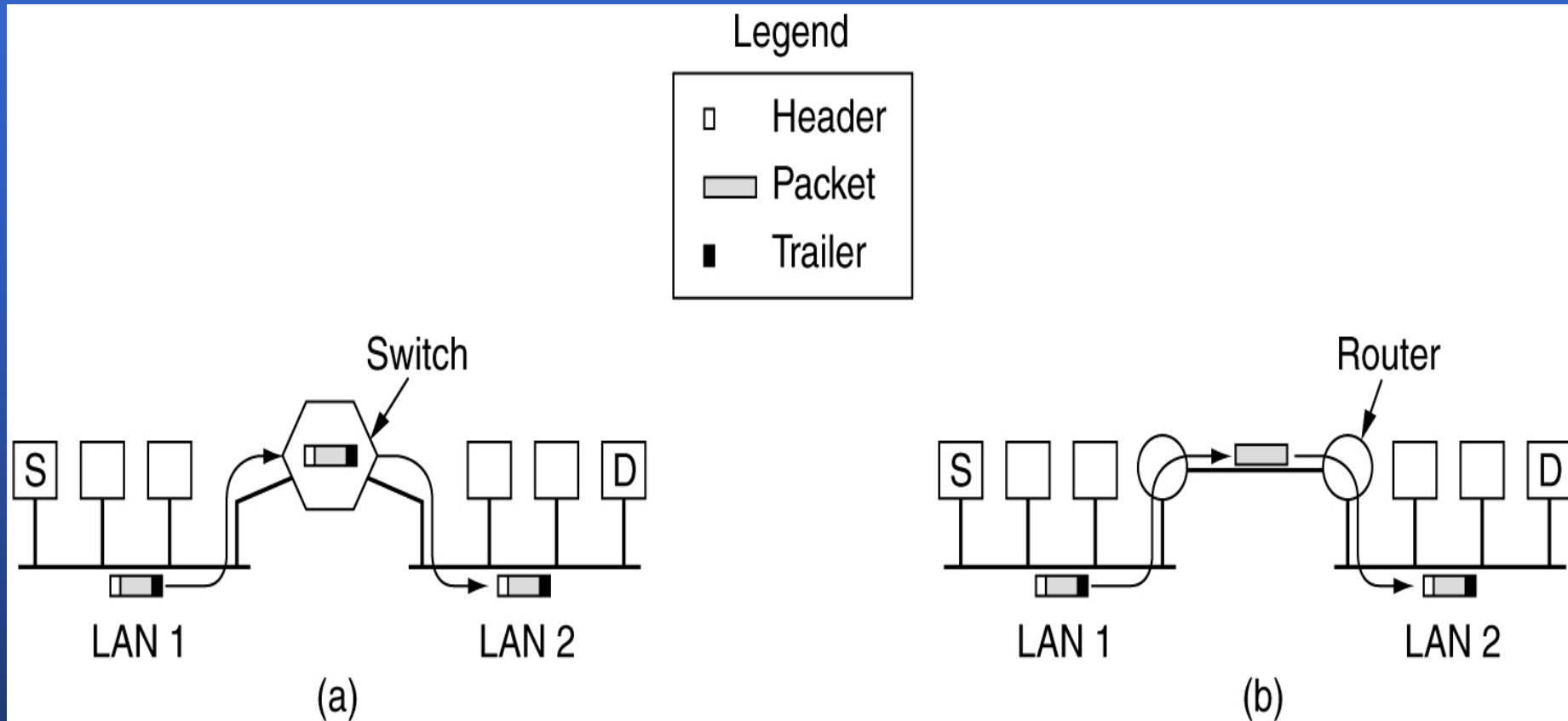


## 5.5.1 How Networks Differ

| Item               | Some Possibilities                                 |
|--------------------|----------------------------------------------------|
| Service offered    | Connection-oriented versus connectionless          |
| Protocols          | IP, IPX, CLNP, AppleTalk, DECnet, etc.             |
| Addressing         | Flat (802) versus hierarchical (IP)                |
| Multicasting       | Present or absent (also broadcasting)              |
| Packet size        | Every network has its own maximum                  |
| Quality of service | May be present or absent; many different kinds     |
| Error handling     | Reliable, ordered, and unordered delivery          |
| Flow control       | Sliding window, rate control, other, or none       |
| Congestion control | Leaky bucket, choke packets, etc.                  |
| Security           | Privacy rules, encryption, etc.                    |
| Parameters         | Different timeouts, flow specifications, etc.      |
| Accounting         | By connect time, by packet, by byte, or not at all |

**Fig. 5-35.** Some of the many ways networks can differ.

## 5.5.2 How Networks Can Be Connected



(a) Two Ethernet networks connected by a switch.

(b) Two Ethernet networks connected by routers.

## 5.5.3 Concatenated Virtual Circuits (\*)

- 两种常用的网络互联方式
  - 面向连接的连锁虚电路
  - 数据报网络互联方式
- 连锁虚电路原理-[see fig 5-36]
  - 采用连锁虚电路的网络互联的工作模式
  - 这种方法的关键-
  - 全网关建立的连接/半网关建立的连接
  - 连锁虚电路也常用于传输层

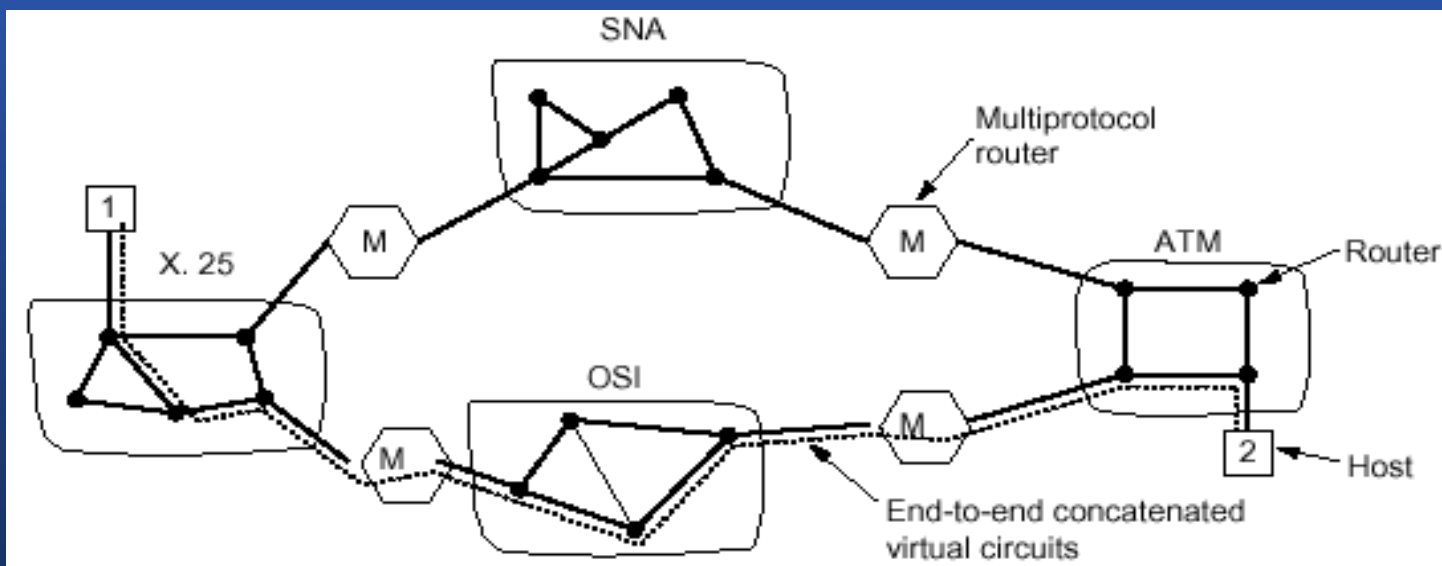


Fig. 5-36. Internetworking using concatenated virtual circuits.



## 5.5.4 Connectionless Internetworking (\*)

- 数据报模式工作原理-[see fig 5-37]
- 关键：
  - 格式转换
  - 地址问题
    - » 给每个OSI主机分配一个32位因特网地址
    - » 设计一个通用互联网分组，让每个路由器都能识别它（IP的目的地）

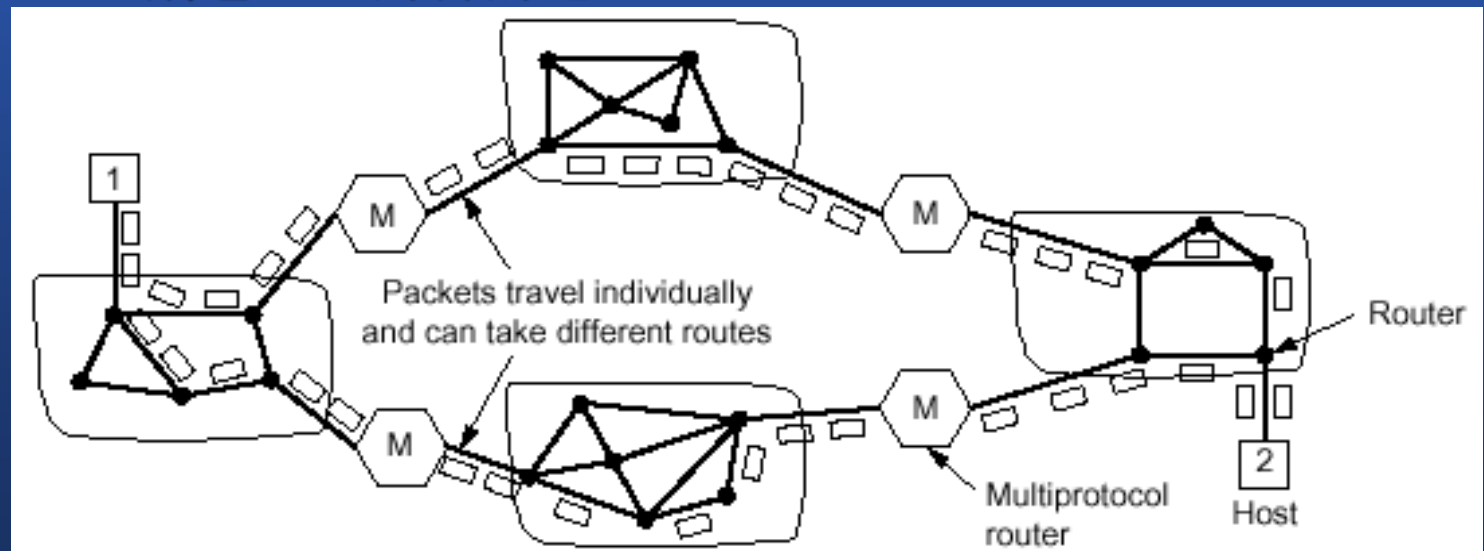


Fig. 5-37. A connectionless internetwork.

## 5.5.4 Connectionless Internetworking (\*)

- 实现网络互联的两种方法总结：
  - 连锁虚电路优点
    - » 可以预先保留
    - » 可以保证顺序发送
    - » 可以使用较短的信息头
    - » 可以避免由延迟重复分组造成的错误
  - 连锁虚电路缺点
    - » 需要给每个打开的连接分配表空间，没有后备路由绕过拥塞区域，沿途路由器崩溃的脆弱性
    - » 当所涉及到的网络中有一个不可靠数据报网络时，实现很困难



## 5.5.4 Connectionless Internetworking (\*)

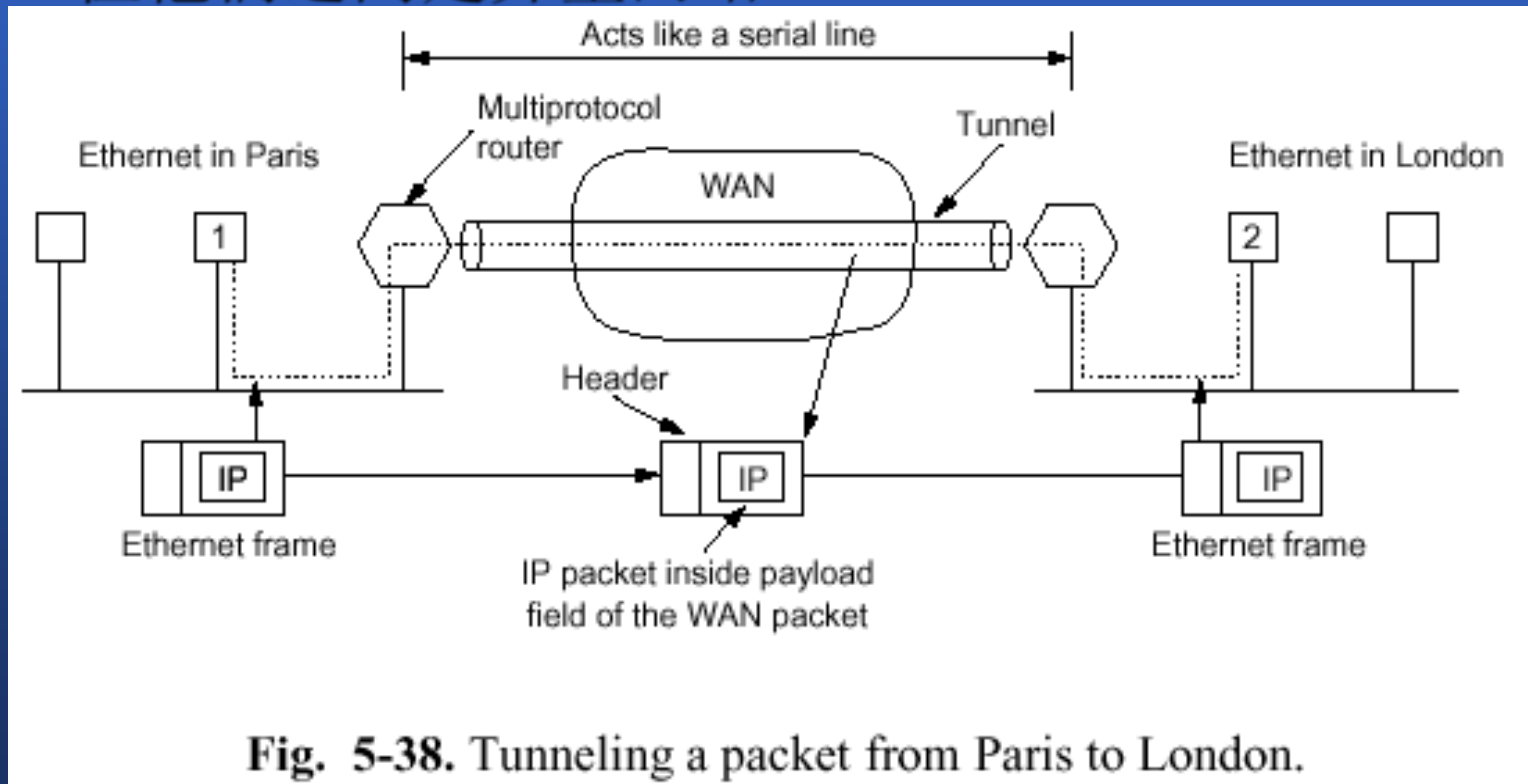
- 实现网络互联的两种方法总结：
  - 数据报方法特性
    - 更有可能造成拥塞，但也更能适应拥塞
    - 更可能适应拥塞
    - 路由器崩溃时的健壮性
    - 需要更长的信息头
    - 各种适应性路由选择算法也适用于互联网
  - 数据报方法的主要优点：可用于覆盖网络互联中内部不使用虚电路的子网



# 5.5.5 Tunneling

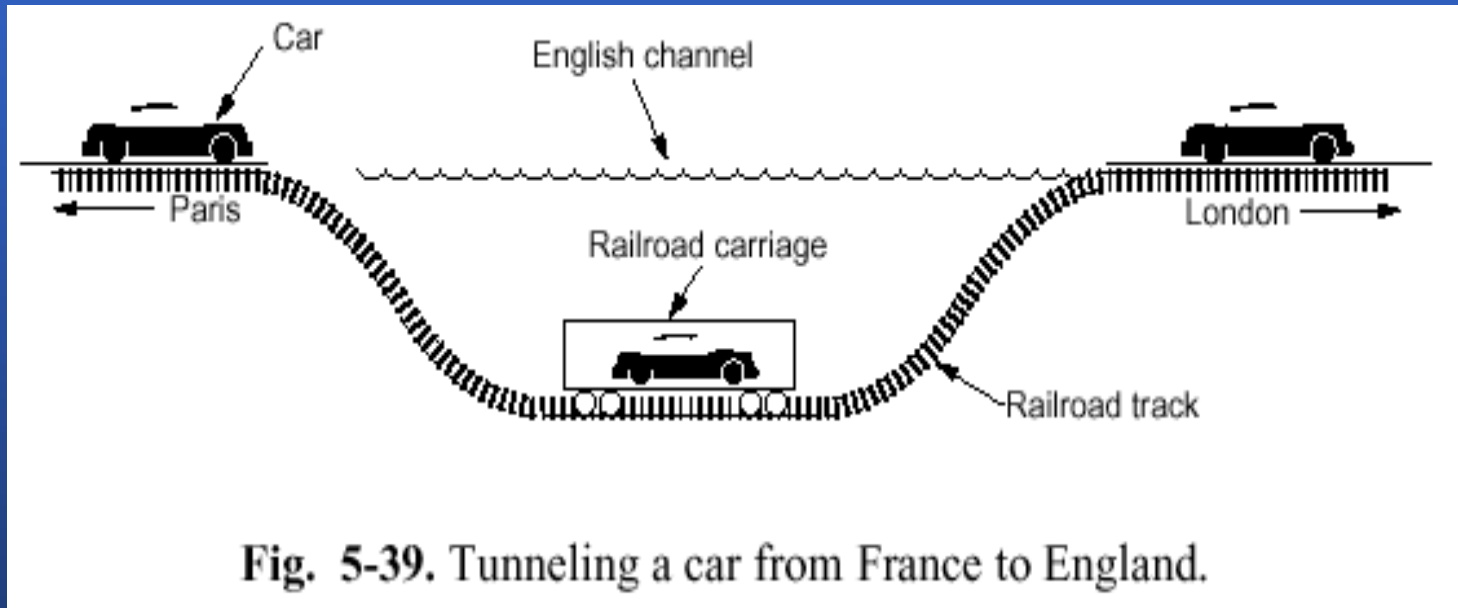
- 特例-[see fig 5-38]

源端主机和目的端主机处于同种类型的网络中，但他们之间是异型网络



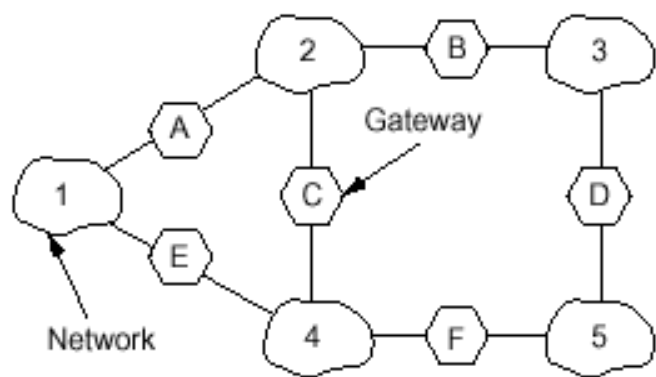
# 5.5.5 Tunneling

- 隧道(tunneling)
  - 概念的理解 – [see fig 5-39]

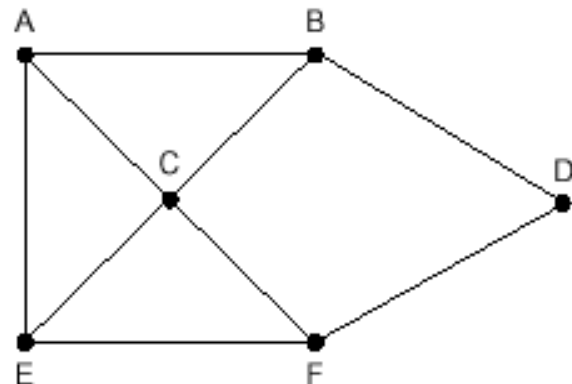


## 5.5.6 Internetwork Routing

- 路由选择算法：
  - 互联网和互联网图形-[see fig 5-40]
  - 两级路由选择算法
    - 内部网关协议(interior gateway protocol)
    - 外部网关协议(exterior gateway protocol)
    - 自治系统AS(autonomous system) (互联网中的每个网络)
- 互联网路由与内部互联网路由的区别
  - 互联网需要国界
  - 费用



(a)



(b)

Fig. 5-40. (a) An internetwork. (b) A graph of the internetwork.

## 5.5.7 Fragmentation (\*)

- 分组的最大长度受到限制的原因(六个方面)
  - 硬件
  - 操作系统
  - 协议
  - 遵从的国际（国内）标准
  - 希望减少重传带来的错误
  - 希望防止分组占用信道时间过长





## 5.5.7 Fragmentation (\*)

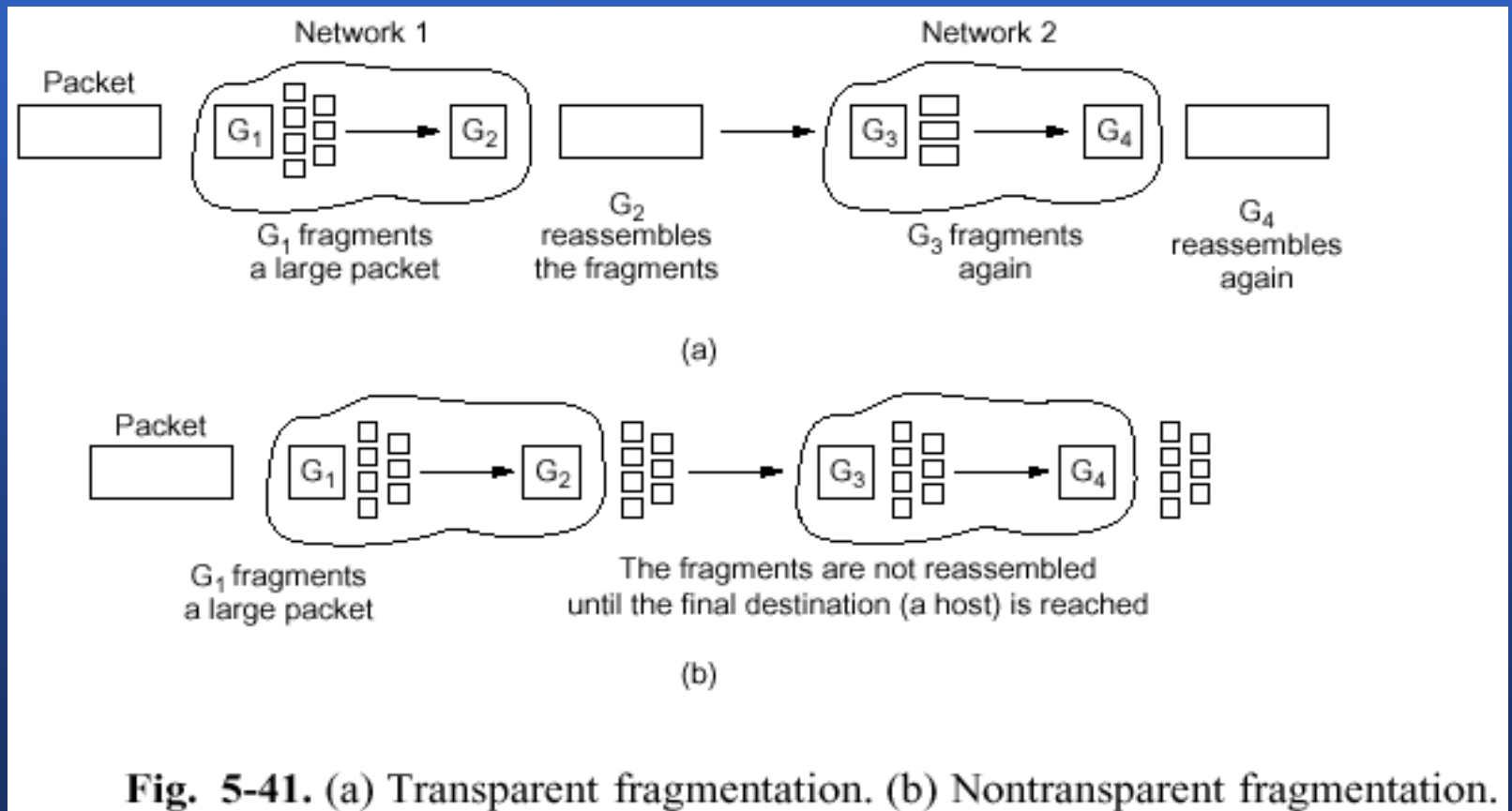
- 分段的引出
  - 一个大分组想穿过一个最大分组长度过小的网络时,出现了问题;
  - 解决的办法是将分组划分为段(fragment):
  - 将分组重组为原来的分组方法:
    - 透明分段-[see fig 5-41a]
    - 不透明分段-[see fig 5-41b]



## 5.5.7 Fragmentation (\*)

— 将分组重组为原来的分组方法:

- 透明分段-[see fig 5-41a]
- 不透明分段-[see fig 5-41b]

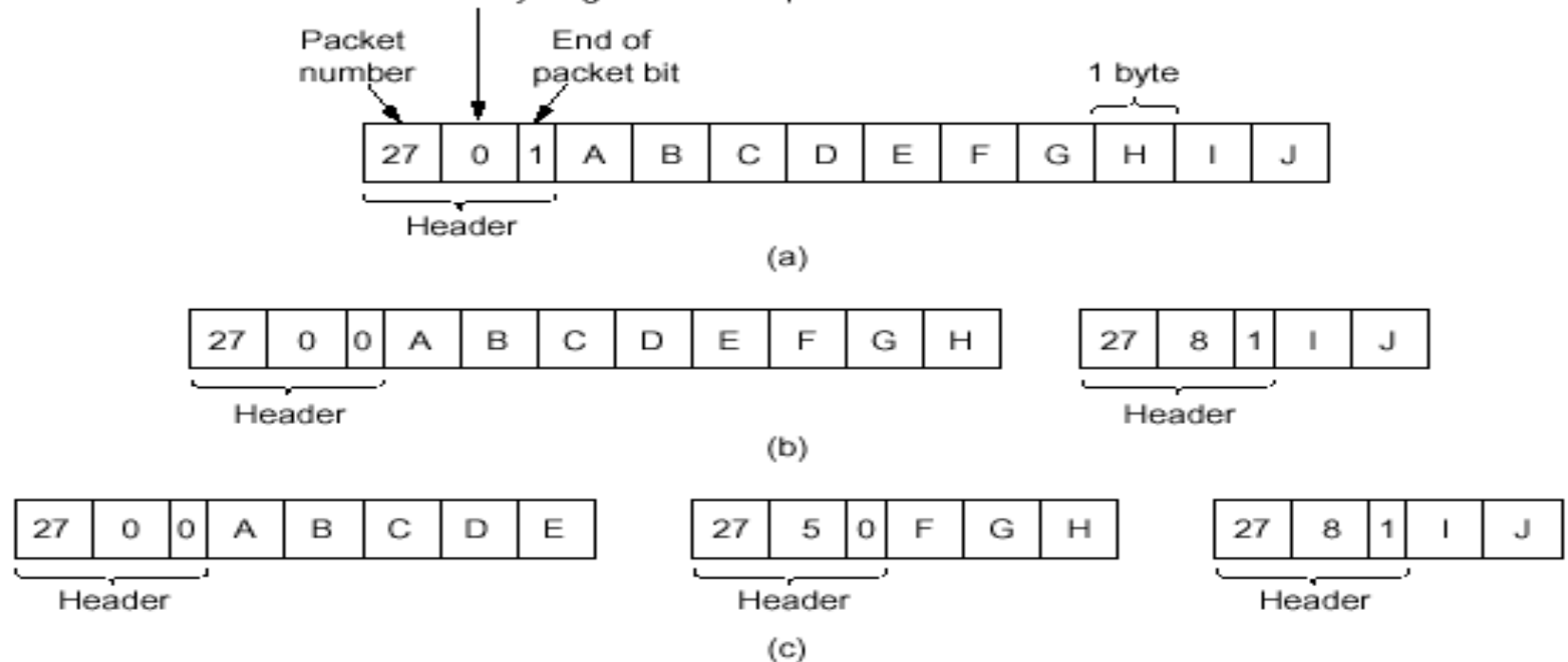


# 5.5.7 Fragmentation (\*)

## – 分段的编码方式

- 采用树结构
- 另一种方法-[see fig 5-42]

Number of the first elementary fragment in this packet



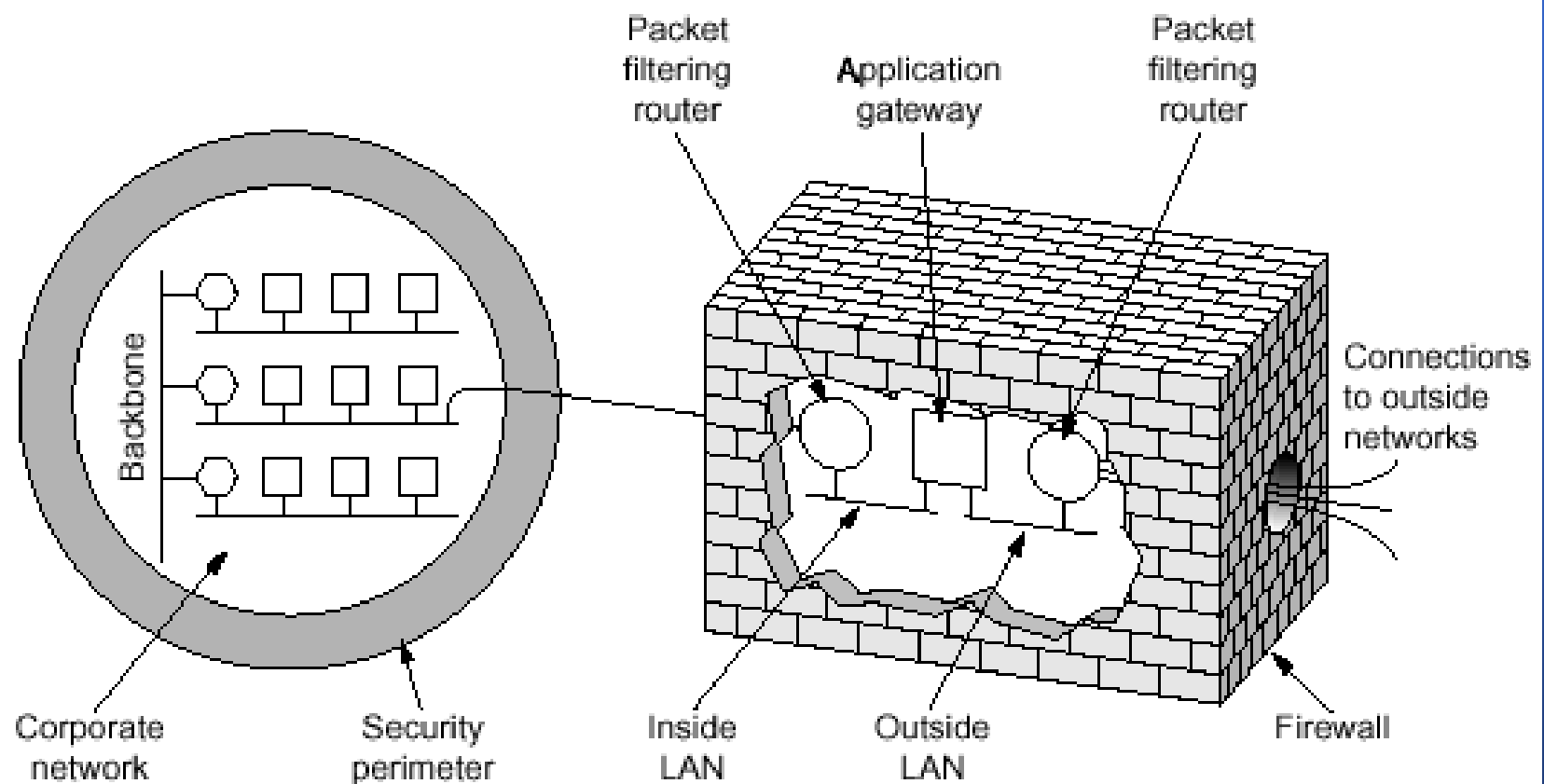
**Fig. 5-42.** Fragmentation when the elementary data size is 1 byte. (a) Original packet, containing 10 data bytes. (b) Fragments after passing through a network with maximum packet size of 8 bytes. (c) Fragments after passing through a size 5 gateway.

## 5.5.8 Firewall (\*)

- 解决信息外泄和信息渗入的危险的两种办法:加密、防火墙
- 防火墙 (firewall) - [see fig 5-43]
  - 组成:两个用作分组筛选的路由器和一个应用程序网关
  - 分组筛选器(packet filter)
    - 有额外功能的标准路由器
      - 额外功能:用来检查每个进出的分组
    - 由系统管理员配置的表驱动
  - 应用程序网关



## 5.5.8 Firewall (\*)



**Fig. 5-43.** A firewall consisting of two packet filters and an application gateway.

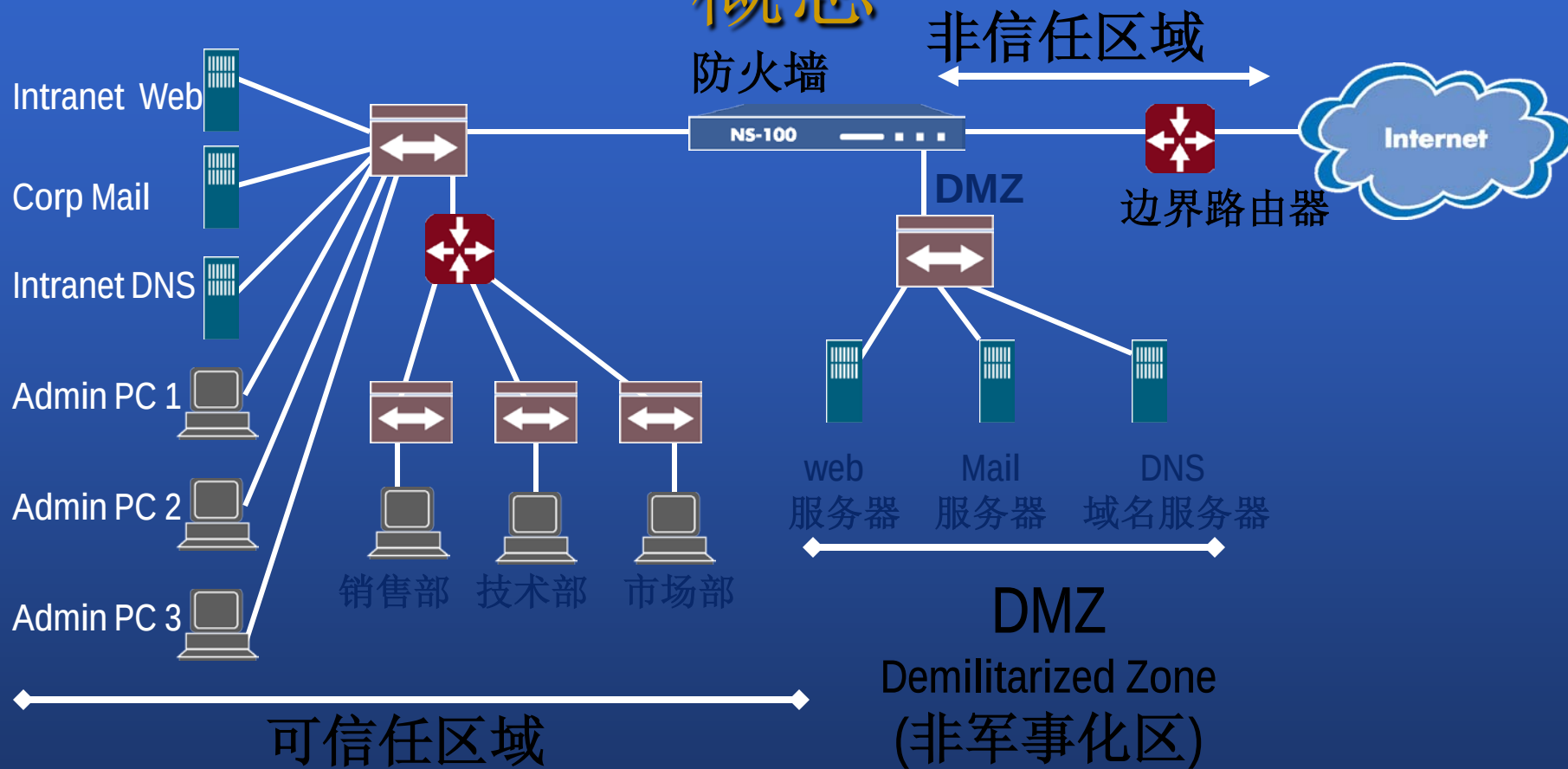
# 防火墙的基本概念

- Fire Wall:

是指一种将内部网和公众网络（如 **Internet**）分开的方法，它实际上是一种隔离技术。它能允许你“同意”的人和数据进入你的网络，同时将你“不同意”的人和数据拒之门外，最大限度地阻止黑客访问你的网络。



# 概念





# 概念

- 防火墙

- 增强内部网络安全性的系统。它防止外部用户非法使用内部网络的资源，控制内部用户对外部资源的访问
- 防火墙只允许授权的数据通过，其本身也必须能够免于渗透

- 边界路由器

- 第三层路由器，提供对外网（Internet）的连接
- 主要包括访问控制列表以提供额外的保护

- DMZ 区（Militarized Zone）

- 被防火墙或一些访问控制保护起来，但仍然对外公开的区域，相对于不公开的区域，它在安全方面有不足



# 概念

- 可信任区域 (Trust)
  - 被防火墙或其它访问控制保护的区域，它对外是不公开的
- Internet (非信任区, Untrust)
  - 不受防火墙保护的外部网络



# 防火牆的种类

- 包过滤（Packet filter）
- PROXY（Application Proxy）
- 基于状态检测（Stateful packet inspection）
- 混合型（Hybrid Firewalls）



# 什么是防火墙?

- 增强内部网络安全性的一套安全系统。它防止外部用户非法使用内部网络的资源,控制内部用户对外部资源的访问
- 四种类型:
  1. 包过滤
  2. 应用代理服务器
  3. 状态检测
  4. 混合型防火墙 (例如 NetScreen 防火墙)



# 什么是防火墙

- 包过滤 (**Packet filter**)

- 根据数据包头中的各项信息来控制站点与站点、站点与网络、网络与网络间的相互访问，但不能控制传输数据的内容。
- 包过滤检查：IP源地址和目的地址，源端口和目的端口，协议类型，ICMP消息类型，TCP报头的ACK位、序列号、确认号、IP校验和、分割偏移等。
- 大多数基于UDP的RPC服务是非常危险的，除了DNS、NTP、syslog等必须服务外，其他RPC服务要全部阻止。
- 包过滤只能访问包头中有限信息，不能保持与传输及应用相关的状态信息，对信息的处理能力非常有限。
- 一些协议不适合用数据包过滤，如基于RPC的“r”命令





# 什么是防火墙？

## • 应用代理（**Application proxy**）

- 应用代理为一种特定的服务提供代理服务，不但转发流量而且对应用层协议做出解释，起到外网与内网进行IP通讯的中间转接作用。
- 代理的主要特点是有状态性，能完全提供与应用相关的状态和部分传输方面的状态，也能处理和管理信息。
- 缺点是有限的连接性和有限的技术。只为特定的服务提供代理服务，不能为RPC、TALK和其他一些基于通用协议族的服务提供代理。
- 不能有效检查底层的信息，会明显降低系统性能



# 什么是防火墙?

- 包状态检测 (**Stateful packet examination**)
  - 状态检查模块访问和分析从各层次得到的数据, 存储和更新状态数据和上下文信息, 为跟踪无连接的协议提供虚拟的会话信息。
  - 当用户访问到达网关操作系统前, 状态监视器要抽取有关数据进行分析, 结合网络配置和安全策略作出接纳、拒绝、鉴定或加密等决定。
  - 检测模块支持多种协议和应用程序, 可以很容易地实现应用和服务的扩充。
  - 状态监视器能检测RPC和UDP之类的端口信息, 包过滤和代理网关不支持此类端口。
  - 提供比包过滤方式更高级别的安全性, 比应用代理速度快得多, 但是不会提供应用代理那么详细的应用





# 什么是防火墙?

## 4. 混合型防火墙

- 可以实现包过滤、应用代理和状态检测的功能
- NetScreen 使用包状态检测和集成在硬件中的应用代理功能实现安全性能
  - NetScreen还具备 NAT/PAT 和 VPN 的功能
  - NetScreen使用状态检测（状态过滤）和集成于硬件的应用代理技术实现包过滤和状态检测功能



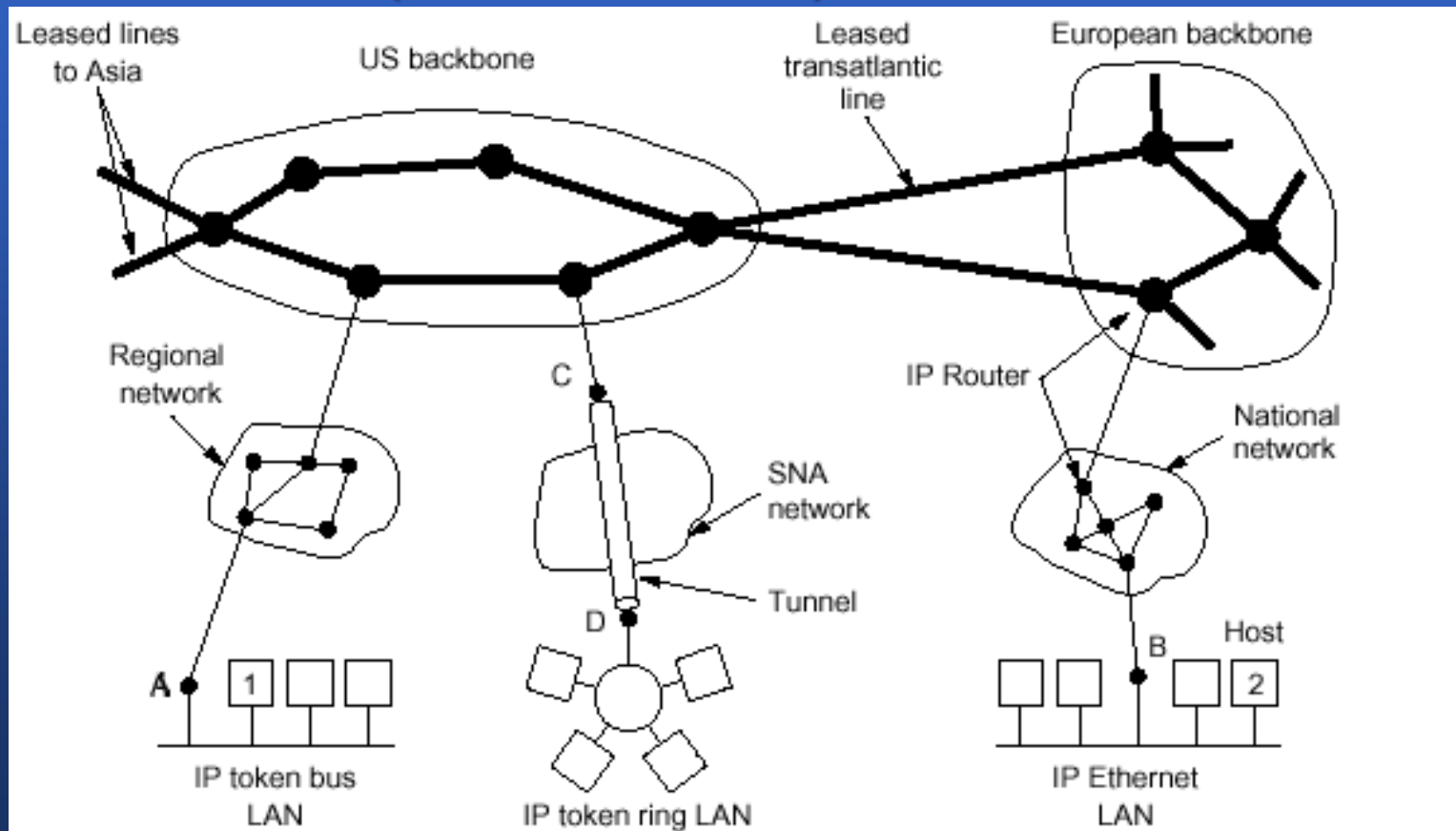
# 防火墙不能抵御什么

- 物理上的问题
  - 断电
  - 物理上的损坏或偷窃
- 人为的因素
  - 内部人员窃取了用户名和密码或者类似的行为
- 病毒
  - 内嵌的自带寻址信息的数据包，防火墙将允许其通过
- 某些恶意的雇员通过防火墙访问
- 防火墙的配置不当



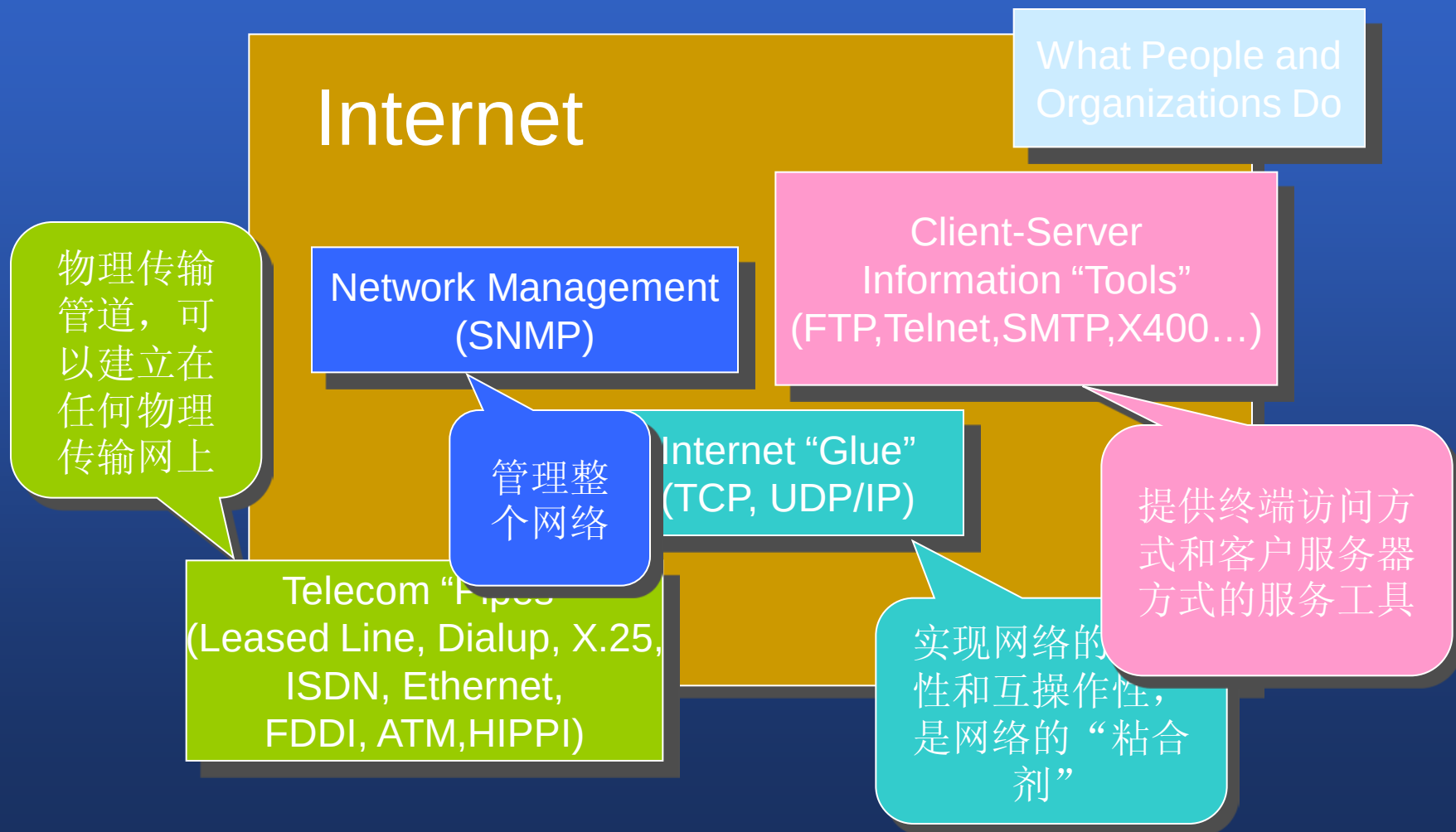
# 5.6 The Network Layer in the Internet

- 自治系统AS-[see fig 5-44]
- 因特网协议IP(Internet Protocol)



**Fig. 5-44.** The Internet is an interconnected collection of many networks.

# Internet体系结构框架



# TCP/IP协议概述

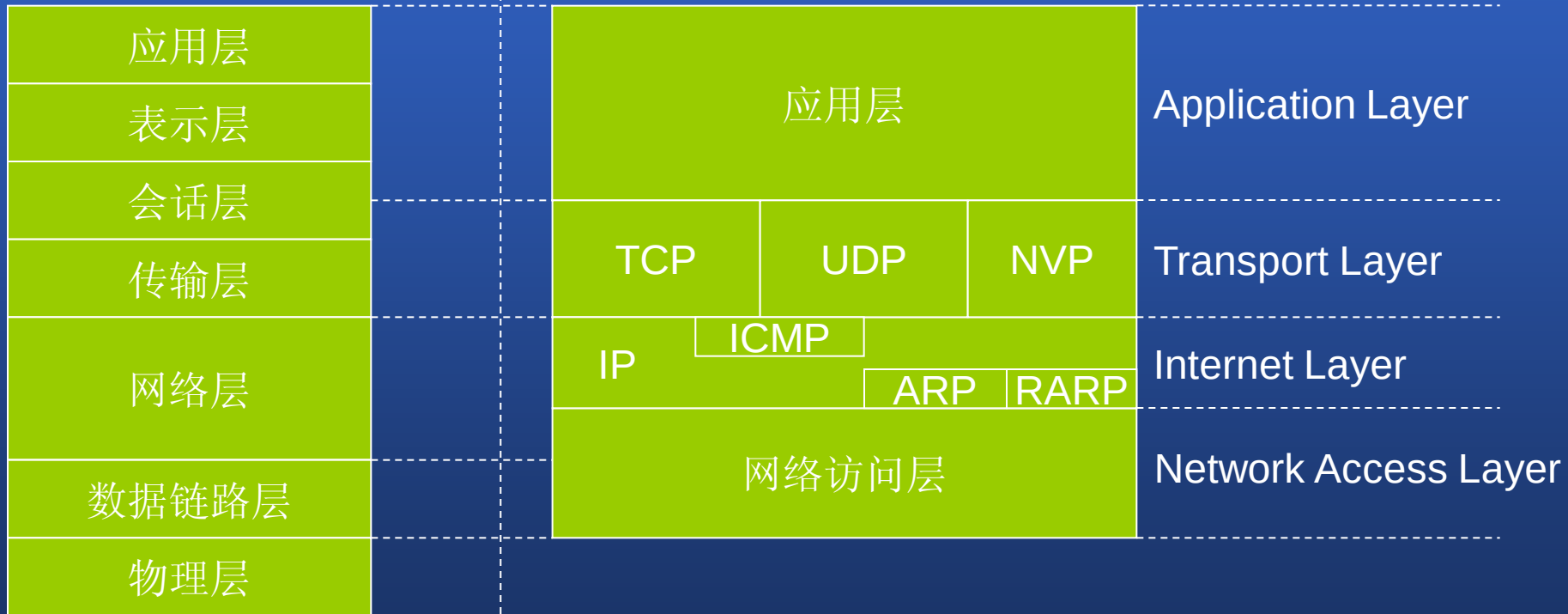
- TCP/IP (Transmission Control Protocol /Internet Protocol) 体系包含了一百多个协议，它虽然不是国际标准，但它的广泛应用，确立了它事实国际标准的地位，也可以成为工业标准。
- 特点：
  - 连接不同系统的技术
  - 开放系统
  - 提供强有力的WAN连接：可路由，为广域网设计的



# TCP/IP与OSI

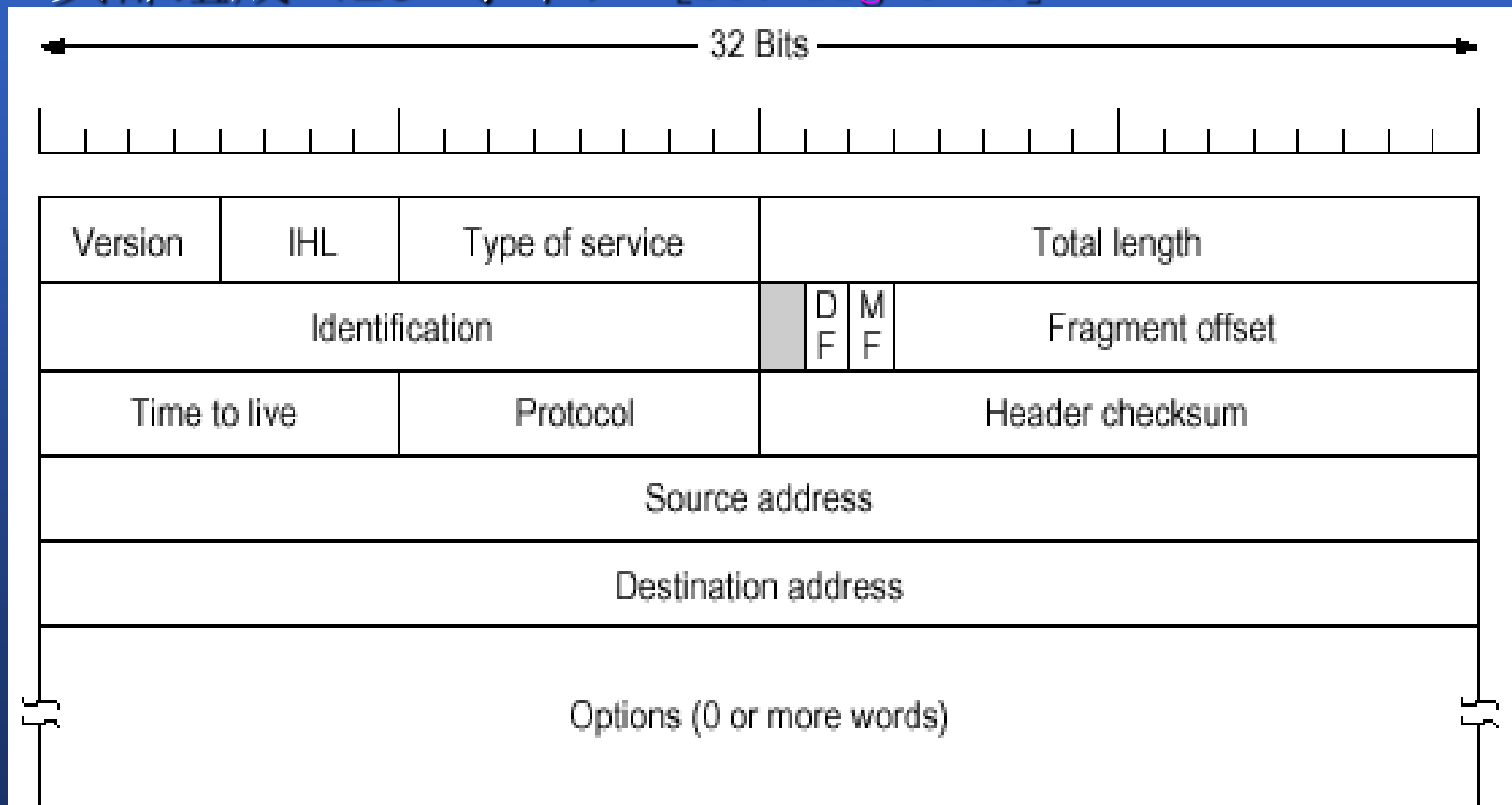
## ISO/OSI

## TCP/IP



# 5.6.1 The IP Protocol

- IP数据报： 头部+正文
- 头部组成（20+字节） - [see fig 5-45]



**Fig. 5-45.** The IP (Internet Protocol) header.



# 5.6.1 The IP Protocol

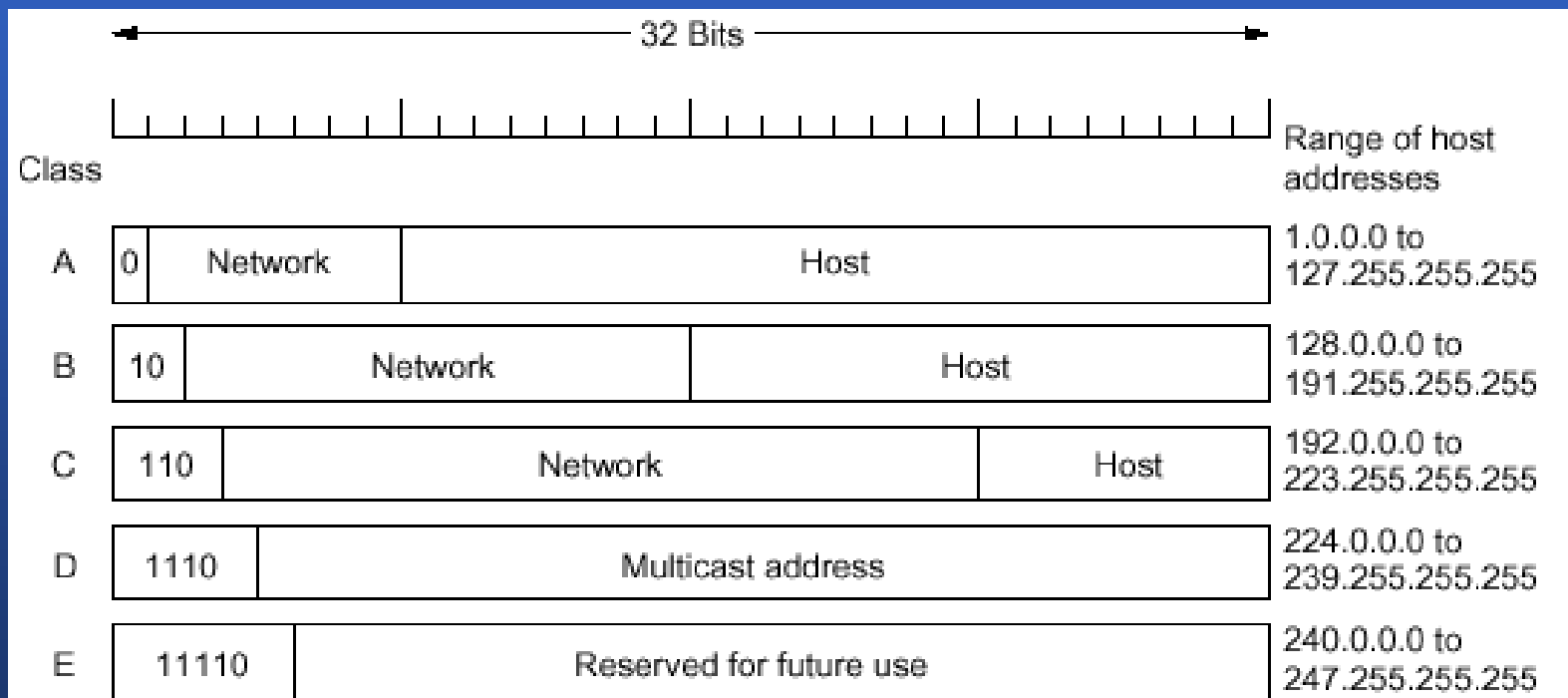
- 可选项-[see fig 5-46]

| Option                | Description                                        |
|-----------------------|----------------------------------------------------|
| Security              | Specifies how secret the datagram is               |
| Strict source routing | Gives the complete path to be followed             |
| Loose source routing  | Gives a list of routers not to be missed           |
| Record route          | Makes each router append its IP address            |
| Timestamp             | Makes each router append its address and timestamp |

**Fig. 5-46.** IP options.

## 5.6.2 IP Addresses

- 包括:网络号和主机号
- 地址类型: A、B、C、D和E
- 地址格式-[see fig 5-47]



**Fig. 5-47.** IP address formats.

# IP地址

- Special IP addresses.

|                                                                 |  |  |  |  |  |  |  |            |  |  |  |  |  |  |  |         |  |  |  |  |  |  |  |     |  |  |  |  |  |  |  |                                |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |                        |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |                                |
|-----------------------------------------------------------------|--|--|--|--|--|--|--|------------|--|--|--|--|--|--|--|---------|--|--|--|--|--|--|--|-----|--|--|--|--|--|--|--|--------------------------------|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|------------------------|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--------------------------------|
| 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 |  |  |  |  |  |  |  |            |  |  |  |  |  |  |  |         |  |  |  |  |  |  |  |     |  |  |  |  |  |  |  | This host                      |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |                        |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |                                |
|                                                                 |  |  |  |  |  |  |  |            |  |  |  |  |  |  |  |         |  |  |  |  |  |  |  |     |  |  |  |  |  |  |  |                                |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |                        |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |                                |
| 0 0                                                             |  |  |  |  |  |  |  | ...        |  |  |  |  |  |  |  |         |  |  |  |  |  |  |  | 0 0 |  |  |  |  |  |  |  | Host                           |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  | A host on this network |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |                                |
|                                                                 |  |  |  |  |  |  |  |            |  |  |  |  |  |  |  |         |  |  |  |  |  |  |  |     |  |  |  |  |  |  |  |                                |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |                        |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |                                |
| 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 |  |  |  |  |  |  |  |            |  |  |  |  |  |  |  |         |  |  |  |  |  |  |  |     |  |  |  |  |  |  |  | Broadcast on the local network |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |                        |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |                                |
|                                                                 |  |  |  |  |  |  |  |            |  |  |  |  |  |  |  |         |  |  |  |  |  |  |  |     |  |  |  |  |  |  |  |                                |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |                        |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |                                |
| Network                                                         |  |  |  |  |  |  |  |            |  |  |  |  |  |  |  | 1 1 1 1 |  |  |  |  |  |  |  |     |  |  |  |  |  |  |  | ...                            |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  | 1 1 1 1                |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  | Broadcast on a distant network |
|                                                                 |  |  |  |  |  |  |  |            |  |  |  |  |  |  |  |         |  |  |  |  |  |  |  |     |  |  |  |  |  |  |  |                                |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |                        |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |                                |
| 127                                                             |  |  |  |  |  |  |  | (Anything) |  |  |  |  |  |  |  |         |  |  |  |  |  |  |  |     |  |  |  |  |  |  |  | Loopback                       |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |                        |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |                                |

# IP地址

- IP地址是逻辑地址（不是物理地址）
- 32bits
- 包含网络号和主机号
- 每台主机都必须有唯一的地址
- IP地址必须统一分配
- 两个重要特性：
  - 每台主机分配了一个唯一的地址
  - 网络表示号的分配必须全球统一，但主机号可由本地分配，不需全球一致。



# IP地址的格式

- 在Ipv4中，IP地址由四个八位域（叫作octets）组成。Octets被点号分开代表在0到达255范围内的十进制数字。用二进制格式时共有32位组成，为了方便记忆，用点号每八位一分割，称为点分十进制。

如：dotted decimal notation: 202.96.96.68

二进制: 11001010. 01100000. 01100000. 01000100

- 从理论上计算全部32位都用上可以允许有超过四十亿的地址！这几乎可以为地球三分之二的人提供一个地址。但事实上，随着Internet的发展，可用的IP地址已经快要吃完了。
- 在将来的Ipv6中，IP地址由十六个八位域组成，共128位二进制形式的IP地址组成，还是用点号每八位一分割，在现在看来是足够了，但不知道还会有什么意想不到的事情令IP地址又不够用了。



# IP地址的基本类型

- A类IP地址的第一段数字范围为1~127，每个A类地址可连接163877064台主机，Internet上有126个A类地址。
- B类IP地址的第一段数字范围为128~191，每个B类地址可连接64516台主机，Internet上有16256个B类地址。
- C类IP地址的第一段数字范围为192~223，每个C类地址可连接254台主机，Internet上有2054512个C类地址。
- D类IP地址的第一段数字范围为224~239，D类地址用作多目的的地信息的传输，作为备用。
- E类IP地址的第一段数字范围为240~254，E类地址保留，仅作为Internet的实验和开发之用。



# 广播地址和网络地址

- IP广播地址的主机号全为“1”。  
(191.22.255.255/16)
- IP广播不一定是真正的广播，它依赖于底层的硬件技术。
- 主机号全部为“0”的IP地址叫做网络地址，代表整个网络。 (191.22.0.0/16)
- NID不能为127





# 哪几个不是合法的主机IP地址

**A. 233.100.2.2**

**B. 120.1.0.0**

**C. 127.120.50.30**

**D. 131.107.256.60**

**E. 188.56.4.255**

**F. 200.18.65.255**

D类地址

loopback

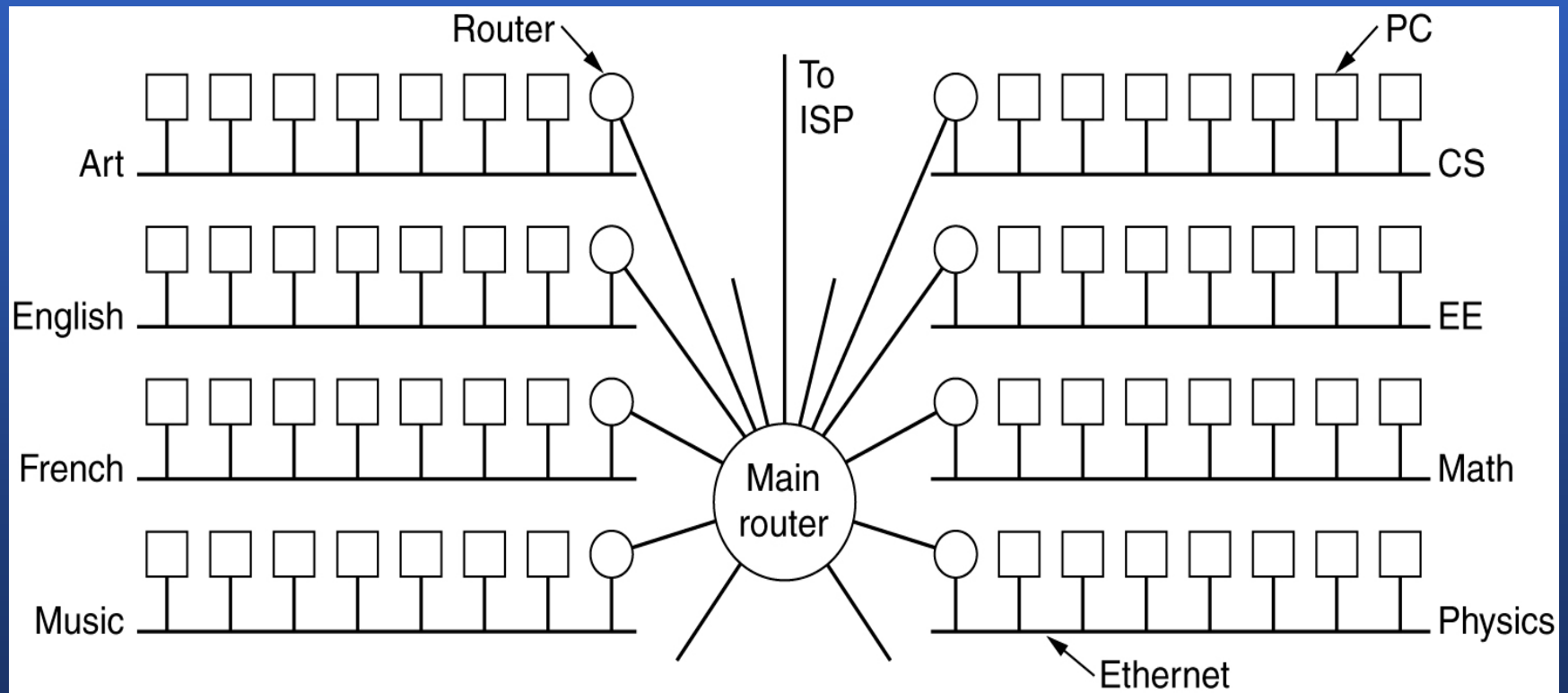
256非法

广播地址



# 子网(Subnets)

A campus network consisting of LANs for various departments.



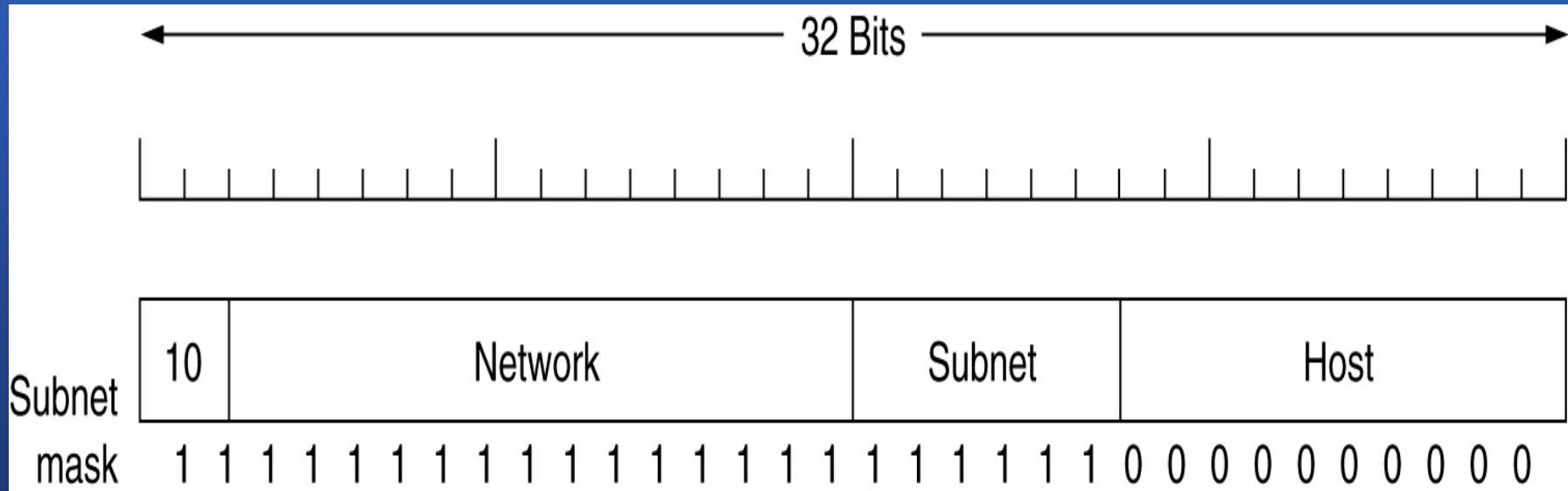
# 子网划分

- 为什么要进行子网划分？
  - 网间网规模的迅速扩展对IP地址模式的威胁并不是它不能保证主机地址的唯一性，而是会带来两方面的负担：
    - 第一，巨大的网络地址管理开销；
    - 第二，网关寻径急剧膨胀。其中第二点尤为突出，寻径表的膨胀不仅会降低网关寻径效率（甚至可能使寻径表溢出，从而造成寻径故障），更重要的是将增加内外部路径刷新时的开销，从而加重网络负担。



# Subnets (2)

- 便于管理
- 子网屏蔽



A class B network subnetted into 64 subnets.

# 子网掩码

- 子网掩码IP协议标准规定：
  - 每一个使用子网的网点都选择一个32位的位模式，若位模式中的某位置1，则对应IP地址中的某位为网络地址（包括网间网部分和物理网络号）中的一位；若位模式中的某位置0，则对应IP地址中的某位为主机地址中的一位。
- 为了使用的方便，常常使用“点分整数表示法”来表示一个IP地址和子网掩码，例如C类地址子网掩码（11111111 11111111 11111111 00000000）为：255.255.255.0
- 确定网络与主机地址
  - 将两个二进制数逻辑与（AND）运算后得出的结果即为网络部分
  - 将子网掩码取反再与IP地址逻辑与（AND）后得到的结果即为主机部分



# 子网划分技术

- 将要划分的子网数目转换为2的 $m$ 次方。如需要8 ( $2^3$ ) 个子网时,  $m=3$ ;
- 将上一步确定的幂 $m$ 按高序占用主机地址 $m$ 位后转换为十进制, 得到子网掩码。

**11100000**  **224**

  
3

# A类地址子网掩码

| 子网数目 占用位数 |   | 子网掩码           | 子网中主机数    |
|-----------|---|----------------|-----------|
| 2         | 1 | 255. 128. 0. 0 | 8,388,606 |
| 4         | 2 | 255. 192. 0. 0 | 4,194,302 |
| 8         | 3 | 255. 224. 0. 0 | 2,097,150 |
| 16        | 4 | 255. 240. 0. 0 | 1,048,574 |
| 32        | 5 | 255. 248. 0. 0 | 524,286   |
| 64        | 6 | 255. 252. 0. 0 | 262,142   |
| 128       | 7 | 255. 254. 0. 0 | 131,070   |
| 256       | 8 | 255. 255. 0. 0 | 65,534    |





# B类地址子网掩码

| 子网数目 占用位数 |   | 子网掩码             | 子网中主机数 |
|-----------|---|------------------|--------|
| 2         | 1 | 255. 255. 128. 0 | 32,766 |
| 4         | 2 | 255. 255. 192. 0 | 16,382 |
| 8         | 3 | 255. 255. 224. 0 | 8,190  |
| 16        | 4 | 255. 255. 240. 0 | 4,094  |
| 32        | 5 | 255. 255. 248. 0 | 2,046  |
| 64        | 6 | 255. 255. 252. 0 | 1,022  |
| 128       | 7 | 255. 255. 254. 0 | 510    |
| 256       | 8 | 255. 255. 255. 0 | 254    |



# C类地址子网掩码

| 子网数目 占用位数 |   | 子网掩码               | 子网中主机数 |
|-----------|---|--------------------|--------|
| 2         | 1 | 255. 255. 255. 128 | 126    |
| 4         | 2 | 255. 255. 255. 192 | 62     |
| 8         | 3 | 255. 255. 255. 224 | 30     |
| 16        | 4 | 255. 255. 255. 240 | 14     |
| 32        | 5 | 255. 255. 255. 248 | 6      |
| 64        | 6 | 255. 255. 255. 252 | 2      |



# CIDR – Classless InterDomain Routing

- 问题
  - B类地址对于大多数机构来说太大了
  - IP地址很快就会被用光
  - 路由选择表暴涨
  - 路由选择问题通过增加级别解决
- 解决方案:无类域间路由CIDR(Classless InterDomain Routing)
  - 基本思想
    - 以可变长分块的方式分配所剩的C 类网络
    - C 类地址分配规则:欧洲 北美 中美南美 亚洲太平洋地区
    - 分组到达欧洲,要更详细的路由选择表:将目的地址屏蔽,再与其中表目比较,以找到相匹配的地址.



# CIDR – Classless InterDomain Routing

A set of IP address assignments.

| University  | First address | Last address  | How many | Written as     |
|-------------|---------------|---------------|----------|----------------|
| Cambridge   | 194.24.0.0    | 194.24.7.255  | 2048     | 194.24.0.0/21  |
| Edinburgh   | 194.24.8.0    | 194.24.11.255 | 1024     | 194.24.8.0/22  |
| (Available) | 194.24.12.0   | 194.24.15.255 | 1024     | 194.24.12/22   |
| Oxford      | 194.24.16.0   | 194.24.31.255 | 4096     | 194.24.16.0/20 |



# CIDR

**试题：** A router has the following (CIDR) entries in its routing table:

| <u>Address/mask</u> | <u>Next hop</u> |
|---------------------|-----------------|
| 135.46.56.0/22      | Interface 0     |
| 135.46.60.0/22      | Interface 1     |
| 192.53.40.0/23      | router 1        |
| default             | router 2        |

**for each of the following IP addresses, which one does the router select for the next hop if a packet with that address arrives? (5 bonus)**

- (1) 135.46.63.10    (2) 135.46.57.14    (3) 135.46.52.2  
(4) 192.53.40.7    (5) 192.53.56.7



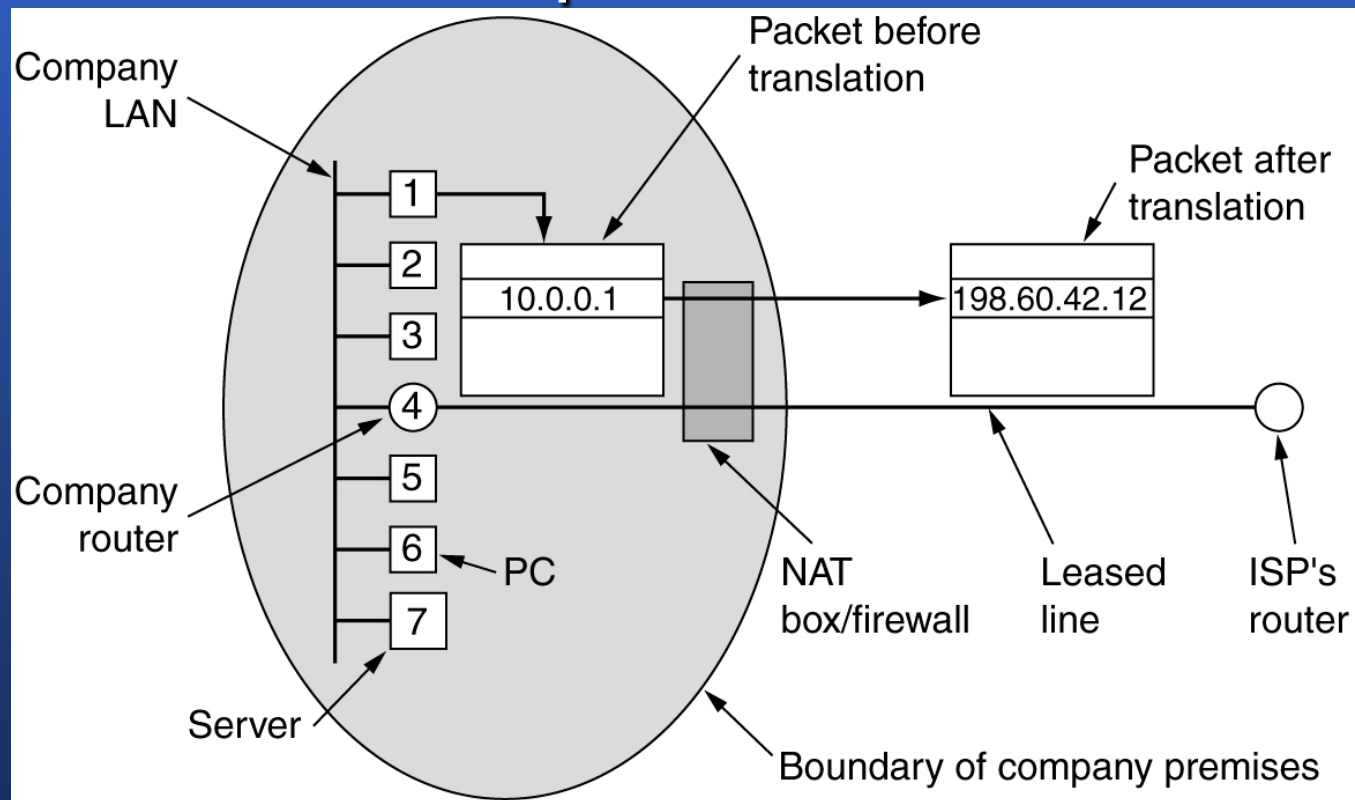
# NAT – Network Address Translation

- IP address are scarce
- Dynamically assign an IP address
- Three reserved ranges
  - 10.0.0.0 – 10.255.255.255/8 (16,777,216 hosts)
  - 172.16.0.0 – 172.31.255.255/12 (1,048,576 hosts)
  - 192.168.0.0 – 192.168.255.255/16 (65,536 hosts)



# NAT – Network Address Translation

## Placement and operation of a NAT box.





# NAT – Network Address Translation

- Source port & destination port of UDP or TCP
- Source port field is used in NAT to solve mapping system
- Just like main telephone number
  - Main number  $\leftrightarrow$  a company's IP address
  - Extensions  $\leftrightarrow$  ports of an extra 16-bits field to identify which process gets which incoming packet
- NAT can be used to alleviate the IP shortage for ADSL and cable users



# NAT – Network Address Translation

- Problems of NAT:
  - NAT violate the architectural model of IP
  - NAT changes the internet from a connectionless network to a kind of connection-oriented network
  - NAT violates the most fundamental rule of protocol layering
  - NAT box will cause the application to fail as the NAT box will not be able to locate the TCP source port correctly
  - Some applications insert IP address in the body of the text, the remote side may doesn't understand the address
  - As TCP source port field is 16 bits, at most 65,536 machines can be mapped onto an IP address



## 5.6.3 因特网控制协议

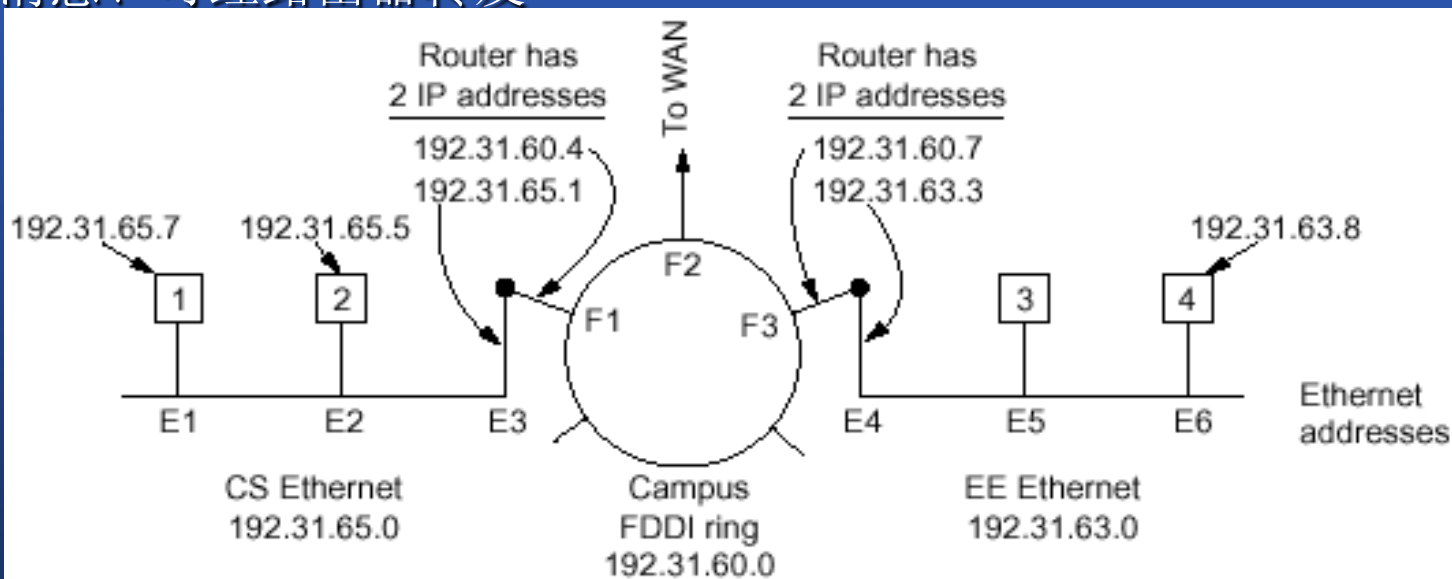
- 其它网络层控制协议：
  - ICMP、ARP、RARP、BOOTP、DHCP
- 因特网控制消息协议(ICMP)

| Message type            | Description                              |
|-------------------------|------------------------------------------|
| Destination unreachable | Packet could not be delivered            |
| Time exceeded           | Time to live field hit 0                 |
| Parameter problem       | Invalid header field                     |
| Source quench           | Choke packet                             |
| Redirect                | Teach a router about geography           |
| Echo request            | Ask a machine if it is alive             |
| Echo reply              | Yes, I am alive                          |
| Timestamp request       | Same as Echo request, but with timestamp |
| Timestamp reply         | Same as Echo reply, but with timestamp   |

**Fig. 5-50.** The principal ICMP message types.

# 因特网控制协议

- 地址分辨（解析）协议(A R P)
  - IP -> MAC
- 反向地址分辨协议(RARP)
  - MAC -> IP
- BOOTP(bootstrap)
  - UDP消息、可经路由器转发
- DHCP



**Fig. 5-51.** Three interconnected class C networks: two Ethernets and an FDDI ring.

# ARP & RARP

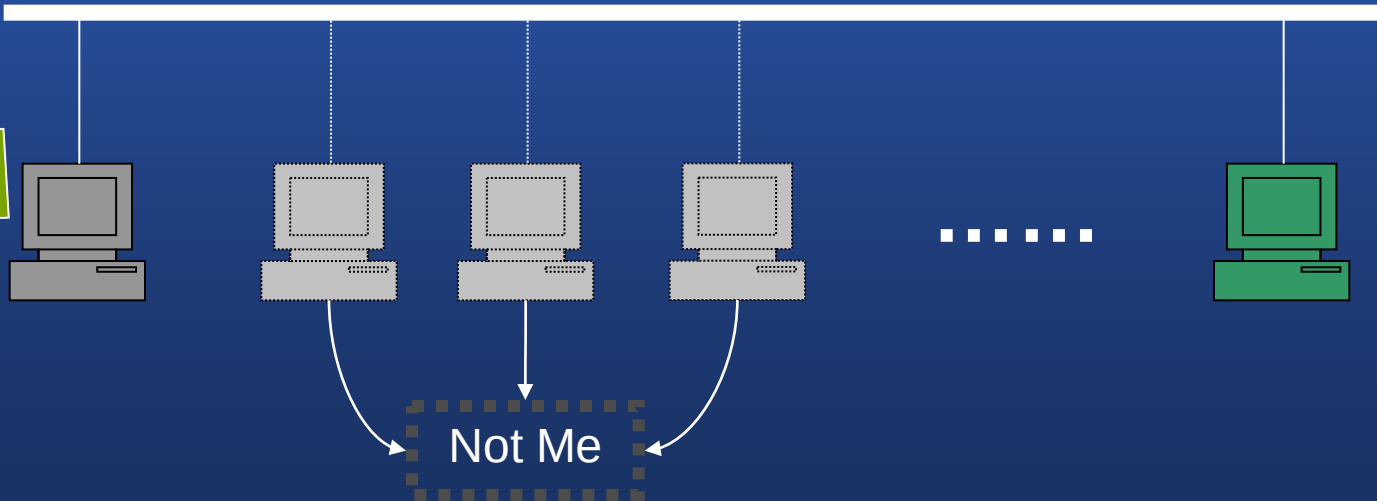
- ARP: 地址解析协议 (Address Resolution Protocol) 在已知目的IP地址, 需要知道目的硬件地址时使用。
- RARP: 由已知硬件地址查找IP地址的过程叫做反向地址解析 (Reverse Address Resolution)。
- ARP、RARP都是广播协议——网络上的每一台机器都能收到请求。
- 每一台机器都检查请求的IP或Ethernet Address, 符合要求的主机回答请求。



# ARP

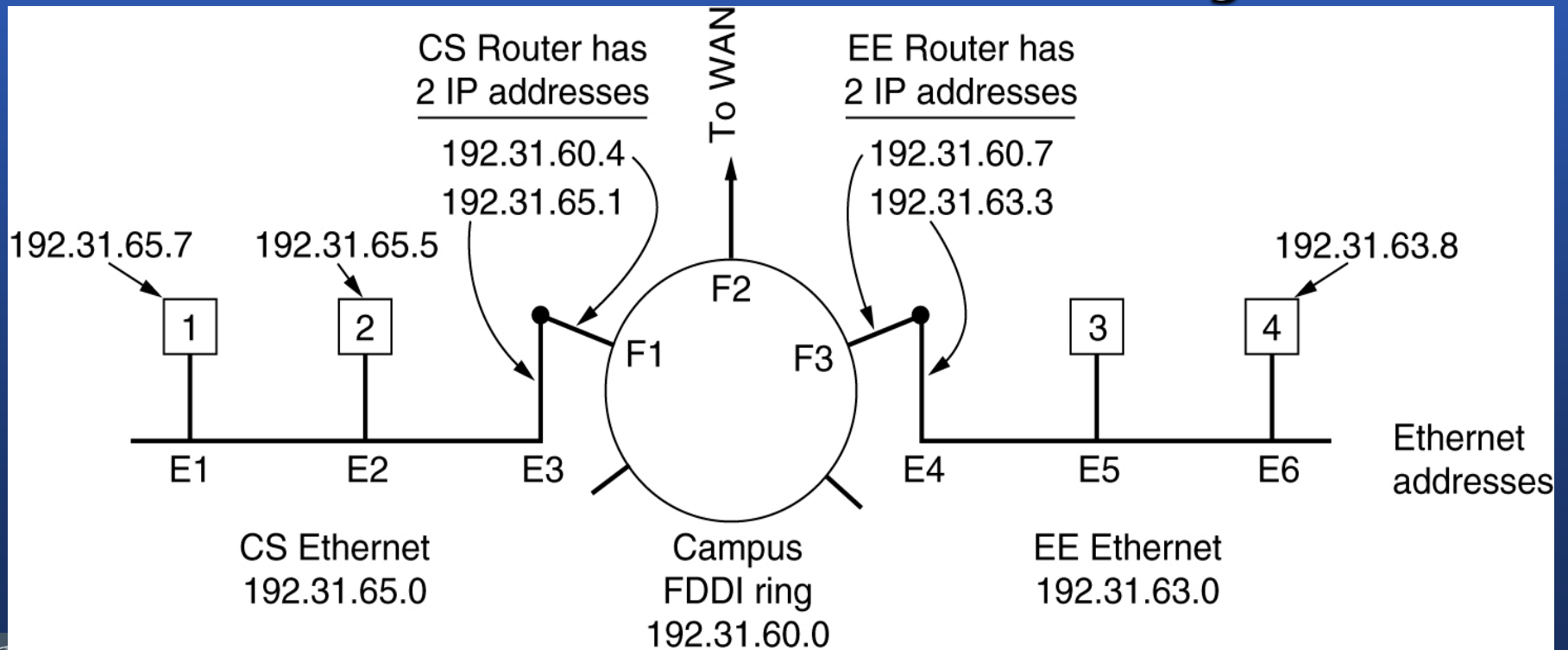
Hi guys here, listen please, Will 192.168.12.3  
tell me his ethernet address?

Oh, it's me. Red guy, my ethernet address is  
**00-50-BA-22-34-CC**



# ARP– The Address Resolution Protocol

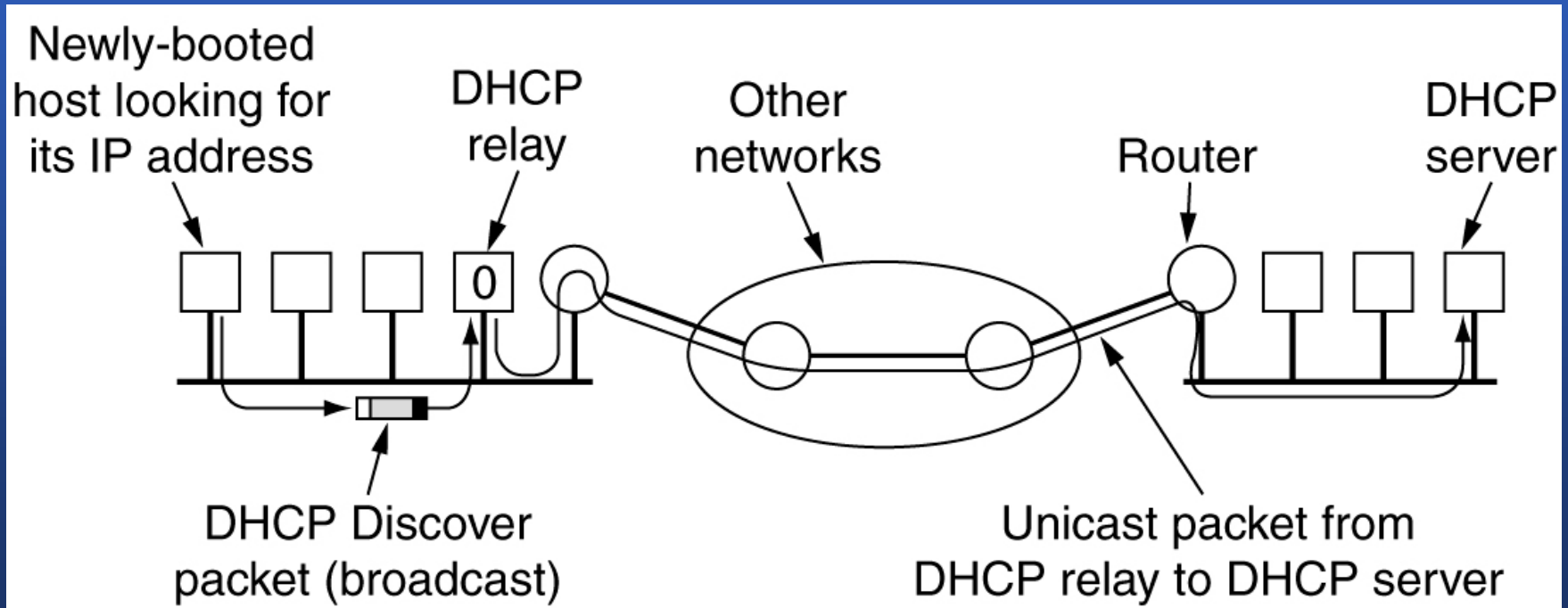
Three interconnected /24 networks: two Ethernets and an FDDI ring.





# Dynamic Host Configuration Protocol

## Operation of DHCP.

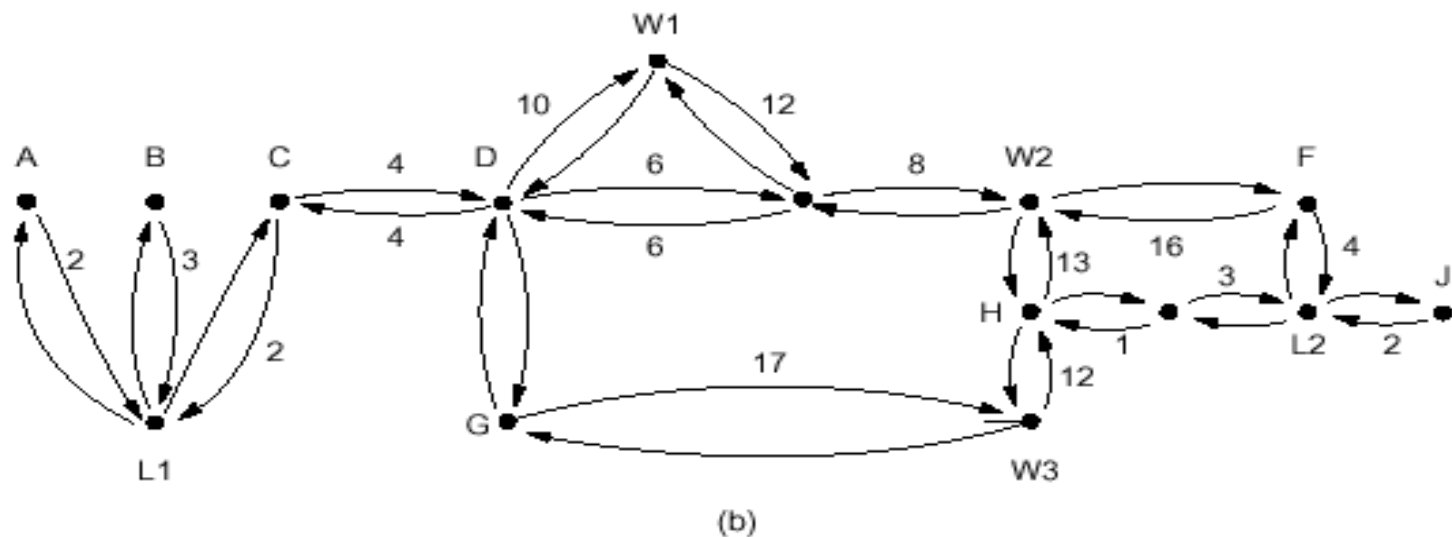
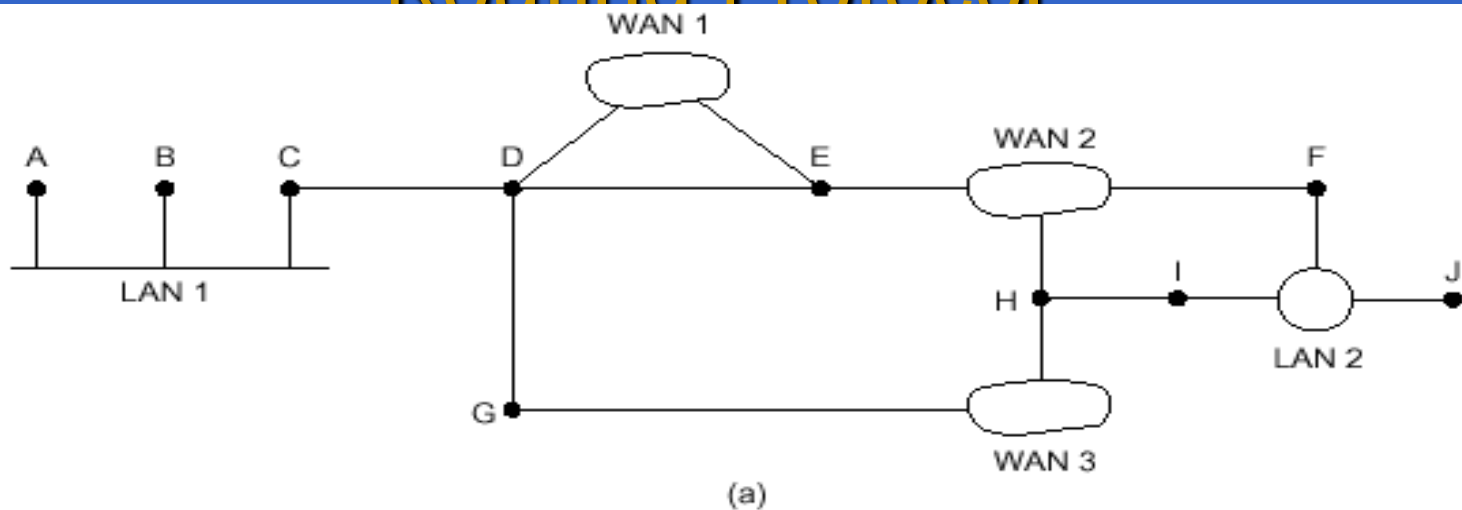


## 5.6.4 OSPF – The Interior Gateway Routing Protocol

- 内部网关协议/外部网关协议
- 开放最短路径优先(OSPF)
  - 工作原理-[see fig 5-52]
    - 支持3种类型的连接和网络
      - 点-点、LAN、WAN
      - 多路访问
    - OSPF所做的工作，就是用图形代表实际的网络，并计算从每个路由器到其他路由器的最佳路径



## 5.6.4 OSPF – The Interior Gateway Routing Protocol



**Fig. 5-52.** (a) An autonomous system. (b) A graph representation of (a).

## 5.6.4 OSPF – The Interior Gateway Routing Protocol

### – OSPF工作原理

- 管理AS、区域、主干（区域0）
- 3种可能的路由：
  - 区域内
  - 区域间
  - AS间
- 要区分4类路由器

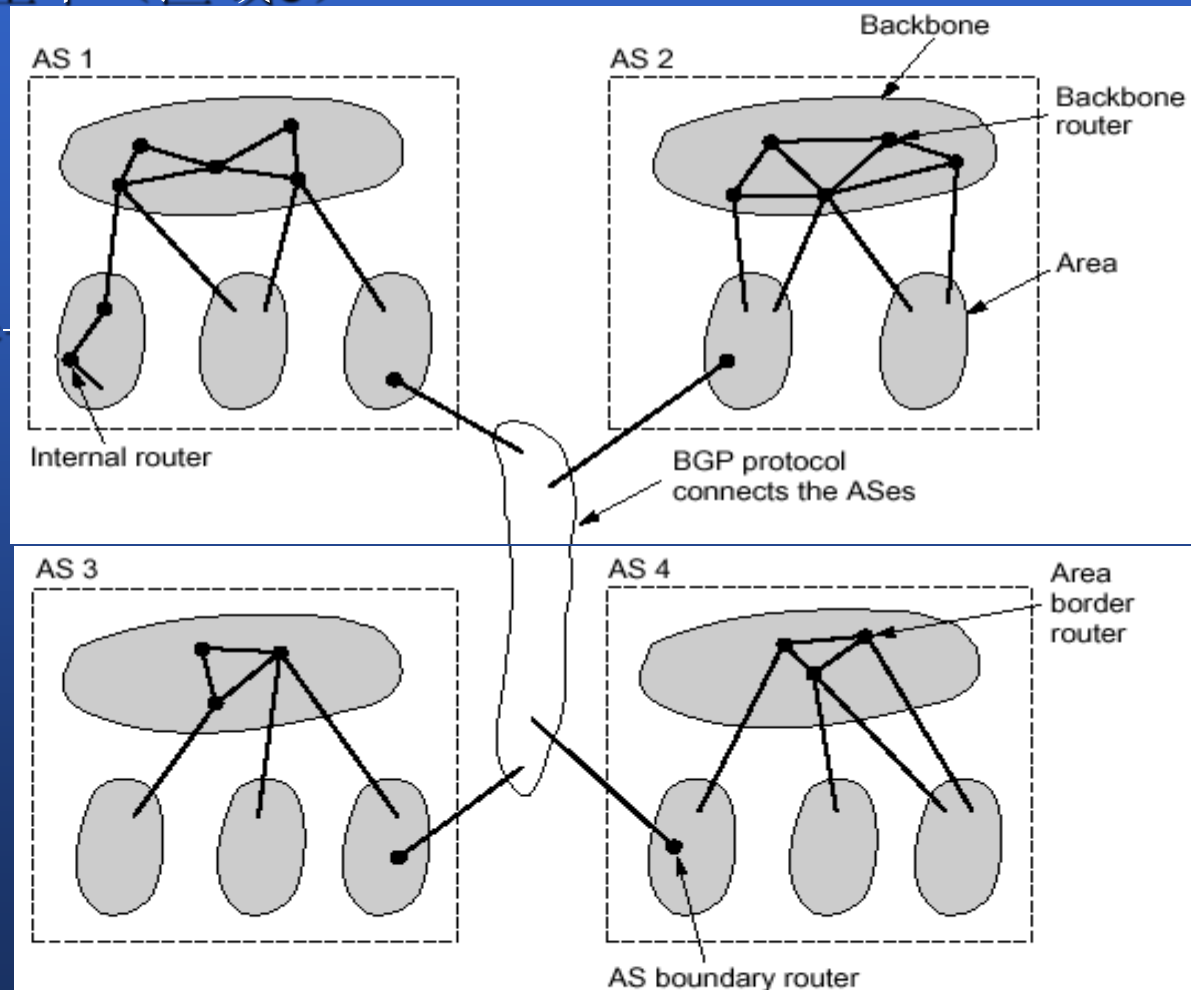


Fig. 5-53. The relation between ASes, backbones, and areas in OSPF.

## 5.6.4 OSPF – The Interior Gateway Routing Protocol

### – OSPF工作原理

- 与邻接路由器通过交换链接状态信息来更新路由
- 5类OSPF消息-[see fig 5-54]

| Message type         | Description                                  |
|----------------------|----------------------------------------------|
| Hello                | Used to discover who the neighbors are       |
| Link state update    | Provides the sender's costs to its neighbors |
| Link state ack       | Acknowledges link state update               |
| Database description | Announces which updates the sender has       |
| Link state request   | Requests information from the partner        |

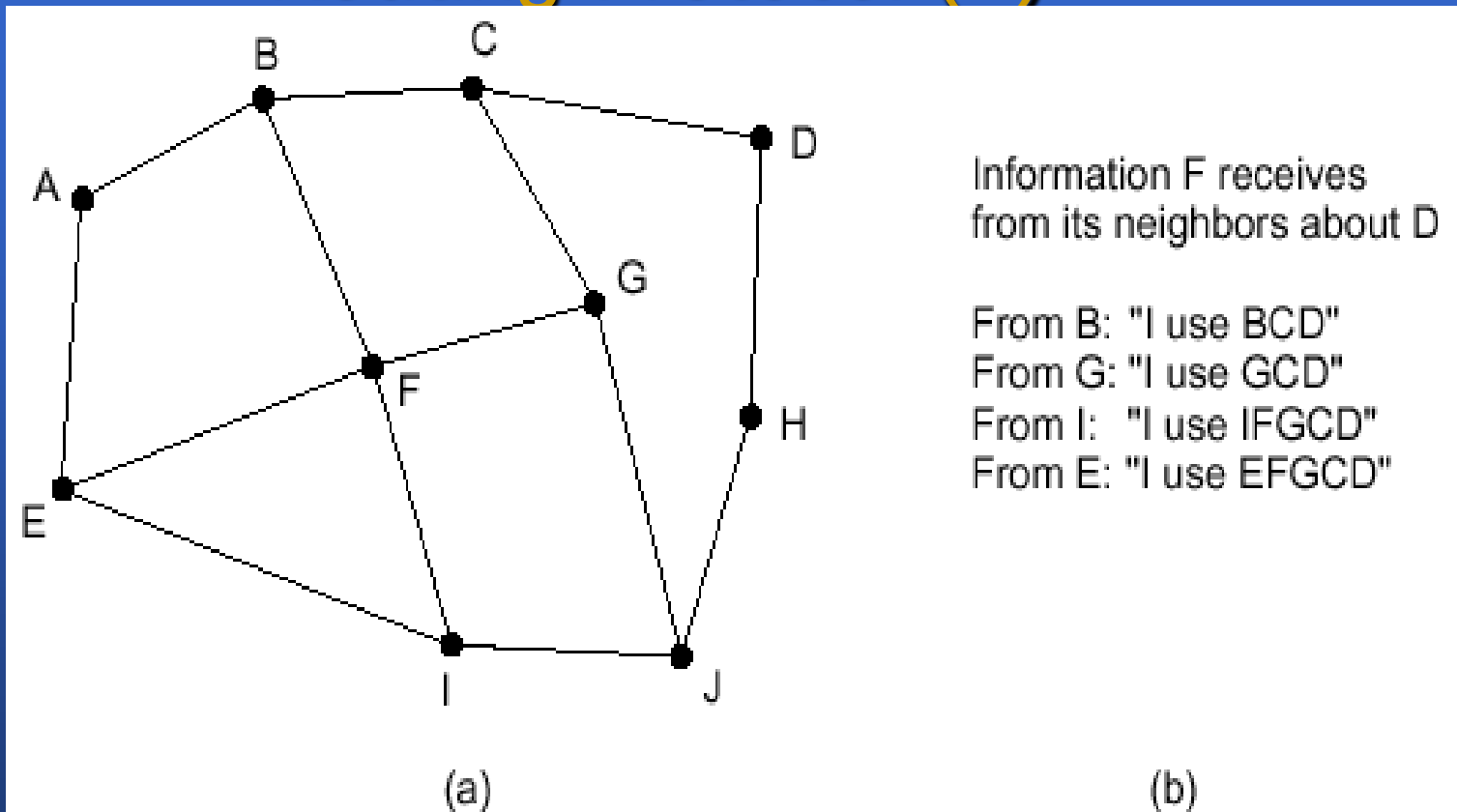
**Fig. 5-54.** The five types of OSPF messages.

## 5.6.5 BGP – The Exterior Gateway Routing Protocol (\*)

- 边界网关协议:用于AS之间
- 必须大量考虑策略
  - 路由限制的例子
- 策略由手工配置
- 从BGP观点,网络分三类:
  - 支线网络
  - 多连接网络
  - 中转网络
- 属于距离矢量协议
  - 记录使用确切路由-[see fig 5-55]



## 5.6.5 BGP – The Exterior Gateway Routing Protocol (\*)



**Fig. 5-55.** (a) A set of BGP routers. (b) Information sent to *F*.



## 5.6.6 Internet Multicasting

- 多点播送:同时向大量接收者发送信息
- IP采用D类地址来支持多点播送
- 因特网支持两类组地址:
  - 永久地址:总是存在,不必创建
  - 临时地址:使用前先创建
- 由特殊的多点播送路由器实现
- 因特网组管理协议IGMP :
  - 只有两种分组: 询问和响应
- 路由选择通过生成树实现



## 5.6.7 Mobile IP (\*)

- 实现IP地址系统的异地工作的最大障碍:  
编制方案本身
  - 给新位置要通知大量人,程序及数据库,不可取
  - 利用完整IP地址作路由选择,开销太大
- 解决方案
  - 目标(5个)
  - 创建一个原地代理及外地代理
  - 给出关照地址
  - 无偿A R P技术
- 移动主机IETF解决方案解决的另一一些问题
  - 代理如何定位?广而告之
  - 如何处理走时不说再见的移动主机?登录只在固定时间间隔内有效
  - 安全性问题?证明请求的源,加密身份验证.



## 5.6.8 IPv6

- 增强的简单的因特网协议SIPP,即IPv6  
(Simple Internet Protocol Plus)
- 特性
  - 更长的地址
  - 对头部的简化
  - 对选项的更好支持
  - 安全性
  - 在服务类型上比以前集中了更多的注意力

## 5.6.8 IPv6

- 主要的IPv6头部-[see fig 5-56]
  - 固定的头部构成
  - 任意播送

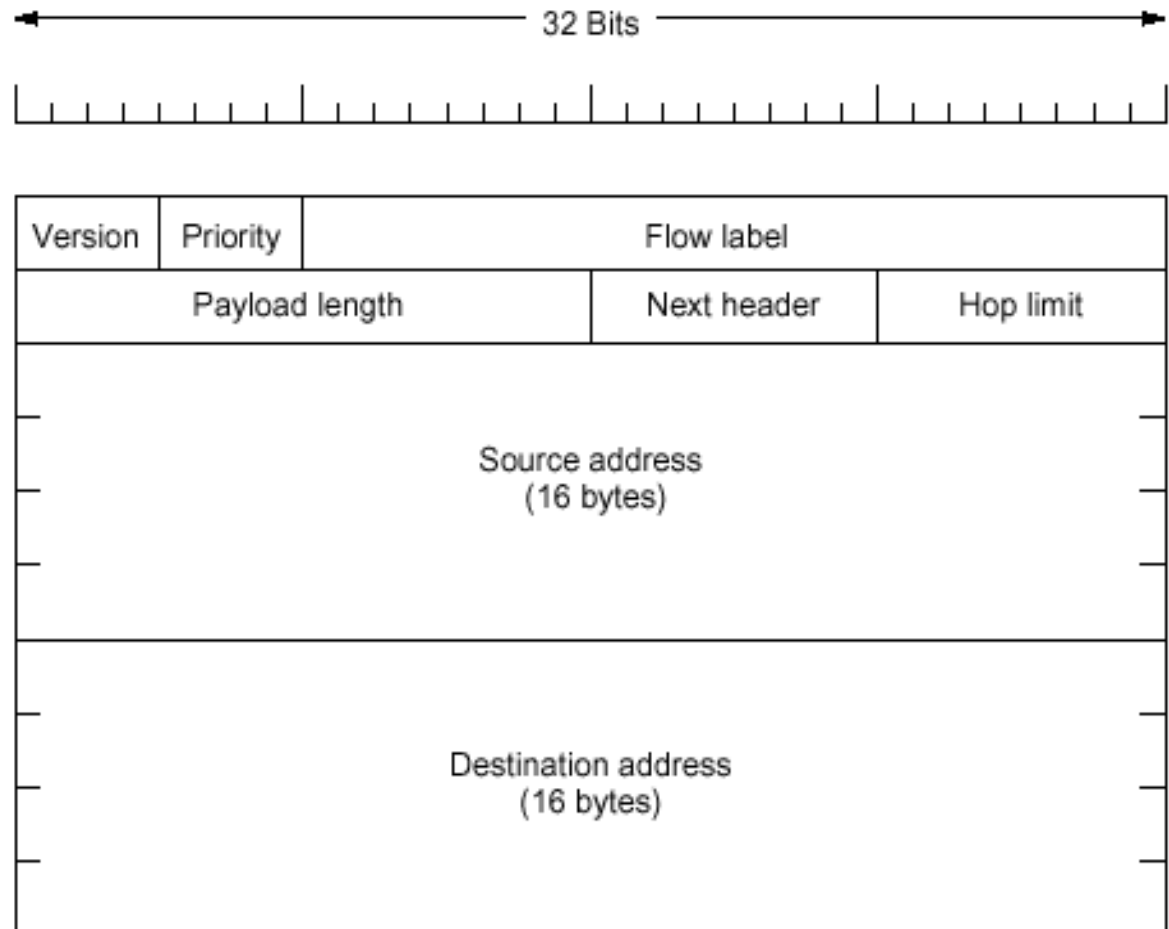


Fig. 5-56. The IPv6 fixed header (required).

# 5.6.8 IPv6

- IP地址空间的切分

| Prefix (binary) | Usage                        | Fraction |
|-----------------|------------------------------|----------|
| 0000 0000       | Reserved (including IPv4)    | 1/256    |
| 0000 0001       | Unassigned                   | 1/256    |
| 0000 001        | OSI NSAP addresses           | 1/128    |
| 0000 010        | Novell NetWare IPX addresses | 1/128    |
| 0000 011        | Unassigned                   | 1/128    |
| 0000 1          | Unassigned                   | 1/32     |
| 0001            | Unassigned                   | 1/16     |
| 001             | Unassigned                   | 1/8      |
| 010             | Provider-based addresses     | 1/8      |
| 011             | Unassigned                   | 1/8      |
| 100             | Geographic-based addresses   | 1/8      |
| 101             | Unassigned                   | 1/8      |
| 110             | Unassigned                   | 1/8      |
| 1110            | Unassigned                   | 1/16     |
| 1110            | Unassigned                   | 1/16     |
| 1111 0          | Unassigned                   | 1/32     |
| 1111 10         | Unassigned                   | 1/64     |
| 1111 110        | Unassigned                   | 1/128    |
| 1111 1110 0     | Unassigned                   | 1/512    |
| 1111 1110 10    | Link local use addresses     | 1/1024   |
| 1111 1110 11    | Site local use addresses     | 1/1024   |
| 1111 1111       | Multicast                    | 1/256    |



# 5.6.8 IPv6

- 扩展头部-[see fig 5-58]
  - 提供额外信息,以有效方式编码
  - 紧跟固定头部之后,按次序排序

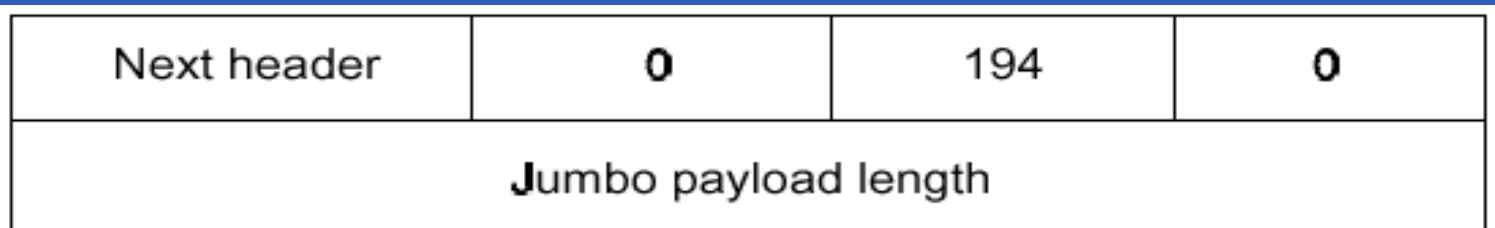
| Extension header           | Description                                |
|----------------------------|--------------------------------------------|
| Hop-by-hop options         | Miscellaneous information for routers      |
| Routing                    | Full or partial route to follow            |
| Fragmentation              | Management of datagram fragments           |
| Authentication             | Verification of the sender's identity      |
| Encrypted security payload | Information about the encrypted contents   |
| Destination options        | Additional information for the destination |

**Fig. 5-58.** IPv6 extension headers.

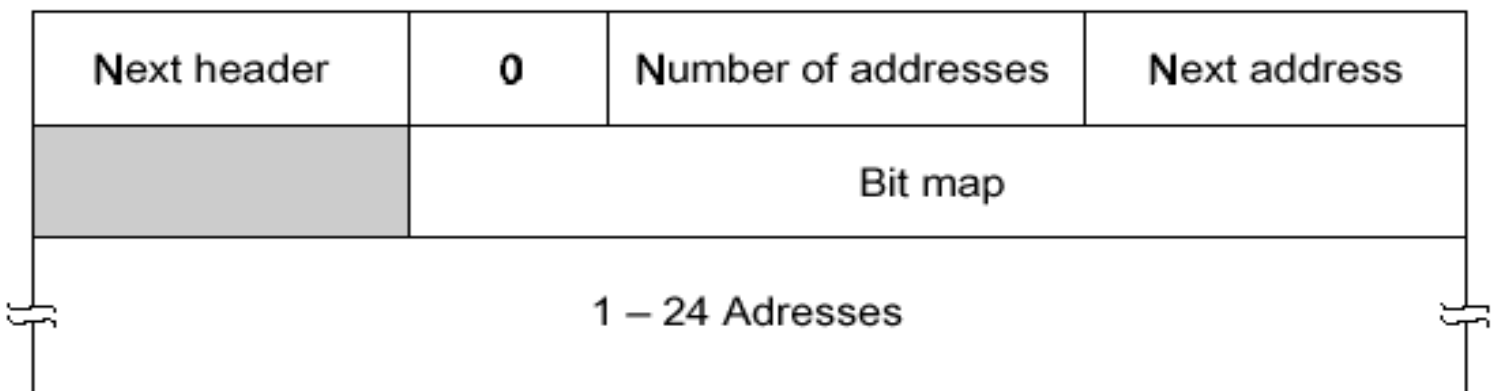
# 5.6.8 IPv6

## – 扩展头部

- 站接站扩展头部-[see fig 5-59]
- 路由选择的扩展头部-[see fig 5-60]



**Fig. 5-59.** The hop-by-hop extension header for large datagrams (jumbograms).



**Fig. 5-60.** The extension header for routing.



## 5.6.8 IPv6

### — 扩展头部

- 段头部
- 身份认证头部
- 加密的安全性有效载荷
- 目的地选项

### — 一些争议

- 地址长度、站段限制长度、最大分组长度、取消IPv4校验和、安全性



# 习题

– 1、 6、 17、 22、 23、 27-30、 36

– 9, 34, 38, 43

